

Gym Tracker V1

Complete Technical Guide

A first-principles walkthrough of a full-stack web application
— from the browser down to SQLite

Repository: github.com/Sumit-0610/gym-tracker

Frozen at commit `0d2eb73` · branch `main`

Stack: React 19 + Vite · Express 4 · `node:sqlite` · nginx · Android/Termux

Status: V1 complete, frozen, and deployed & verified on-device (2026-09-06)

Generated from the actual source tree — the code is authoritative

How to read this guide

This document is generated **from the repository**, not from memory. Where a Markdown doc in the repo and the source code disagreed, the code won. Everything is tagged so you always know what kind of claim you are reading:

Tag	Meaning
IMPLEMENTED	present in the source at commit <code>0d2eb73</code>
VERIFIED	additionally exercised by a test or an observed run
DEFERRED	named in <code>V2-BACKLOG.md</code> ; not built
NOT VERIFIED	true in code but not independently confirmed

Audience. You can program a little and want to learn full-stack development deeply. Terms are defined the first time they matter. The glossary (Chapter 44) is the fallback.

The one question this guide answers: *what actually happens between "I tap a button" and "I see the result on screen"?* Chapters 41–43 trace that chain end to end; the earlier chapters build up the pieces.

Table of Contents

How to read this guide

1. Project overview
 - 1.1 What the application does
 - 1.2 Why it exists
 - 1.3 V1 scope IMPLEMENTED
 - 1.4 Intended scale
 - 1.5 Deployment target VERIFIED
 - 1.6 Learning objectives
 - 1.7 Why a self-hosted SQLite application is the right choice here
2. Complete system architecture
 - 2.1 The parts
 - 2.2 Development vs production — and why they differ
 - 2.3 The request path, in one sentence each
3. Repository structure
 - 3.1 The tree (source files only; node_modules, dist, and the DB are excluded)
 - 3.2 What calls what (backend)
 - 3.3 What calls what (frontend)
 - 3.4 Files that exist but carry little logic
4. Backend runtime
 - 4.1 The vocabulary
 - 4.2 Server startup, line by line — server/src/index.js
 - 4.3 How a request travels through Express
 - 4.4 The response lifecycle
 - 4.5 Error handling
5. Express middleware pipeline
 - 5.1 The pipeline for this app
 - 5.2 What "middleware" means and why order matters
 - 5.3 req, res, next in this codebase
6. Authentication
 - 6.1 First principles
 - 6.2 Signup — POST /api/signup IMPLEMENTED
 - 6.3 Password hashing — bcrypt / bcryptjs
 - 6.4 Login — POST /api/login IMPLEMENTED
 - 6.5 Sessions and cookies
 - 6.6 Logout — POST /api/logout IMPLEMENTED
 - 6.7 GET /api/me — the "who am I" endpoint IMPLEMENTED
 - 6.8 Sequence diagrams
 - 6.9 Authentication vs authorization — and why sessions here, not JWT
7. Authorization / ownership security
 - 7.1 The threat: IDOR
 - 7.2 The fix used everywhere: filter by owner in the SQL

- 7.3 Why this beats "fetch, then check in JavaScript"
- 7.4 404, not 403 — existence disclosure
- 7.5 Frontend guards are not security
- 7.6 The two-user model, verified
- 8. Database architecture
 - 8.1 SQLite from first principles
 - 8.2 Keys, constraints, types
 - 8.3 Foreign keys
 - 8.4 Transactions
 - 8.5 Parameterized queries
- 9. Database schema
 - 9.1 users
 - 9.2 exercises
 - 9.3 routines
 - 9.4 routine_exercises
 - 9.5 workouts
 - 9.6 workout_sets
 - 9.7 Entity-Relationship diagram
 - 9.8 Planned data vs actual data — the key conceptual split
 - 9.9 Indexes — what exists and what doesn't
- 10. SQL walkthrough
 - 10.1 GET /api/exercises — the simplest query
 - 10.2 Routine ownership + list
 - 10.3 Routine detail — a JOIN across the link table
 - 10.4 INNER JOIN vs LEFT JOIN
 - 10.5 Workout history — LEFT JOIN + GROUP BY + COUNT in one round trip
 - 10.6 The N+1 query problem — and how §10.5 avoids it
 - 10.7 Workout detail — two queries, second JOINS exercise names
 - 10.8 The write queries
- 11. API architecture — the complete reference
 - 11.1 Authentication
 - POST /api/signup VERIFIED
 - POST /api/login VERIFIED
 - POST /api/logout VERIFIED
 - GET /api/me VERIFIED
 - 11.2 Exercises
 - GET /api/exercises VERIFIED
 - 11.3 Routines
 - POST /api/routines VERIFIED
 - GET /api/routines VERIFIED
 - GET /api/routines/:id VERIFIED
 - POST /api/routines/:id/exercises VERIFIED
 - 11.4 Workouts — logging
 - POST /api/workouts VERIFIED
 - POST /api/workouts/:id/sets VERIFIED
 - 11.5 Workouts — history
 - GET /api/workouts VERIFIED
 - GET /api/workouts/:id VERIFIED
- 12. API request lifecycle
 - 12.1 Create a routine (a mutation)
 - 12.2 Log a workout set (a mutation with an ownership check)
 - 12.3 View history (a query)
- 13. Validation
 - 13.1 Two validations, two purposes
 - 13.2 The backend helpers — server/src/validation.js IMPLEMENTED
 - parseId(raw) → number | null
 - nonEmptyString(value, field, max = 100) → string | null
 - positiveInt(value, field) → string | null
 - optionalPositiveInt(value, field) → string | null

nonNegativeNumber(value, field) → string | null

13.3 The frontend validators

14. Frontend architecture

14.1 The stack, and what each piece is

14.2 Entry point — index.html → main.jsx → App.jsx

14.3 App.jsx — the shell

14.4 The dependency graph

14.5 Folder conventions

14.6 The design system — styles/tokens.css

15. React fundamentals, taught through this project

15.1 Components & JSX

15.2 Props

15.3 State — useState

15.4 Effects — useEffect

15.5 Memoised derived values — useMemo

15.6 Refs — useRef

15.7 Context

15.8 Controlled inputs

15.9 Rendering, re-rendering, conditional rendering, lists & keys

15.10 Derived data — the recurring lesson

16. State management

16.1 Server state

16.2 UI state

16.3 Derived state

17. The custom router — src/router.jsx

17.1 The problem it solves

17.2 The History API primitives

17.3 <Router>

17.4 matchPath(pattern, path) — route matching & params

17.5 The route table & how a param reaches a component — App.jsx

17.6 <Link> and <Redirect>

17.7 Browser Back / Forward / refresh — end to end

17.8 Why not react-router

17.9 Documentation note

18. The API client — src/api.js

18.1 ApiError

18.2 request(method, path, body)

18.3 HTTP error vs network error — where each originates

18.4 The global 401 handler

18.5 The named helpers

19. The useApi hook — src/hooks/useApi.js

19.1 What it is

19.2 The three (four) states every screen renders

19.3 The cancelled flag

19.4 Why fetcher is not in the dep array

20. The auth context — src/auth.jsx

20.1 The core idea: "the cookie is the source of truth"

20.2 The state machine

20.3 The two effects

20.4 login / signup / logout

20.5 useAuth

20.6 Refresh, precisely

21. Component-by-component walkthrough

21.1 Button.jsx

21.2 Input.jsx

21.3 Select.jsx

21.4 Card.jsx

21.5 Spinner.jsx

21.6 ErrorMessage.jsx

21.7 EmptyState.jsx

- 21.8 ExerciseSelect.jsx
- 21.9 Nav.jsx
- 22. Screen-by-screen walkthrough
 - 22.1 Login — /login (public)
 - 22.2 Signup — /signup (public)
 - 22.3 Dashboard — /
 - 22.4 Exercises — /exercises
 - 22.5 Routines — /routines
 - 22.6 Routine Detail — /routines/:id
 - 22.7 Workout (dispatcher) — /workout and /workout/:id
 - 22.8 Workout Start — /workout
 - 22.9 Workout Session — /workout/:id
 - 22.10 SetForm & SetList (workout building blocks)
 - 22.11 History — /history
 - 22.12 Workout Detail — /history/:id
 - 22.13 NotFound (inline in App.jsx)
 - 22.14 Desktop behaviour
- 23. The complete workout flow
 - 23.1 Routine workout — every layer
 - 23.2 Freestyle workout — the differences
 - 23.3 The relationship chain
- 24. Workout state & double submission
 - 24.1 The bug disabled={pending} alone does not fix
 - 24.2 The fix: a synchronous useRef guard
 - 24.3 Mutations vs GETs — why they're handled differently
 - 24.4 Intent vs confirmed server state
 - 24.5 Form reset policy
- 25. History flow
 - 25.1 GET /api/workouts → the list
 - 25.2 GET /api/workouts/:id → the detail
 - 25.3 Server data model vs frontend presentation model
 - 25.4 Dates — stored UTC, shown local
 - 25.5 Empty workout ≠ error
- 26. Error handling — browser to database
 - 26.1 Where each status originates
 - 26.2 Backend: the central handler
 - 26.3 Frontend: describeError — status → sentence
 - 26.4 The full chain for a 400
 - 26.5 The full chain for a 404 (cross-user access)
 - 26.6 Empty vs error — always distinguished
- 27. Security model
 - 27.1 Password storage
 - 27.2 Sessions
 - 27.3 httpOnly cookie
 - 27.4 SameSite=Lax
 - 27.5 No HTTPS (a documented concession)
 - 27.6 Parameterized SQL
 - 27.7 XSS protection
 - 27.8 Error disclosure
 - 27.9 Network exposure
 - 27.10 Secret management
 - 27.11 Login rate limiting
 - 27.12 .gitignore — what never reaches the repo
 - 27.13 Verified security tests
- 28. Deployment architecture
 - 28.1 The target
 - 28.2 Termux and why node:sqlite matters here
 - 28.3 Why Claude Code / any dev tooling is not on the phone
 - 28.4 On-device verification VERIFIED 2026-09-06
 - 28.5 Known operational limitation
- 29. nginx

- 29.1 What a reverse proxy is
- 29.2 The server block — `deploy/nginx-gym-tracker.conf`
- 29.3 Location matching — why `/api/me` must not return `index.html`
- 29.4 SPA fallback — why `/history/5` must return `index.html`
- 29.5 Proxy headers
- 29.6 The whole-file config — `deploy/nginx.conf.example`
- 29.7 Port 8080, no root
- 30. Startup & shutdown
 - 30.1 `deploy/start.sh`
 - 30.2 Keeping it running
 - 30.3 Shutdown / restart
 - 30.4 The `HOST` / `PORT` / `DB_PATH` env contract
- 31. Backups
 - 31.1 `deploy/backup.sh`
 - 31.2 Why SQLite can be backed up while the server runs
 - 31.3 Retention
 - 31.4 What to back up, and what not to
 - 31.5 Recovery
- 32. Health check
 - 32.1 `deploy/health-check.sh` — what each probe proves
 - 32.2 What it does not prove
 - 32.3 A "healthy" health check that was actually a bug
- 33. Testing architecture
 - 33.1 `server/test/smoke.sh` — 65 end-to-end API checks VERIFIED
 - 33.2 `E2E-CHECKLIST.md` — the manual browser pass
 - 33.3 There are no unit tests
- 34. Testing strategy
- 35. Real debugging examples
 - 35.1 Corrupt npm inside the nvm-installed Node
 - 35.2 `<Button>` silently dropped its CSS classes
 - 35.3 Mobile inputs ignored `background-color`
 - 35.4 `GET /api/me`'s 401 bounced logged-out users off `/signup`
 - 35.5 Native number-input validation blocked form submission
 - 35.6 Synchronous double-submit slipped past `disabled={pending}`
 - 35.7 Nav tab not highlighted on detail pages
 - 35.8 `NotFound` lost the nav and did a full reload
 - 35.9 nginx config hard-coded a pid path this build doesn't use VERIFIED (on phone)
 - 35.10 `health-check.sh` couldn't see a running nginx VERIFIED (on phone)
 - 35.11 `Node/node:sqlite` compatibility (a concern that was checked, not a bug)
- 36. Failure modes
 - 36.1 Node stopped
 - 36.2 nginx stopped
 - 36.3 Database unavailable / path unwritable at startup
 - 36.4 Port 3000 occupied
 - 36.5 nginx 502 Bad Gateway on `/api/...`
 - 36.6 SPA deep link returns nginx 404
 - 36.7 `/api/...` returns HTML instead of JSON
 - 36.8 Cookie doesn't stick (logged in, then instantly logged out)
 - 36.9 "I was logged out" after a restart
 - 36.10 Frontend build missing / `client/dist` stale
 - 36.11 Failure-mode summary table
- 37. Development workflow
 - 37.1 The safe loop
 - 37.2 Never commit
 - 37.3 Commit-message convention
- 38. Production update workflow
- 39. V1 design decisions
- 40. Tradeoffs
- 41. What happens for a real user
 - 41.1 "I open the app."

41.2	"I log in."
41.3	"I create a routine."
41.4	"I start a workout." / "I log a set." / "I view history."
41.5	"I log out."
42.	Request/response traces
42.1	Signup
42.2	Login
42.3	Protected GET (/api/routines)
42.4	Create routine
42.5	Add exercise to routine
42.6	Start workout (routine)
42.7	Log set
42.8	History list
42.9	Logout
42.10	Unauthorized request
43.	Data lifecycle
44.	Glossary
45.	Interview knowledge
	"Why React and not vanilla JS?"
	"Why SQLite and not Postgres?"
	"Why sessions and not JWT?"
	"How does authentication work here, end to end?"
	"How does authorization work? How do you stop IDOR?"
	"Explain a JOIN from this project."
	"What's your security posture — and what are its limits?"
	"Why nginx? What does it do?"
	"How is state managed on the frontend?"
	"You wrote your own router. Why, and how does it work?"
	"Tell me about a bug you fixed."
	"How did you test it? How did you deploy it?"
46.	How to extend the project (V2)
46.1	Persistent SQLite-backed session store DEFERRED
46.2	HTTPS DEFERRED
46.3	Login rate limiting DEFERRED
46.4	Workout completion state DEFERRED
46.5	Resume current workout DEFERRED
46.6	Previous performance DEFERRED
46.7	Set types DEFERRED
46.8	Rest timer DEFERRED
46.9	1RM (one-rep max) DEFERRED
46.10	Progress charts DEFERRED
46.11	Unit preference (kg/lb) DEFERRED
46.12	Editing / deletion DEFERRED
46.13	History pagination DEFERRED
46.14	The bigger goal: "reachable from anywhere"
47.	Learning roadmap
	Beginner — solidify the foundations
	Intermediate — the core stack
	Advanced — operations & scale
	Appendix A — Complete API table
	Appendix B — Database reference
	Appendix C — Important commands
	Development
	Testing
	Build (frontend)
	Deployment (on the phone, Termux)
	Backup / restart
	Update to a new version
	Git
	Appendix D — Environment variables
	Appendix E — Complete project map

1. Project overview

1.1 What the application does

Gym Tracker is a self-hosted web app for logging weight-training. A logged-in user can:

1. Browse a shared **exercise library** (21 seeded exercises, each with a muscle group).
2. Create **routines** — named templates like "Push Day" — and add exercises to them, optionally with target sets and reps.
3. **Start a workout**, either *from a routine or freestyle* (no routine).
4. **Log sets** during the workout: exercise, set number, reps, weight.
5. Review **history** — a list of past workouts, newest first — and open any one to see its sets grouped by exercise.

It is inspired by the commercial app *Hevy*, reduced to the core loop.

1.2 Why it exists

It is a **learning project**. The goal is not a product; it is to understand, concretely, how a full-stack application is wired: browser → HTTP → server → database → back again, and how authentication, authorization, state, and deployment actually work. Every design choice favours *clarity you can learn from over sophistication*.

1.3 V1 scope IMPLEMENTED

Included	Excluded from V1 (see Ch. 46 for the deferred list)
Username/password signup & login	Password reset, email, OAuth, "remember me" beyond the 7-day cookie
Session-cookie authentication	JWT / token auth
Exercise library (read-only)	Creating or editing exercises
Routines: create, view, add exercises	Renaming, deleting, reordering routines
Workouts: start (routine or freestyle), log sets	Editing or deleting a workout or a set; a "finish/complete" state
History: list + detail	Progress charts, personal records, 1-rep-max, "previous performance" hints, rest timer, set types
One shared SQLite database	Multi-tenant isolation beyond per-row <code>user_id</code> ; pagination

1.4 Intended scale

About 10 users. This number drives almost every decision: SQLite instead of a client/server database, an in-memory session store, a single Node process, no pagination, no caching layer. At 10 users these are correct simplifications, not shortcuts. At 10,000 users several would have to change (Ch. 40).

1.5 Deployment target VERIFIED

An old **Android phone running Termux** (a terminal-emulator app that provides a Linux userland without root). On it:

```
Phone browser → nginx :8080 → Express 127.0.0.1:3000 → SQLite file
                  serves the built
                  React files;
                  proxies /api/*
                  ~/gym-tracker-data/app.db
```

Verified on-device on 2026-09-06: Termux 0.119.0-beta.3, Node v26.4.0, nginx 1.31.5. Also reachable from other devices on the same Wi-Fi at <http://192.168.31.200:8080>. Full details: [V1-STATUS.md](#), Chapter 28.

1.6 Learning objectives

The project was built to make the following *tangible*: the HTTP request/response cycle; Express middleware; password hashing; sessions and cookies; authentication vs authorization; SQL (especially `JOIN`, `LEFT JOIN`, `GROUP BY`) and parameterized queries; React components, state, effects, refs, context, and *derived data*; a hand-rolled client-side router; the difference between a query and a mutation; and a real deployment behind a reverse proxy.

1.7 Why a self-hosted SQLite application is the right choice here

SQLite is a database that lives in a single file and runs *inside* your program — there is no separate database server to install, configure, secure, or keep running. For ~10 users with modest write traffic (a few dozen rows per workout) this is ideal:

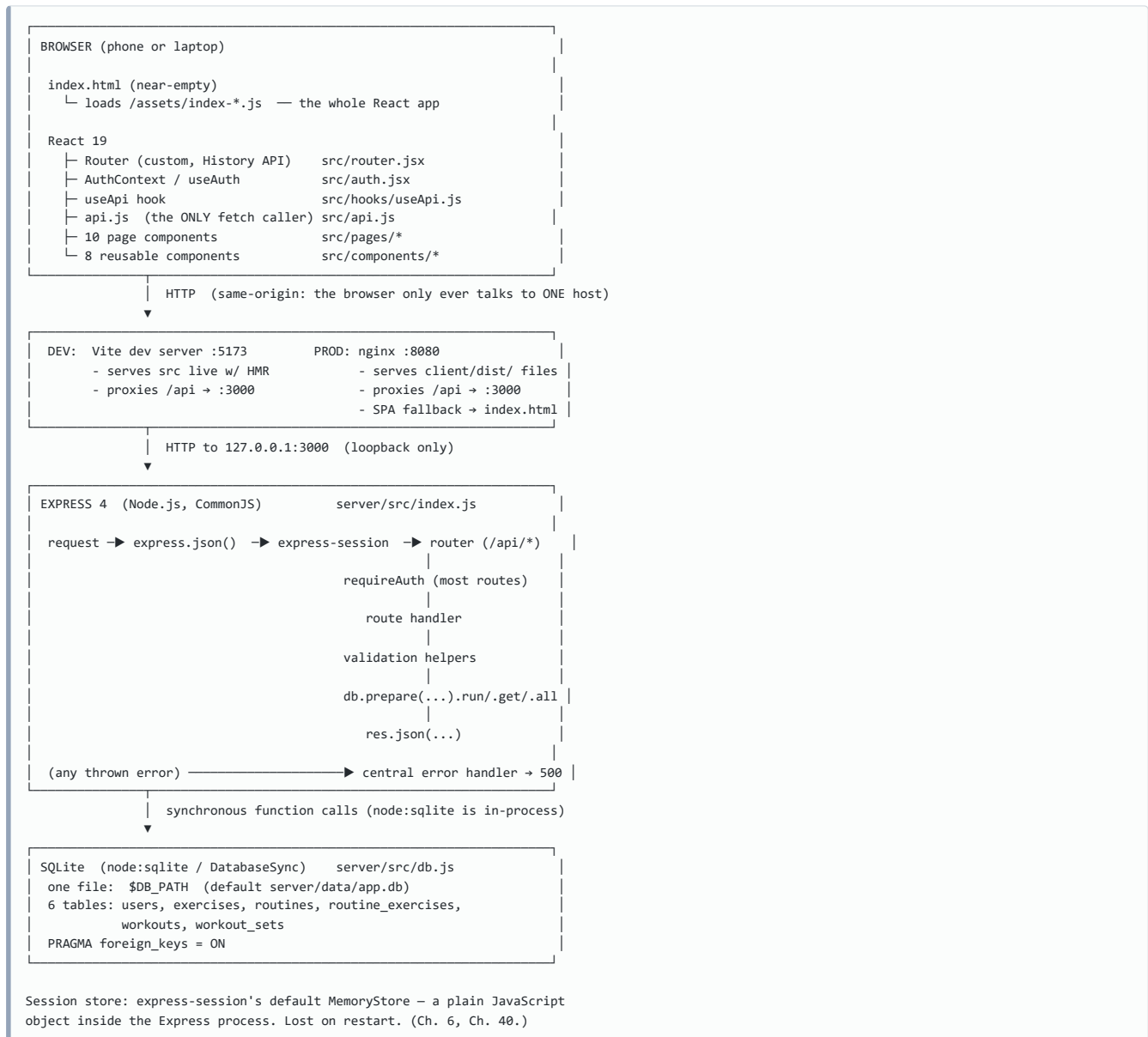
- **Zero operational surface.** One file to back up (Ch. 31). No ports, no credentials, no service to monitor.
- **It runs anywhere Node runs**, including a non-rooted Android phone — which a PostgreSQL server does not, easily.
- **It forces you to learn raw SQL** before reaching for an ORM (Object-Relational Mapper — a library that hides SQL behind method calls). Every query in this project is hand-written and visible (Ch. 10).

- The project uses `node:sqlite`, SQLite compiled *into the Node binary itself* (stable since Node 22.5). There is no native add-on to compile, which is exactly why it works on Termux where compiled add-ons are fragile.

"Self-hosted" means *you* run it, on hardware you control, and your data never leaves it. The trade-off — you are also the operations team — is acceptable at this scale and is itself part of the learning.

2. Complete system architecture

2.1 The parts



2.2 Development vs production — and why they differ

The browser must always talk to **one origin** (scheme + host + port), otherwise the session cookie gets complicated (cross-origin cookies need `SameSite=None; Secure` and CORS headers). Both environments give the browser one origin; they differ only in *what serves the front end* and *how /api is forwarded*.

DEVELOPMENT	PRODUCTION
Browser → http://localhost:5173	Browser → http://<host>:8080
Vite dev server "/ → live React source compiled on the fly, hot-reloaded "/api/* → proxied to :3000 vite.config.js server.proxy	nginx "/ → client/dist/index.html "/assets/* → client/dist/assets/* (cached 1y) "/history/5" → client/dist/index.html (SPA fallback) "/api/* → proxied to 127.0.0.1:3000 deploy/nginx-gym-tracker.conf

Why two setups exist:

- **Dev** optimises the *edit* → *see it* loop: Vite serves your `.jsx` files transformed in the browser, and **Hot Module Replacement (HMR)** swaps changed modules without a full reload. There is no build step.
- **Prod** optimises *delivery*: `npm run build` compiles everything once into a handful of hashed, minified static files in `client/dist/`. nginx serves those as plain files (fast, cacheable) and never runs any JavaScript tooling. The phone needs Node only for the API, never for the front end.

The `vite.config.js` `preview` block (port 4173) runs the *built* `dist/` locally with the same `/api` proxy — a local stand-in for nginx, used to smoke-test a production build before shipping it.

2.3 The request path, in one sentence each

- **Static asset** (GET `/assets/index-abc.js`): browser → nginx → file on disk → browser. Express is not involved.
- **SPA route** (GET `/history/5` typed in the address bar): browser → nginx → *no such file* → `try_files` falls back to `index.html` → React boots → its router reads `/history/5` → renders `WorkoutDetail`.
- **API call** (GET `/api/workouts`): browser `fetch` → nginx `location /api/` → `proxy_pass 127.0.0.1:3000` → Express → `express.json` → `express-session` (reads cookie) → router → `requireAuth` → handler → SQL → `res.json` → back through nginx → `fetch` resolves → React `setState` → re-render.

3. Repository structure

3.1 The tree (source files only; `node_modules`, `dist`, and the DB are excluded)

```
gym-tracker/
├── README.md                project intro, stack, run commands, limitations
├── DEPLOYMENT.md            Termux + nginx runbook (Ch. 28-32)
├── E2E-CHECKLIST.md         manual browser test checklist (Ch. 33)
├── V1-STATUS.md             what is done / verified / deferred + on-device results
├── V2-BACKLOG.md            candidate next work – NOT implemented (Ch. 46)
├── .gitignore               node_modules/, *.db, .env, *.log, client/dist/
├── .gitattributes           force LF line endings (matters on Windows+Termux)
├── .claude/launch.json      local dev-server descriptor (tooling only)
├── server/                  — THE BACKEND —
│   ├── package.json         deps: express, express-session, bcryptjs; engine node>=22.5
│   ├── package-lock.json    exact dependency versions
│   ├── .env.example         template for PORT / HOST / DB_PATH / SESSION_SECRET
│   ├── requests.http        hand-runnable HTTP examples (editor "REST Client")
│   └── src/
│       ├── index.js         app entry: middleware wiring, route mounting, listen()
│       ├── db.js            opens SQLite, runs schema.sql, seeds, PRAGMA foreign_keys
│       ├── schema.sql       the 6 CREATE TABLE statements
│       ├── seed.js          21 exercises, inserted only if the table is empty
│       ├── validation.js    parseId, nonEmptyString, positiveInt, ... (Ch. 13)
│       └── middleware/
│           ├── auth.js      requireAuth – the 401 gate (Ch. 5, 6)
│           └── routes/
│               ├── auth.js  /signup /login /logout /me
│               ├── exercises.js /exercises
│               ├── routines.js /routines (x4)
│               └── workouts.js /workouts (x4: logging + history)
├── test/
│   └── smoke.sh             65 end-to-end API checks (Ch. 33)
├── client/                  — THE FRONTEND —
│   ├── package.json         deps: react, react-dom; dev: vite, @vitejs/plugin-react
│   ├── index.html           the near-empty HTML shell React mounts into
│   ├── vite.config.js       dev proxy, preview proxy, build outDir
│   ├── ARCHITECTURE.md      the "why" behind the frontend decisions
│   └── src/
│       ├── main.jsx         createRoot(...).render(<App>)
│       ├── App.jsx          <Router><AuthProvider><Shell> + the route table
│       ├── api.js           request() + ApiError + api.* helpers (Ch. 18)
│       ├── auth.js          AuthContext / AuthProvider / useAuth (Ch. 20)
│       ├── router.jsx       Router, Link, useNavigate, Redirect, matchPath (Ch. 17)
│       ├── format.js        describeError(), formatDate()
│       ├── hooks/useApi.js   { data, error, loading, reload } (Ch. 19)
│       ├── components/      Button Input Select Card Spinner ErrorMessage
│       │                   EmptyState ExerciseSelect Nav (+ one .css each)
│       ├── pages/           Login Signup Dashboard Exercises Routines
│       │                   RoutineDetail Workout WorkoutStart WorkoutSession
│       │                   SetForm SetList History WorkoutDetail (+ .css)
│       └── styles/
│           ├── tokens.css    CSS custom properties: colours, spacing, radius
│           └── global.css    reset + base element styles
├── deploy/                  — DEPLOYMENT ARTIFACTS —
│   ├── nginx-gym-tracker.conf the server{} block: static + /api proxy + SPA fallback
│   ├── nginx.conf.example    a complete minimal nginx.conf that includes the above
│   ├── start.sh              source .env, assert node:sqlite, exec npm start
│   ├── backup.sh             sqlite3 .backup + keep last 14
│   └── health-check.sh        probe every layer; print HEALTHY / UNHEALTHY
```

3.2 What calls what (backend)

```
node src/index.js
├── require('./db').init() → reads schema.sql, runs seed.js
├── express()
├── app.use(express.json())
├── app.use(session({...}))
├── app.use('/api', routes/auth) → require('./db').db, require('bcryptjs'), middleware/auth
├── app.use('/api', routes/exercises) → db, middleware/auth
├── app.use('/api', routes/routines) → db, middleware/auth, validation
├── app.use('/api', routes/workouts) → db, middleware/auth, validation
├── app.use(errorHandler)
└── app.listen(PORT, HOST)
```

Every route file imports the SAME `db` object from `db.js` (one connection, shared, because `node:sqlite` is synchronous and single-process).

3.3 What calls what (frontend)

```
index.html → main.jsx → App.jsx
App.jsx renders <Router> (router.jsx)
               <AuthProvider> (auth.jsx → api.js, router.jsx)
               <Shell>
                 matchPath(...) picks one of 10 page components
                 page components → useApi (hooks/useApi.js) → api.js
                                   → components/* (Button, Input, ...)
                 if authed also renders <Nav> (components/Nav.jsx)

api.js is imported by: auth.jsx, useApi callers, and directly by mutation
handlers (Routines, RoutineDetail, WorkoutStart, SetForm). It is the ONLY
module that calls fetch().
```

3.4 Files that exist but carry little logic

- `client/src/**/*.css` — styling only; the design system is `styles/tokens.css` (Ch. 14 §14.6).
- `server/requests.http` — a scratch pad of example requests for the VS Code "REST Client" extension; not part of the running app.
- `.claude/launch.json` — a descriptor used by the developer's editor tooling to start the dev servers; ignored in production.

4. Backend runtime

4.1 The vocabulary

- **Node.js** — a program that runs JavaScript *outside a browser*. It gives JavaScript access to the filesystem, network sockets, processes, and — since v22.5 — a built-in SQLite (`node:sqlite`).
- **Express 4** — the most widely used Node web framework. It is a thin layer over Node's raw HTTP server that provides *routing* (map a method + path to a function) and *middleware* (a pipeline of functions each request passes through). This project pins `express@^4.21.2`. Express 5 exists but 4 has the larger body of tutorials — deliberate, for a learning project.
- **CommonJS** — the older of Node's two module systems. Files use `require('x')` to import and `module.exports = y` to export. (The newer system is ESM: `import` / `export`.) The **backend is CommonJS**; the **frontend is ESM** (`client/package.json` has `"type": "module"`). Two systems in one repo is normal and fine — they never import each other.

4.2 Server startup, line by line — `server/src/index.js`

```
const express = require('express');
const session = require('express-session');
const { init } = require('./db');

init(); // 1. create tables + seed exercises BEFORE serving
const app = express(); // 2. the application object

app.use(express.json()); // 3. middleware: parse JSON request bodies
app.use(session({ ... })); // 4. middleware: read/write the session cookie

app.get('/', (req, res) => res.send('hello')); // 5. a liveness route (not /api)

app.use('/api', require('./routes/auth')); // 6. mount the four routers
app.use('/api', require('./routes/exercises')); // all under the /api prefix
app.use('/api', require('./routes/routines'));
app.use('/api', require('./routes/workouts'));

app.use((err, req, res, next) => { // 7. central error handler
  console.error(err);
  res.status(500).json({ error: 'internal server error' });
});

const PORT = process.env.PORT || 3000;
const HOST = process.env.HOST || '127.0.0.1';
app.listen(PORT, HOST, () => { ... }); // 8. start listening
```

Notes:

- **init() is synchronous and runs first.** `node:sqlite`'s `DatabaseSync` is a *synchronous* API — `db.exec(...)` blocks until done. So by the time `app.listen` runs, the schema exists and the exercise library is seeded. No request can ever hit an un-migrated database.
- **app.get('/', ...) returns the text `hello`.** This is **not** one of the 13 API endpoints. In production nginx serves `index.html` at `/`, so a browser never reaches this. It exists as a **liveness probe**: `health-check.sh` and `smoke.sh` do `curl http://127.0.0.1:3000/` and expect `200` to confirm "the process is up and answering".
- **HOST defaults to `127.0.0.1`** (loopback). Only nginx (also on the phone) talks to Express, so port 3000 has no reason to be reachable from the network. `HOST=0.0.0.0` overrides this — used only to `curl` the API from another device during debugging. **VERIFIED** on-device: `http://192.168.31.200:3000` refuses connections; `:8080` (nginx) works.
- **PORT and HOST come from the environment**, with defaults, so the same code runs unchanged in dev and on the phone. `deploy/start.sh` loads `server/.env` before `npm start`.

4.3 How a request travels through Express

Express keeps an ordered list of *middleware* and *route handlers*. For each incoming request it walks that list from the top. Each entry is a function (`req, res, next`):

- **req** — the request: `req.method`, `req.path`, `req.headers`, `req.body` (after `express.json`), `req.params` (route params like `:id`), `req.session` (after `express-session`), and any properties earlier middleware attached (this project attaches `req.userId`).
- **res** — the response: `res.status(n)`, `res.json(obj)`, `res.send(text)`, `res.clearCookie(name)`. Calling one of the *sending* methods ends the request.
- **next** — a function. Call `next()` to hand control to the **next** entry in the list. Call `next(err)` to **skip straight to the error handler**. If a middleware neither sends a response nor calls `next`, the request hangs forever.

So a `GET /api/routines` request walks:

```

express.json()      → not JSON body, nothing to do → next()
session()           → reads the 'connect.sid' cookie, loads req.session → next()
router (routes/auth) → no path match for GET /api/routines → next()
router (routes/exercises) → no match → next()
router (routes/routines) → router.use(requireAuth) runs first:
                        req.session.userId set? yes → req.userId = it → next()
                        GET /routines handler:
                        db.prepare('SELECT id,name FROM routines WHERE user_id=? ORDER BY name')
                        .all(req.userId)
                        res.json(rows) ← request ends here
(error handler never reached)

```

4.4 The response lifecycle

A handler builds a response by calling `res.status(...)` (optional; default 200) then one terminating method:

- `res.json(value)` — serialises `value` to JSON, sets `Content-Type: application/json`, sends it.
- `res.send(string)` — sends text/HTML.
- `res.status(204).end()` — no body (not used here).

Express also sets `ETag` and `Content-Length` automatically. Once any of these is called, the request is finished; calling a second one throws "Cannot set headers after they are sent".

4.5 Error handling

Express recognises a middleware with **four** parameters (`err, req, res, next`) as an *error handler*. It is only invoked when some earlier code calls `next(err)` or (in Express 4) throws **synchronously** inside a handler.

```

app.use((err, req, res, next) => {
  console.error(err); // full detail to the server log
  res.status(500).json({ error: 'internal server error' }); // generic text to the client
});

```

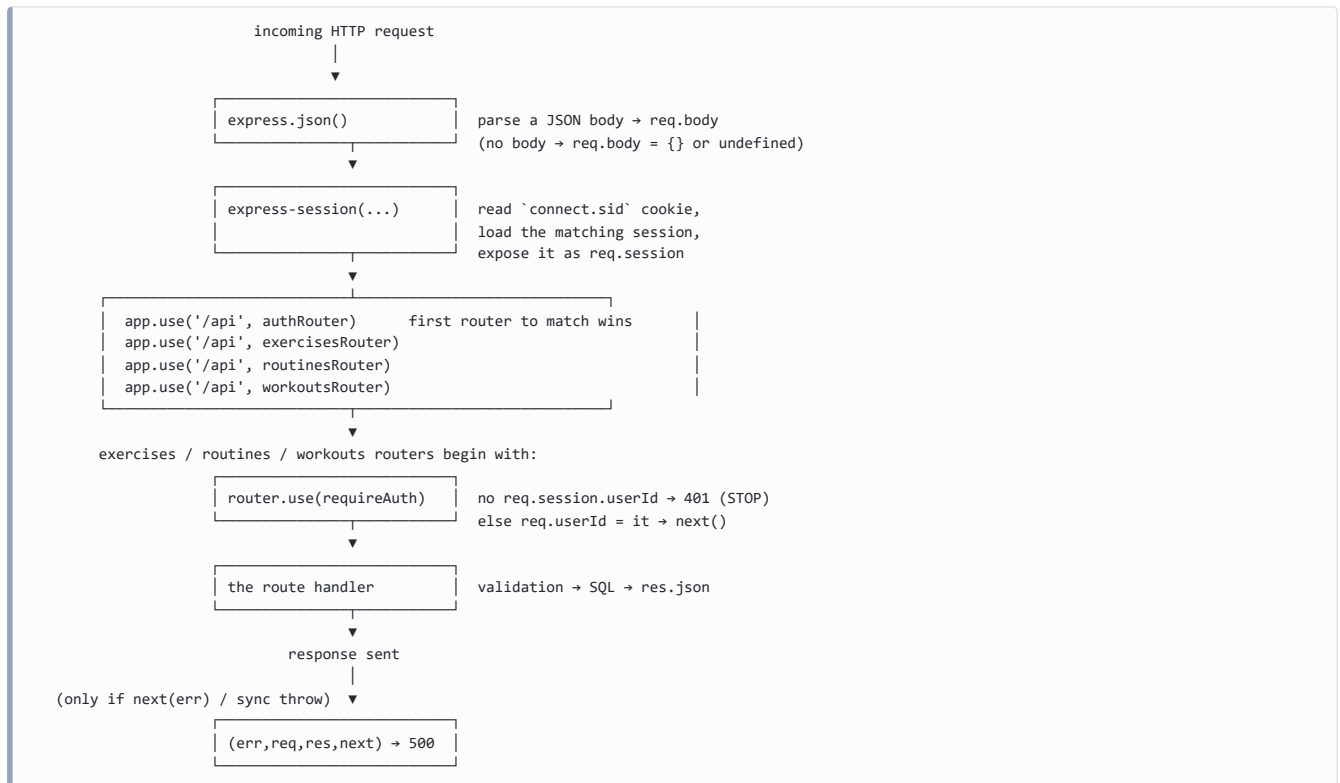
Two deliberate properties:

1. **The client never sees** `err.message` or a **stack trace**. It gets a fixed string. Leaking internal errors (SQL text, file paths) is an information disclosure vulnerability (Ch. 27).
2. **The full error is** `console.error 'd`, so the operator can diagnose from the Termux log (`~/gym-tracker.log`).

Express 4 does not catch `async` rejections. If an `async` handler's promise rejects, Express 4 does *not* route it to the error handler automatically. The `auth` routes work around this: their handlers are `async (req, res, next)` wrapped in `try/catch`, and the `catch` calls `next(err)` explicitly. The `routines/workouts/exercises` handlers are **synchronous** (`node:sqlite` is synchronous — no `await` needed), so a synchronous `throw` there *is* caught. In practice those handlers don't throw because every failure path is an explicit `return res.status(4xx).json(...)`.

5. Express middleware pipeline

5.1 The pipeline for this app



5.2 What "middleware" means and why order matters

Middleware is just "a function every request runs through, in order." Ordering is not cosmetic — it is the whole design:

- `express.json()` **must** come before any handler that reads `req.body`. It is registered first, so by the time a route handler runs, `req.body` is a parsed object.
- `session(...)` **must** come before `requireAuth`, because `requireAuth` reads `req.session.userId`. If session middleware ran later, `req.session` would be `undefined` and every protected request would 401.
- The **error handler must be last** (`app.use(...)` after every route). Express only falls through to it; it never "jumps back up."
- The four routers are all mounted at `/api`. Express tries them **in registration order** and the first one with a matching method+path handles the request. Since their paths don't overlap (`/signup` vs `/exercises` vs `/routines` vs `/workouts`), order among them is irrelevant *except* that a no-match router calls `next()` and the request continues to the following one.

5.3 `req`, `res`, `next` in this codebase

`server/src/middleware/auth.js` **IMPLEMENTED** — the whole file:

```
module.exports = function requireAuth(req, res, next) {
  if (!req.session || !req.session.userId) {
    return res.status(401).json({ error: 'Not authenticated' });
  }
  req.userId = req.session.userId; // convenience for handlers
  next();
};
```

- Reads `req.session` (put there by the session middleware above it).
- On failure: `res.status(401).json(...)` — **sends a response and returns**; `next` is never called, so the route handler never runs.
- On success: attaches `req.userId` (so handlers write `req.userId`, not `req.session.userId`, everywhere) and calls `next()` to proceed.

`routes/exercises.js`, `routes/routines.js`, `routes/workouts.js` each do `router.use(requireAuth)` at the top — one line that guards *every* route in the file. `routes/auth.js` does **not** (signup/login must be reachable while logged out); it applies `requireAuth` only to `GET /me`.

6. Authentication

6.1 First principles

Authentication answers "*who is this request from?*" HTTP is stateless — every request arrives with no memory of previous ones. To have a "logged-in user" the server must:

1. On login, verify the caller knows a secret (the password).
2. Hand the browser a token that proves "I already logged in."
3. On every later request, read that token and recover the user.

This project uses **session cookies** for steps 2–3 (the alternative, JWTs, is discussed in §6.9).

6.2 Signup — `POST /api/signup` **IMPLEMENTED**

```
router.post('/signup', async (req, res, next) => {
  try {
    const { username, password } = req.body || {};
    if (!username || !password)
      return res.status(400).json({ error: 'username and password are required' });
    if (password.length < 6)
      return res.status(400).json({ error: 'password must be at least 6 characters' });

    const passwordHash = await bcrypt.hash(password, BCRYPT_ROUNDS); // BCRYPT_ROUNDS = 12

    const result = db
      .prepare('INSERT INTO users (username, password_hash) VALUES (?, ?)')
      .run(username, passwordHash);

    const id = Number(result.lastInsertRowid);
    req.session.userId = id; // log them in immediately
    res.status(201).json({ id, username });
  } catch (err) {
    if (String(err.message).includes('UNIQUE')) // users.username is UNIQUE
      return res.status(409).json({ error: 'username already taken' });
    next(err);
  }
});
```

Step by step: validate → **hash the password** → insert → **put the new user's id on the session** (so signup logs you straight in) → respond `201` with the public fields. A duplicate username hits the DB's `UNIQUE` constraint; the `catch` detects it by message text and returns `409 Conflict` instead of a `500`.

6.3 Password hashing — `bcrypt` / `bcryptjs`

Storing passwords as plain text is catastrophic: one database leak exposes every account, and people reuse passwords. Instead the server stores a **hash** — a one-way transformation. Given the hash you cannot get the password back; on login you hash the attempt and compare.

- **bcrypt** is a hashing algorithm designed *for passwords*: deliberately slow, with a tunable cost.
- **bcryptjs** (`^2.4.3`) is a pure-JavaScript implementation. The project uses it, **not** the faster native `bcrypt` package, because native packages need compilation and can fail on Termux (the same reason `node:sqlite` was chosen).
- **Salt**. `bcrypt` generates a random *salt* per password and stores it *inside* the hash string. So two users with password `hunter2` get different hashes, and an attacker cannot pre-compute a lookup table ("rainbow table").
- **Work factor / cost**. `BCRYPT_ROUNDS = 12` means `bcrypt` runs 2^{12} internal iterations. On modern hardware that is ~200–300 ms per hash — invisible for one login, but it makes brute-forcing a stolen hash table astronomically expensive. Each `+1` doubles the cost.

A stored hash looks like `$2a$12$Q1w... : $2a$` = algorithm, `12` = cost, then the 22-char salt and 31-char digest. It is always 60 characters — `smoke.sh` - adjacent checks and a manual DB inspection during development confirmed stored values start `$2a$` and are 60 chars.

6.4 Login — `POST /api/login` **IMPLEMENTED**

```
const user = db.prepare('SELECT * FROM users WHERE username = ?').get(username);
const ok = user && (await bcrypt.compare(password, user.password_hash));
if (!ok) return res.status(401).json({ error: 'invalid username or password' });
req.session.userId = user.id;
res.json({ id: user.id, username: user.username });
```

- `bcrypt.compare(plain, hash)` re-hashes `plain` with the salt embedded in `hash` and checks for a match. It returns a boolean.
- **One generic error** (`invalid username or password`) whether the username is unknown *or* the password is wrong. Distinct messages ("no such user" vs "wrong password") would let an attacker enumerate which usernames exist.
- On success, `req.session.userId = user.id`. That single assignment is "the user is now logged in."

6.5 Sessions and cookies

- A **cookie** is a small `name=value` string. The server sends `Set-Cookie: connect.sid=...` on a response; the browser stores it and **automatically attaches it to every subsequent request to that origin**. You never write code to send it.
- `express-session` works like this: when a handler first writes to `req.session` (here: `req.session.userId = ...`), the middleware
- generates a random **session id**,
- stores `{ userId: N }` in its **store**, keyed by that id,
- sends `Set-Cookie: connect.sid=<signed session id>`. On later requests it reads the cookie, verifies the signature (using `SESSION_SECRET`), looks the id up in the store, and sets `req.session` to the stored object.
- **What the browser holds is only an opaque id**. The `userId` never leaves the server. The client cannot read or forge it.

The cookie configuration (`server/src/index.js`):

```
session({
  secret: process.env.SESSION_SECRET || 'dev-only-secret-change-me',
  resave: false,           // don't rewrite the session if nothing changed
  saveUninitialized: false, // don't create a session for anonymous visitors
  cookie: {
    httpOnly: true,        // JavaScript (document.cookie) cannot read it
    sameSite: 'lax',       // browser won't send it on cross-site POSTs
    secure: false,         // sent over plain HTTP (see below)
    maxAge: 1000*60*60*24*7, // expires after 7 days
  },
})
```

Attribute	Threat it addresses	How
<code>httpOnly: true</code>	XSS cookie theft — a malicious script reading <code>document.cookie</code>	The cookie is invisible to JavaScript; only the browser's HTTP layer sees it
<code>sameSite: 'lax'</code>	CSRF — another site auto-submitting a form to your API using the victim's cookie	The browser refuses to attach the cookie to cross-site <code>POST</code> s; it is attached to top-level GET navigations, which is safe
<code>secure: false</code>	(none — this is a <i>concession</i>)	Would require HTTPS. The V1 deployment is plain HTTP on a LAN, so <code>secure: true</code> would make the cookie never send. Flip it to <code>true</code> when HTTPS is added (Ch. 46)
<code>maxAge</code>	stale sessions living forever	Browser drops the cookie after 7 days; the user re-logs-in

- `SESSION_SECRET` signs the cookie so a client cannot tamper with the session id. In production it must be a long random value (`server/.env`); unset, the code falls back to an insecure literal. On the phone it was set to `openssl`-style 64 hex chars (never shown in chat or committed).
- **MemoryStore**. With no `store:` option, `express-session` uses its **default in-memory store** — a plain object in the Node process. Consequence: **restart the server and every session is gone; users must log in again**. The database is untouched. This is an accepted V1 limitation (Ch. 40) and was **VERIFIED** on-device: after `kill node` + restart, the old cookie returns `401` but all routines/workouts/sets persist.

6.6 Logout — `POST /api/logout` **IMPLEMENTED**

```
router.post('/logout', (req, res) => {
  req.session.destroy(() => {
    res.clearCookie('connect.sid');
    res.json({ ok: true });
  });
});
```

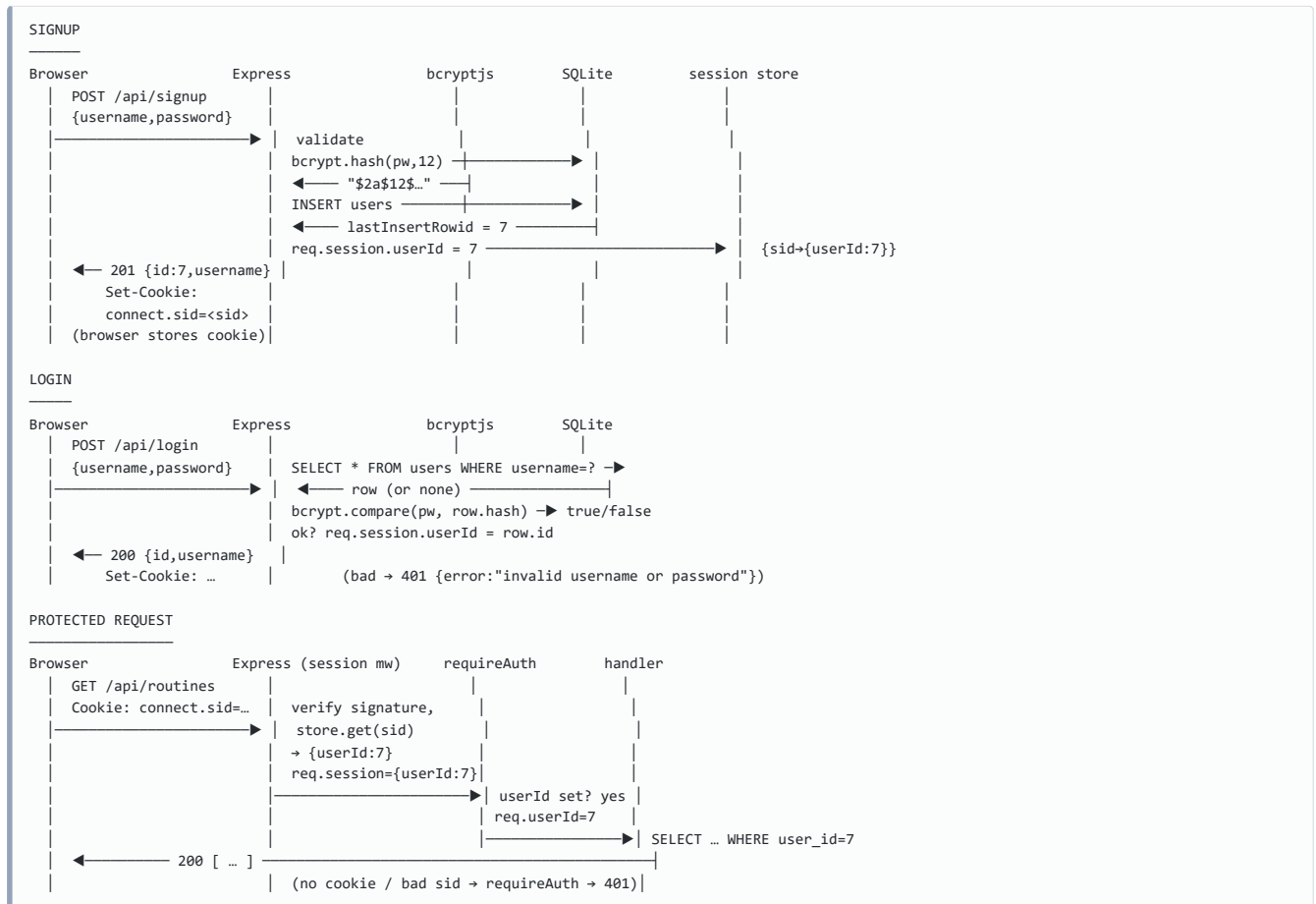
`destroy()` removes the session from the store; `clearCookie` tells the browser to drop it. After this, the same cookie value is meaningless and `GET /api/me` returns `401` **VERIFIED**.

6.7 `GET /api/me` — the "who am I" endpoint **IMPLEMENTED**

```
router.get('/me', requireAuth, (req, res) => {
  const user = db
    .prepare('SELECT id, username, created_at FROM users WHERE id = ?')
    .get(req.userId);
  res.json(user);
});
```

Protected by `requireAuth`. Returns the current user's public fields (**not** `password_hash`). This is the endpoint the front end calls on every page load to rebuild its auth state (Ch. 20).

6.8 Sequence diagrams



6.9 Authentication vs authorization — and why sessions here, not JWT

- **Authentication** = *who are you?* (login, the session lookup).
- **Authorization** = *are you allowed to touch **this** resource?* (Ch. 7). They are separate. A logged-in user (authenticated) still must not be able to read another user's workout (authorization).

Why session cookies and not JWT (a JSON Web Token — a signed blob the client stores and sends, that the server verifies without a lookup):

	Session cookie (this project)	JWT
Where user identity lives	server store; client holds an opaque id	inside the token the client holds
Revoke a login instantly	yes — delete it from the store	hard — must wait for expiry or maintain a blocklist
XSS exposure	<code>httpOnly</code> cookie is unreadable by JS	usually kept in JS-readable storage
Extra infrastructure	a store (here: memory; later: SQLite)	none
Right call at ~10 users, one server	yes — simplest thing that is secure	overkill; its advantage (stateless scaling) doesn't apply

7. Authorization / ownership security

This is the most important security chapter. The pattern is small and appears in every user-owned route.

7.1 The threat: IDOR

IDOR — *Insecure Direct Object Reference*. The user is authenticated, but the server fetches a resource **by id alone** and returns it without checking who owns it. The attacker just increments the id in the URL:

```
GET /api/workouts/41 → my workout, fine
GET /api/workouts/42 → someone else's workout — if the server doesn't check, LEAK
```

7.2 The fix used everywhere: filter by owner in the SQL

routes/workouts.js, GET /api/workouts/:id:

```
const workout = db
  .prepare(
    `SELECT w.id, w.date, w.routine_id, r.name AS routine_name
     FROM workouts w
     LEFT JOIN routines r ON r.id = w.routine_id
     WHERE w.id = ? AND w.user_id = ?`) // ← both conditions
  .get(workoutId, req.userId);
if (!workout) return res.status(404).json({ error: 'workout not found' });
```

`req.userId` comes from the **session**, never from the request. The query asks for "the workout with this id **that also belongs to me**." If there is no such row — wrong id, or someone else's id — `.get()` returns `undefined` and the handler returns `404`.

Every user-owned access follows this shape:

Route	Ownership clause
GET /api/routines	WHERE user_id = ?
GET /api/routines/:id	WHERE id = ? AND user_id = ?
POST /api/routines/:id/exercises	pre-check: SELECT id FROM routines WHERE id = ? AND user_id = ?
POST /api/workouts (with routine_id)	pre-check: SELECT id FROM routines WHERE id = ? AND user_id = ?
POST /api/workouts/:id/sets	pre-check: SELECT id FROM workouts WHERE id = ? AND user_id = ?
GET /api/workouts	WHERE w.user_id = ?
GET /api/workouts/:id	WHERE w.id = ? AND w.user_id = ?

7.3 Why this beats "fetch, then check in JavaScript"

The tempting alternative:

```
// DON'T
const w = db.prepare('SELECT * FROM workouts WHERE id = ?').get(id);
if (w.user_id !== req.userId) return res.status(403).json(...);
return res.json(w);
```

Problems:

1. **The row is already in memory.** It is now one forgotten `if`, one early `return`, one refactor away from leaking. The SQL version *cannot return a row it didn't select*.
2. `w` **might be** `undefined` (bad id) → `w.user_id` throws → `500`. You'd need a null check too.
3. It spreads the security rule across two places (query + check) that can drift apart. The SQL version keeps it in one clause.

The principle: **make "not yours" unrepresentable**, don't detect it after the fact.

7.4 404, not 403 — existence disclosure

When a resource exists but isn't yours, the project returns `404 Not Found`, the same as when it doesn't exist at all. A `403 Forbidden` would confirm "a workout with id 42 exists" — a small information leak that lets an attacker map which ids are real. Returning `404` for both makes the two cases indistinguishable from outside.

(The `POST /api/workouts` "start from a routine" check returns `400 "routine_id does not exist"` rather than `404` — because `routine_id` is a *body field*, not the resource being addressed, and the wording still doesn't confirm it exists for someone else.)

7.5 Frontend guards are not security

`App.jsx`'s `Shell` redirects an anonymous user away from a protected route. That is a **convenience** — it stops a confusing empty screen. It is trivially bypassed (disable JavaScript, or call the API directly with `curl`). The real boundary is the server: `requireAuth` 401s every `/api` call without a session, and the ownership clauses 404 every cross-user access. The frontend guard could be deleted and the app would still be secure — just less pleasant.

7.6 The two-user model, verified

The `smoke.sh` suite creates **two** users, Alice and Bob, and asserts:

- Bob's `GET /api/routines` never contains Alice's routine.
- `GET /api/routines/<Alice's id>` as Bob → 404.
- `POST /api/routines/<Alice's id>/exercises` as Bob → 404.
- `POST /api/workouts {routine_id: <Alice's id>}` as Bob → 400.
- `POST /api/workouts/<Alice's id>/sets` as Bob → 404, **and the set count on Alice's workout is unchanged** (the write never happened).
- `GET /api/workouts/<Alice's id>` as Bob → 404; Bob's history is empty.
- Alice can still do everything afterwards.

VERIFIED in `smoke.sh` (65/65) **and** re-run manually against the live phone on 2026-09-06 (Ch. 33, `V1-STATUS.md`).

8. Database architecture

8.1 SQLite from first principles

A **relational database** stores data as **tables** (like spreadsheets): each table has **columns** (typed fields) and **rows** (records). You query it with **SQL**.

SQLite is a relational database that is *not* a server. It is a C library; `node:sqlite` links that library into Node. Your "database" is **one file** (`app.db`). Reads and writes are ordinary function calls into that library — there is no network, no separate process, no connection pool.

- **Connection.** `new DatabaseSync(DB_PATH)` opens the file. This project opens it **once** (`db.js`) and shares the single `db` object with every route. That is correct here: `node:sqlite` is synchronous and the app is one process, so there is never concurrent access to coordinate.
- **DatabaseSync.** The synchronous flavour of `node:sqlite`. `db.exec(sql)` runs statements; `db.prepare(sql)` compiles a statement you then call with `.run(...)` (writes), `.get(...)` (one row), or `.all(...)` (all rows).

8.2 Keys, constraints, types

- **Primary key.** `id INTEGER PRIMARY KEY AUTOINCREMENT` — a unique identifier the database assigns automatically. `AUTOINCREMENT` guarantees ids are never reused even after deletes.
- **NOT NULL** — the column must have a value.
- **UNIQUE** — no two rows may share this value. `users.username` is `UNIQUE`; a duplicate `INSERT` throws an error whose message contains `UNIQUE`, which the signup handler turns into a `409`.
- **DEFAULT CURRENT_TIMESTAMP** — if you don't supply the column on insert, SQLite fills in the current UTC time as `"YYYY-MM-DD HH:MM:SS"`. Used for `users.created_at` and `workouts.date`.
- **Types.** SQLite is loosely typed. `INTEGER`, `TEXT`, `REAL` (floating point — used for `workout_sets.weight` so `42.5` kg is storable). A column with no type affinity stores whatever you give it.

8.3 Foreign keys

A **foreign key** says "this column's value must match an `id` in another table." `routines.user_id` is `FOREIGN KEY ... REFERENCES users(id)` — a routine must belong to a real user.

SQLite disables foreign-key enforcement by default (a 2009 backwards-compat decision). It must be turned on **per connection**:

```
db.exec('PRAGMA foreign_keys = ON'); // db.js, right after opening
```

Without this line, every `FOREIGN KEY` clause in `schema.sql` is *decorative* — you could insert a `workout_sets` row referencing exercise id 9999. With it, such an insert throws. The project relies on this: `POST /workouts/:id/sets` checks the exercise exists *before* inserting to turn a would-be FK-violation `500` into a clean `400`, but the FK is the backstop.

A `PRAGMA` is an SQLite-specific command to configure or inspect the database engine (not standard SQL).

8.4 Transactions

A **transaction** groups multiple writes so they either *all* apply or *none* do. **This project uses none** — and that is a deliberate, documented decision, not an omission. Every write endpoint performs **exactly one** `INSERT`:

- `POST /routines` → one insert.
- `POST /routines/:id/exercises` → one insert (the ownership + existence checks before it are *reads*).
- `POST /workouts` → one insert.
- `POST /workouts/:id/sets` → one insert.

A transaction matters when *two* writes must stay consistent (e.g. "create a workout **and** copy in all the routine's exercises as planned sets"). V1 has no such endpoint, so there is nothing to make atomic. The route comments say this explicitly.

8.5 Parameterized queries

Every value that comes from a user is passed as a **bound parameter** (`?`), never concatenated into the SQL string:

```
db.prepare('SELECT * FROM users WHERE username = ?').get(username); // ✓  
// NEVER: db.prepare('SELECT * FROM users WHERE username = '${username}')
```

The `?` placeholders are sent to SQLite **separately** from the query text. SQLite treats them as pure data — they can never change the *structure* of the query. This is the defence against **SQL injection**, where an input like `' OR '1'='1` would otherwise rewrite the query's logic. There is no string interpolation of user input anywhere in this codebase.

9. Database schema

The complete `server/src/schema.sql`, table by table. Every statement is `CREATE TABLE IF NOT EXISTS`, so running the file on every startup is safe — it creates only what is missing.

9.1 users

```
CREATE TABLE IF NOT EXISTS users (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  username    TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  created_at  TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

- **Purpose:** one row per account.
- `username` — `UNIQUE` (drives the 409 on duplicate signup) and `NOT NULL`.
- `password_hash` — the bcrypt string (60 chars). The raw password is never stored.
- `created_at` — auto-filled UTC timestamp; returned by `GET /api/me`.
- **Relationships:** referenced by `routines.user_id` and `workouts.user_id`.

9.2 exercises

```
CREATE TABLE IF NOT EXISTS exercises (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  name        TEXT NOT NULL,  
  muscle_group TEXT  
);
```

- **Purpose:** the shared exercise library. **Not** per-user — every logged-in user sees the same 21 rows.
- `muscle_group` is nullable in the schema, though every seeded row has one (`seed.js`). The frontend defensively handles a missing muscle group.
- **Why shared:** exercises are reference data ("Barbell Bench Press" means the same thing for everyone). Making them per-user would multiply storage and complicate the UI for no benefit. V1 has no endpoint to add or edit them — they exist only via `seed.js`.
- **Relationships:** referenced by `routine_exercises.exercise_id` and `workout_sets.exercise_id`.

9.3 routines

```
CREATE TABLE IF NOT EXISTS routines (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id    INTEGER NOT NULL,  
  name        TEXT NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

- **Purpose:** a user-created named template ("Push Day").
- **Ownership:** `user_id NOT NULL` — every routine belongs to exactly one user. The API only ever sets this from `req.userId`.

9.4 routine_exercises

```
CREATE TABLE IF NOT EXISTS routine_exercises (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  routine_id  INTEGER NOT NULL,  
  exercise_id INTEGER NOT NULL,  
  target_sets INTEGER,  
  target_reps INTEGER,  
  FOREIGN KEY (routine_id) REFERENCES routines(id),  
  FOREIGN KEY (exercise_id) REFERENCES exercises(id)  
);
```

- **Purpose:** the **link table** (also called a "join table" or "junction table") between routines and exercises. A routine has many exercises; an exercise appears in many routines; this table represents that many-to-many relationship, **plus** the per-routine planning data.
- `target_sets`, `target_reps` — nullable "planned" numbers. Optional in the API.
- **Duplicates are allowed** — the same `exercise_id` can appear twice in one routine (an opener and a burnout set). There is no `UNIQUE(routine_id, exercise_id)` constraint, on purpose.

9.5 workouts

```
CREATE TABLE IF NOT EXISTS workouts (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id     INTEGER NOT NULL,  
  routine_id  INTEGER,           -- NULLABLE  
  date        TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  FOREIGN KEY (routine_id) REFERENCES routines(id)  
);
```

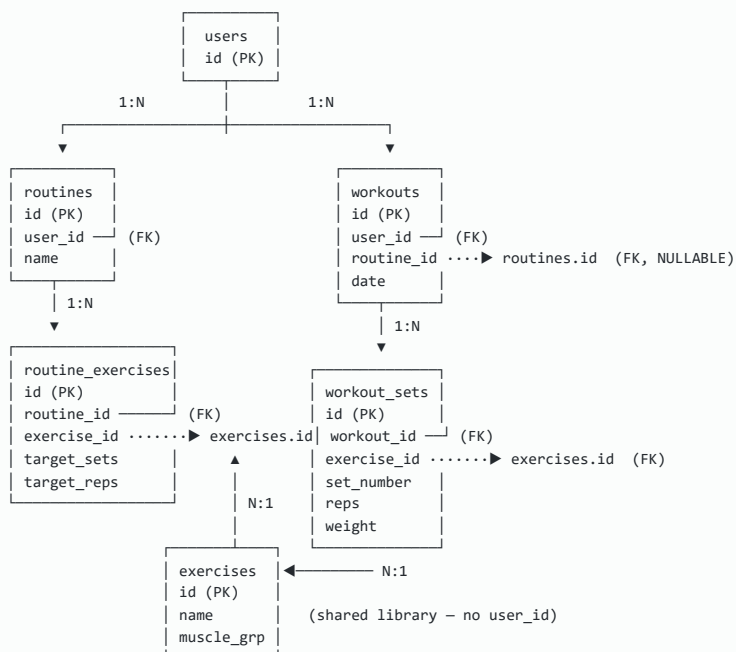
- **Purpose:** one row per workout *session*.
- `routine_id` is **nullable**. This one nullable column is what makes **freestyle workouts** possible: `NULL` = "no routine, just logging as I go." A routine-based workout stores the routine's id.
- `date` — auto-filled UTC timestamp of when the workout was started. There is **no** "ended"/"completed" column — a workout is simply a row that exists (Ch. 40).

9.6 workout_sets

```
CREATE TABLE IF NOT EXISTS workout_sets (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  workout_id  INTEGER NOT NULL,  
  exercise_id INTEGER NOT NULL,  
  set_number  INTEGER,  
  reps        INTEGER,  
  weight      REAL,  
  FOREIGN KEY (workout_id) REFERENCES workouts(id),  
  FOREIGN KEY (exercise_id) REFERENCES exercises(id)  
);
```

- **Purpose:** one row per set *actually performed*.
- `set_number` — supplied by the client (the frontend computes it; \$23, \$24).
- `weight` — **REAL** so decimals work; `0` is valid and means "bodyweight".
- The schema comment notes a future `set_type` column (`warmup / normal / dropset / failure`) — **DEFERRED**, mentioned so the schema isn't designed in a way that makes it hard to add.

9.7 Entity-Relationship diagram



Reading it:

- one user has many routines and many workouts
- one routine has many routine_exercises (its planned exercises)
- one workout has many workout_sets (its performed sets)
- one exercise is referenced by many routine_exercises and many workout_sets
- a workout's routine_id may be NULL → freestyle

9.8 Planned data vs actual data — the key conceptual split

"Planned" (a template)	"Actual" (what happened)
<code>routines</code> + <code>routine_exercises</code>	<code>workouts</code> + <code>workout_sets</code>
"On Push Day I intend to do Bench 4×8, OHP 3×10"	"On Sat 6 Sep I benched 10 reps at 60 kg, then 9 at 60 kg"
<code>target_sets</code> , <code>target_reps</code> (nullable, aspirational)	<code>set_number</code> , <code>reps</code> , <code>weight</code> (recorded)
Editable plan, reused across many workouts	Immutable log of one session

A routine-based workout copies **nothing** from the routine into `workout_sets` — the routine is shown as a *suggestion* ("Today's plan" chips), and the user still records every actual set. This keeps the two models fully independent: editing a routine later never rewrites past workouts.

9.9 Indexes — what exists and what doesn't

SQLite automatically indexes every `PRIMARY KEY` and every `UNIQUE` column. So there are implicit indexes on all six `id` columns and on `users.username`.

There are **no explicit secondary indexes** — in particular the foreign-key columns (`routines.user_id`, `workouts.user_id`, `workout_sets.workout_id`, ...) are **not** indexed. At ~10 users and a few hundred rows total, full table scans are microseconds; an index would be premature. If the data grew to tens of thousands of workouts, `CREATE INDEX idx_workouts_user ON workouts(user_id)` (and similar) would be the first optimisation. **DEFERRED**.

10. SQL walkthrough

Every non-trivial query in the codebase, with each clause explained.

10.1 GET /api/exercises — the simplest query

```
SELECT id, name, muscle_group
FROM exercises
ORDER BY muscle_group, name
```

- **SELECT id, name, muscle_group** — return exactly these three columns. Listing them (rather than `SELECT *`) means the response shape is fixed and no column can accidentally leak.
- **FROM exercises** — the only table.
- **No WHERE** — the library is public to any logged-in user; there is nothing to filter.
- **ORDER BY muscle_group, name** — SQLite gives **no ordering guarantee** without an explicit `ORDER BY`. Today rows come back in insert order; that is not a contract. Sorting by muscle then name makes the response *stable across calls* and already grouped for the UI. `muscle_group` is nullable — SQLite sorts `NULL` first in ascending order — but every seeded row has one.
- **Result shape:** an array of `{id, name, muscle_group}`. Zero rows → `[]`.
- **No parameters** — there is no user input in this query at all.

10.2 Routine ownership + list

```
-- GET /api/routines
SELECT id, name FROM routines WHERE user_id = ? ORDER BY name
```

`WHERE user_id = ?` is the authorization boundary. The `?` is bound to `req.userId`. There is no code path that returns a row this query didn't select, so a bug elsewhere cannot leak another user's routines. Zero routines → `[]` (an empty list is valid data — the frontend shows an "empty state", not an error).

10.3 Routine detail — a JOIN across the link table

`GET /api/routines/:id` runs **two** queries.

Query 1 — fetch + authorize together:

```
SELECT id, name FROM routines WHERE id = ? AND user_id = ?
```

No row → the routine doesn't exist or isn't yours → `404` (Ch. 7.4).

Query 2 — the routine's exercises (only runs if query 1 found a row):

```
SELECT e.id, e.name, e.muscle_group,
       re.target_sets, re.target_reps
FROM routine_exercises re
JOIN exercises e ON e.id = re.exercise_id
WHERE re.routine_id = ?
ORDER BY re.id
```

- **FROM routine_exercises re** — start from the link table, aliased `re`.
- **JOIN exercises e ON e.id = re.exercise_id** — an **INNER JOIN**. For each link row, find the one `exercises` row whose `id` equals this link's `exercise_id`, and combine their columns into one result row. This is *why* the link table only stores `exercise_id` (a number) and not the name — the name lives once in `exercises` and the JOIN fetches it. If a link row had a dangling `exercise_id` (impossible here — FK + the add-exercise existence check), an INNER JOIN would silently *drop* that row; that's acceptable because it can't happen.
- **WHERE re.routine_id = ?** — only this routine's links. **Safe without another user_id check** because query 1 already proved the caller owns `routineId`, and this query filters only by that id.
- **ORDER BY re.id** — link rows are inserted in the order the user added exercises; ordering by the link's own auto-increment id preserves "add order."
- **Result:** an array of `{id, name, muscle_group, target_sets, target_reps}`. The handler then returns `{...routine, exercises}` — the routine's `id` and `name` spread in, plus the array.

Why two queries and not one big JOIN? A single `routines JOIN routine_exercises JOIN exercises` would repeat the routine's `name` on every exercise row, and — for a routine with **zero** exercises — return **zero rows**, making "empty routine" indistinguishable from "routine not found" (unless you switch to `LEFT JOIN` and then reconstruct). Two queries keeps each one simple and the empty case obvious.

10.4 INNER JOIN VS LEFT JOIN

- **INNER JOIN a ... JOIN b ON ...** — output rows only where a match exists in **both** tables. A left-side row with no match is **dropped**.
- **LEFT JOIN a LEFT JOIN b ON ...** — keep **every** left-side row; where `b` has no match, its columns come back `NULL`.

Rule of thumb: use `LEFT JOIN` when "the left row must still appear even if there is nothing on the right."

10.5 Workout history — LEFT JOIN + GROUP BY + COUNT in one round trip

GET /api/workouts:

```
SELECT w.id,
       w.date,
       r.name AS routine_name,
       COUNT(ws.id) AS set_count
FROM workouts w
LEFT JOIN routines r ON r.id = w.routine_id
LEFT JOIN workout_sets ws ON ws.workout_id = w.id
WHERE w.user_id = ?
GROUP BY w.id
ORDER BY w.date DESC, w.id DESC
```

Clause by clause:

- **FROM workouts w** — one row per workout to start.
- **LEFT JOIN routines r ON r.id = w.routine_id** — attach the routine's name. **LEFT** because `w.routine_id` is **NULL** for freestyle workouts; a plain **JOIN** would **drop every freestyle workout** from the history. With **LEFT JOIN**, a freestyle row survives and `r.name` (aliased `routine_name`) is **NULL** — which the frontend renders as "Freestyle".
- **LEFT JOIN workout_sets ws ON ws.workout_id = w.id** — attach the sets so we can count them. **LEFT** because a just-started workout has **no** sets yet and must still appear in history. This join *multiplies* rows: a workout with 3 sets now appears as 3 rows.
- **WHERE w.user_id = ?** — the ownership filter (bound to `req.userId`).
- **GROUP BY w.id** — collapse those multiplied rows back to **one per workout**. Everything not in **GROUP BY** must be either an aggregate or functionally determined by the grouping key. `w.date` and `r.name` are fine because we group by `workouts`' primary key (SQLite's "bare column" rule).
- **COUNT(ws.id) AS set_count** — `COUNT(column)` counts **non-NULL** values. For a workout with no sets, the **LEFT JOIN** produced one row with `ws.id = NULL`, so `COUNT(ws.id)` is 0 (not 1). Exactly the desired behaviour.
- **ORDER BY w.date DESC, w.id DESC** — newest first. `date` is only second-precision (`CURRENT_TIMESTAMP`), so two workouts started in the same second would tie; `w.id DESC` breaks the tie deterministically (higher id = created later).
- **Result:** `[{ id, date, routine_name, set_count }]`. The frontend renders this array **as-is**, no client-side sorting (Ch. 25).

10.6 The N+1 query problem — and how §10.5 avoids it

The naive way to build a history list: fetch the workouts, then **loop** and fire one "get sets for this workout" query per workout. For N workouts that is **1 + N queries** — the "N+1 problem." It gets slow, and worse, it scales with the data.

Query §10.5 does it in **one** query using the **LEFT JOIN + GROUP BY + COUNT** aggregate. The database does the counting; Node makes a single round trip. This is the standard cure for N+1: express the whole thing as one set-based query.

10.7 Workout detail — two queries, second JOIN s exercise names

GET /api/workouts/:id:

Query 1 — authorize + metadata:

```
SELECT w.id, w.date, w.routine_id, r.name AS routine_name
FROM workouts w
LEFT JOIN routines r ON r.id = w.routine_id
WHERE w.id = ? AND w.user_id = ?
```

`AND w.user_id = ?` is the ownership gate; no row → 404. **LEFT JOIN** again so a freestyle workout (**NULL** `routine_id`) still returns its row.

Query 2 — the sets, with human-readable exercise fields:

```
SELECT ws.id,
       ws.exercise_id,
       e.name AS exercise_name,
       e.muscle_group,
       ws.set_number,
       ws.reps,
       ws.weight
FROM workout_sets ws
JOIN exercises e ON e.id = ws.exercise_id
WHERE ws.workout_id = ?
ORDER BY ws.id
```

- **JOIN (not LEFT)** to `exercises` — every set has a valid `exercise_id` (enforced on insert and by the FK), so there is always a match; a plain **INNER JOIN** is correct and slightly clearer.
- **WHERE ws.workout_id = ?** — safe without a user check because query 1 proved ownership.
- **ORDER BY ws.id** — sets come back in the order they were logged (the set's own auto-increment id = insertion order).
- **Result:** `{ id, date, routine_id, routine_name, sets: [ {id, exercise_id, exercise_name, muscle_group, set_number, reps, weight} ] }`. A **flat** array — the frontend groups it by exercise for display (Ch. 25).

10.8 The write queries

```
-- POST /api/routines
INSERT INTO routines (user_id, name) VALUES (?, ?)
--           req.userId, name.trim()

-- POST /api/routines/:id/exercises (after ownership + existence checks)
INSERT INTO routine_exercises (routine_id, exercise_id, target_sets, target_reps)
VALUES (?, ?, ?, ?)
--           routineId, exercise_id, target_sets ?? null, target_reps ?? null

-- POST /api/workouts
INSERT INTO workouts (user_id, routine_id) VALUES (?, ?)
--           req.userId, routine_id ?? null
-- then read the row back for its DB-generated `date`:
SELECT id, routine_id, date FROM workouts WHERE id = ?

-- POST /api/workouts/:id/sets (after ownership + exercise-existence checks)
INSERT INTO workout_sets (workout_id, exercise_id, set_number, reps, weight)
VALUES (?, ?, ?, ?, ?)
```

- `.run(...)` returns `{ changes, lastInsertRowid }`. The handlers do `Number(info.lastInsertRowid)` (it is a `BigInt`) to build the response.
- `?? null` (nullish coalescing) turns a missing optional field into a real SQL `NULL`.
- After `INSERT INTO workouts`, a follow-up `SELECT` fetches the row so the response can include the `date` the database generated (the client doesn't send it).

11. API architecture — the complete reference

All 13 endpoints. Every route except `POST /api/signup` and `POST /api/login` requires the session cookie (`requireAuth` → `401 { "error": "Not authenticated" }` without one). Every failure response is `{ "error": "<message>" }`.

(A 14th route, `GET /` → `heLLo`, is a plain-text liveness probe on port 3000, not part of the API.)

11.1 Authentication

`POST /api/signup` VERIFIED

Auth	none
Body	<code>{ "username": string, "password": string }</code>
Validation	both present; <code>password.length >= 6</code>
Authorization	n/a
DB	<code>INSERT INTO users ; req.session.userId set(auto-login)</code>
Success	<code>201 { "id": number, "username": string }</code>
Errors	<code>400</code> missing field / short password · <code>409</code> username taken

```
curl -i -c jar -X POST http://localhost:3000/api/signup \  
  -H 'Content-Type: application/json' \  
  -d '{"username":"alice","password":"secret1"}'  
# HTTP/1.1 201 Created  
# Set-Cookie: connect.sid=s%3A...; Path=/; HttpOnly; SameSite=Lax  
# {"id":1,"username":"alice"}
```

`POST /api/login` VERIFIED

Auth	none
Body	<code>{ "username": string, "password": string }</code>
DB	<code>SELECT * FROM users WHERE username = ? ; bcrypt.compare ; on success req.session.userId set</code>
Success	<code>200 { "id": number, "username": string }</code>
Errors	<code>400</code> missing field · <code>401</code> "invalid username or password" (same message for unknown user and wrong password)

`POST /api/logout` VERIFIED

Auth	any (works with or without a session)
Body	none
Effect	<code>req.session.destroy() , res.clearCookie('connect.sid')</code>
Success	<code>200 { "ok": true }</code>

`GET /api/me` VERIFIED

Auth	required
DB	<code>SELECT id, username, created_at FROM users WHERE id = ?</code>
Success	<code>200 { "id", "username", "created_at" }</code>
Errors	<code>401</code> no session

```
curl -s -b jar http://localhost:3000/api/me  
# {"id":1,"username":"alice","created_at":"2026-09-06 09:00:04"}
```

11.2 Exercises

GET /api/exercises VERIFIED

Auth	required
DB	SELECT id, name, muscle_group FROM exercises ORDER BY muscle_group, name
Success	200 [{ "id", "name", "muscle_group" }] — 21 rows, deterministic order
Errors	401

```
curl -s -b jar http://localhost:3000/api/exercises
# [{"id":10,"name":"Barbell Row","muscle_group":"Back"},
# {"id":9,"name":"Conventional Deadlift","muscle_group":"Back"}, ... ]
```

11.3 Routines

POST /api/routines VERIFIED

Auth	required
Body	{ "name": string } (also-sent fields like user_id are ignored)
Validation	name non-empty after trim, ≤ 100 chars
Authorization	owner = req.userId (from session, never body)
DB	INSERT INTO routines (user_id, name)
Success	201 { "id", "name" }
Errors	400 blank/oversized name

GET /api/routines VERIFIED

Auth	required
DB	SELECT id, name FROM routines WHERE user_id = ? ORDER BY name
Success	200 [{ "id", "name" }] — the caller's routines only

GET /api/routines/:id VERIFIED

Auth	required
Validation	:id a positive integer (else 404)
Authorization	WHERE id = ? AND user_id = ?
DB	query 1 (routine) + query 2 (routine_exercises JOIN exercises)
Success	200 { "id", "name", "exercises": [{ "id", "name", "muscle_group", "target_sets", "target_reps" }] }
Errors	404 not found or not yours

```
curl -s -b jar http://localhost:3000/api/routines/1
# {"id":1,"name":"Push Day","exercises":[
# {"id":1,"name":"Barbell Bench Press","muscle_group":"Chest","target_sets":4,"target_reps":8}}]
```

POST /api/routines/:id/exercises VERIFIED

Auth	required
Body	{ "exercise_id": int>0, "target_sets"?: int>0, "target_reps"?: int>0 }
Validation	exercise_id positive int; targets optional positive int; exercise must exist
Authorization	routine WHERE id = ? AND user_id = ? before the write
DB	INSERT INTO routine_exercises (duplicates allowed)
Success	201 { "id", "routine_id", "exercise_id", "target_sets", "target_reps" }
Errors	400 bad body / unknown exercise_id · 404 routine not found / not yours

11.4 Workouts — logging

POST /api/workouts **VERIFIED**

Auth	required
Body	{ "routine_id"? : int>0 } — omit for freestyle
Validation	routine_id optional positive int
Authorization	if given, routine WHERE id = ? AND user_id = ? ; owner = req.userId
DB	INSERT INTO workouts (user_id, routine_id) ; read back for date
Success	201 { "id", "routine_id": number\ null, "date": string }
Errors	400 bad routine_id / not your routine ("routine_id does not exist")

```
# routine-based
curl -s -b jar -X POST http://localhost:3000/api/workouts \
  -H 'Content-Type: application/json' -d '{"routine_id":1}'
# {"id":5,"routine_id":1,"date":"2026-09-06 09:31:02"}

# freestyle
curl -s -b jar -X POST http://localhost:3000/api/workouts \
  -H 'Content-Type: application/json' -d '{}'
```

POST /api/workouts/:id/sets **VERIFIED**

Auth	required
Body	{ "exercise_id": int>0, "set_number": int>0, "reps": int>0, "weight": number>=0 } — all required
Validation	exercise_id / set_number / reps positive integers; weight a number ≥ 0 (0 allowed, decimals allowed, strings rejected); exercise must exist
Authorization	workout WHERE id = ? AND user_id = ? before the INSERT
DB	INSERT INTO workout_sets
Success	201 { "id", "workout_id", "exercise_id", "set_number", "reps", "weight" }
Errors	400 bad body / unknown exercise_id · 404 workout not found / not yours (:id non-numeric also → 404)

11.5 Workouts — history

GET /api/workouts **VERIFIED**

Auth	required
DB	the LEFT JOIN ... GROUP BY query of §10.5
Success	200 [{ "id", "date", "routine_name": string\ null, "set_count": number }] — newest first
Errors	401

GET /api/workouts/:id **VERIFIED**

Auth	required
Validation	:id positive integer (else 404)
Authorization	WHERE w.id = ? AND w.user_id = ?
DB	query 1 (metadata) + query 2 (workout_sets JOIN exercises)
Success	200 { "id", "date", "routine_id": number\ null, "routine_name": string\ null, "sets": [{ "id", "exercise_id", "exercise_name", "muscle_group", "set_number", "reps", "weight" }] }
Errors	404 not found / not yours

12. API request lifecycle

Three representative operations, every layer named. (Chapter 42 has full sequence diagrams for all of them; Chapter 41 tells the same story as a narrative.)

12.1 Create a routine (a mutation)

```
1 User types "Push Day", taps "Create routine"
2 React <form onSubmit> → e.preventDefault() (no browser full-page POST)
3   inFlight.current? yes → return | no → inFlight.current = true
4   name.trim() === '' ? → setError(new ApiError(400, 'Enter a routine name.')) STOP
5   setPending(true) → button disables, shows "Working..."
6   api.createRoutine("Push Day")
   → request('POST', '/api/routines', {name: "Push Day"})
   → fetch('/api/routines', {method: 'POST', headers: {'Content-Type': 'application/json'},
     body: '{"name": "Push Day"}', credentials: 'same-origin'})
7 Browser attaches Cookie: connect.sid=... (automatic)
8 nginx location /api/ → proxy_pass 127.0.0.1:3000
9 Express: express.json() → req.body = {name: "Push Day"}
   express-session → req.session = {userId: 7}
   routes/routines: router.use(requireAuth) → req.userId = 7 → next()
   POST /routines handler:
     nonEmptyString("Push Day", "name") → null (ok)
     db.prepare('INSERT INTO routines (user_id, name) VALUES (?, ?)')
       .run(7, "Push Day") → {lastInsertRowid: 4n}
     res.status(201).json({id: 4, name: "Push Day"})
10 nginx passes the 201 + JSON straight back
11 request(): res.ok → returns {id: 4, name: "Push Day"}
12 api.createRoutine resolves
13 CreateRoutineForm: setName(''); onCreated() ← onCreated === reload from useApi
14 useApi bumps its nonce → useEffect re-runs → GET /api/routines
15 → the new routine is in the fresh array → setState → <Routines> re-renders
16 finally: inFlight.current = false; setPending(false) → button re-enabled
```

The screen updates from **step 14's re-fetch**, not from step 13's local knowledge. The frontend's rule is "after a mutation, ask the server again" (Ch. 16, Ch. 25).

12.2 Log a workout set (a mutation with an ownership check)

```
1 User (on /workout/5) picks "Barbell Bench Press", reps 10, weight 60, taps "Log set"
2 SetForm onSubmit: preventDefault; inFlight guard
3 local validation: exerciseId > 0 ? validReps("10") > 10 ? validWeight("60") > 60 ?
4 nextSetNumber (derived): sets.filter(s => s.exercise_id === 1).length + 1 → e.g. 1
5 api.logSet("5", {exercise_id: 1, set_number: 1, reps: 10, weight: 60})
   → POST /api/workouts/5/sets
6 Express: requireAuth → req.userId = 7
   handler:
     parseId("5") → 5
     positiveInt(1) positiveInt(1) positiveInt(10) nonNegativeNumber(60) → all ok
     SELECT id FROM workouts WHERE id=5 AND user_id=7 → row?
       no → 404 {"error": "workout not found"} ← STOP, nothing written
       yes → continue
     SELECT id FROM exercises WHERE id=1 → exists? no → 400
     INSERT INTO workout_sets (workout_id, exercise_id, set_number, reps, weight)
       VALUES (5, 1, 1, 10, 60)
     res.status(201).json({id: ..., workout_id: 5, exercise_id: 1, set_number: 1, reps: 10, weight: 60})
7 SetForm: onLogged() === workout.reload() (from the parent WorkoutSession's useApi)
8 GET /api/workouts/5 → fresh {..., sets: [{..., exercise_name: "Barbell Bench Press", ...}]}
9 WorkoutSession re-renders → SetList groups the flat sets → the new set appears
10 SetForm keeps exerciseId/reps/weight (fast repeat); nextSetNumber recomputes → 2
```

The set is shown **only after step 8 confirms the server has it** (Ch. 24 §24.3).

12.3 View history (a query)

```
1 User taps "History" in the bottom nav → <Link to="/history"> → navigate('/history')
2 router.pushState + setPath('/history') → Shell matchPath → renders <History>
3 History: useApi(() => api.workouts(), [])
4 GET /api/workouts (cookie attached; requireAuth → userId 7)
5 SQL: workouts w LEFT JOIN routines LEFT JOIN workout_sets, GROUP BY w.id,
   ORDER BY w.date DESC, w.id DESC ($10.5)
6 res.json([ {id, date, routine_name, set_count}, ... ]) newest first
7 useApi: setState({data: [...], loading: false})
8 History renders the array AS-IS (no client sort):
   each → <Link to={`/history/${w.id}`}> routine_name || 'Freestyle'
   set_count + (set_count === 1 ? 'set': 'sets')
   formatDate(w.date) ← UTC → device-local
```


A query is **idempotent** — running it again changes nothing — so `useApi`'s `reload()` (and the retry button on errors) are always safe (Ch. 10.6 note; Ch. 16).

13. Validation

13.1 Two validations, two purposes

Input is validated **twice** — once in the browser, once on the server — and they are *not* redundant:

	Frontend validation	Backend validation
Purpose	UX — instant feedback, no wasted round trip	Correctness & security — the real gate
Bypassable?	trivially (disable JS, use <code>curl</code>)	no
Example	"Enter a routine name." shown the instant you submit blank	<code>nonEmptyString(name, 'name')</code> → 400
If it disagrees with the backend	the backend wins; the user just sees a slightly later error	—

The frontend deliberately does **not** mirror every backend rule (e.g. the 100-char routine name cap is server-only). It covers the *obvious* mistakes; the server covers *all* of them.

13.2 The backend helpers — `server/src/validation.js` **IMPLEMENTED**

Convention: each function returns `null` if the value is acceptable, or a human-readable **error string** if not. Routes chain them with `||` and return the first non-null as a 400:

```
const err = positiveInt(exercise_id, 'exercise_id')
  || optionalPositiveInt(target_sets, 'target_sets')
  || optionalPositiveInt(target_reps, 'target_reps');
if (err) return res.status(400).json({ error: err });
```

No schema library (like Zod or Joi). For rules this small, reading them inline is worth more than the indirection.

`parseId(raw) → number | null`

```
function parseId(raw) {
  const n = Number(raw);
  return Number.isInteger(n) && n > 0 ? n : null;
}
```

| Accepts | "12" → 12, "3" → 3 || Rejects | "abc" → null, "12.5" → null, "" → null (Number("") is 0), "-4" → null, "0" → null |

URL params (`req.params.id`) always arrive as **strings**. This converts and range-checks in one place. `null` → the route returns 404 (a non-numeric id addresses no resource).

`nonEmptyString(value, field, max = 100) → string | null`

Rejects: non-strings, "", whitespace-only (checked after `.trim()`), and strings longer than `max` characters (default 100). Used for `routes.name`.

`positiveInt(value, field) → string | null`

```
if (!Number.isInteger(value) || value <= 0) return `${field} must be a positive integer`;
```

Rejects: non-integers (10.5), zero, negatives, non-numbers ("10" — a string "10" is not `Number.isInteger`). Used for `exercise_id`, `set_number`, `reps`. The strictness matters: JSON bodies must send real numbers, not numeric strings.

`optionalPositiveInt(value, field) → string | null`

`undefined` OR `null` → `null` (fine, the field was omitted). Anything else → runs `positiveInt`. Used for `target_sets`, `target_reps`, and `routine_id` on `POST /workouts`.

`nonNegativeNumber(value, field) → string | null`

```
if (typeof value !== 'number' || !Number.isFinite(value) || value < 0)
  return `${field} must be a number >= 0`;
```

Accepts: 0 (bodyweight), 42.5 (decimals), any finite non-negative number. Rejects: negatives, NaN, Infinity, **strings** ("60" fails `typeof`). Used only for `workout_sets.weight`.

13.3 The frontend validators

Small, local, per-form:

- `Login.jsx`: `!username.trim() || !password` → `ApiError(400, 'Enter your username and password.')`.
- `Signup.jsx`: `!username.trim()` → "Choose a username."; `password.length < 6` → "Password must be at least 6 characters."

- `Routines.jsx` `CreateRoutineForm`: `name.trim() === ''` → "Enter a routine name."
- `RoutineDetail.jsx` `parseTarget(raw, label)`: `''` → not sent; non-integer or `≤ 0` → "{label} must be a whole number above 0."
- `SetForm.jsx` `validReps` / `validWeight`: reps must be `Number.isInteger &&`

`0` ; weight must be `Number.isFinite && >= 0` (empty → invalid).

- `WorkoutStart.jsx`: in routine mode, `Number(routineId)` must be a positive integer → "Choose a routine, or switch to freestyle."

Each constructs a client-side `ApiError` (status `400`) so the same error-display component renders it. Forms carry `noValidate` so the browser's own bubble validation is suppressed and *these* messages are the only ones shown (Ch. 35 §7).

14. Frontend architecture

14.1 The stack, and what each piece is

- **React 19** — a library for building UIs out of **components** (functions that return a description of UI). You describe *what the UI should look like for the current data*; React figures out the minimal DOM changes. (`react@^19.2.8`, `react-dom@^19.2.8`)
- **Vite 8** — the build tool. In dev it serves your source instantly with HMR; `vite build` compiles everything to static files. (`vite@^8.2.2`, `@vitejs/plugin-react@^6.1.0` — the plugin lets Vite understand JSX and enables React fast-refresh.)
- **JSX** — the `<Tag prop={value}>child</Tag>` syntax inside `.jsx` files. It is not HTML; it compiles to `React.createElement(...)` calls. `className` (not `class`), `{expression}` for dynamic values, `{list.map(...)}` for lists.
- **No other runtime dependencies.** No router library, no state library, no UI kit, no TypeScript, no icon set. `client/ARCHITECTURE.md` records why (Ch. 39).

14.2 Entry point — `index.html` → `main.jsx` → `App.jsx`

`client/index.html` is almost empty:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

The server sends this shell. `main.jsx` takes over:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>
);
```

- `createRoot(el).render(...)` hands `<div id="root">` to React; from here down, every visible pixel is drawn by JavaScript.
- `<StrictMode>` is a development-only wrapper that intentionally *double-invokes* effects and renders to surface bugs (impure renders, missing effect cleanup). It has **no effect in a production build**. It is why, in dev, `GET /api/me` and `GET /api/exercises` fire twice on load — expected, not a bug (Ch. 35 §note).

14.3 `App.jsx` — the shell

```
export default function App() {
  return (
    <Router>                {/* provides path + navigate */}
      <AuthProvider>        {/* provides user + status + login/signup/logout */}
        <Shell />           {/* picks a page for the current URL + auth state */}
      </AuthProvider>
    </Router>
  );
}
```

`<Router>` must wrap `<AuthProvider>` because `AuthProvider` calls `useNavigate()`.

`<Shell>`:

1. `status === 'loading'` → full-screen `<Spinner>` (blocks the whole app until the first `GET /api/me` resolves, so a logged-in user never sees a flash of the login page on refresh).
2. Walk the `ROUTES` table, `matchPath(route.path, currentPath)` — first match wins.
3. Guard: protected route + anonymous → `<Redirect to="/login">`; public route (login/signup) + authenticated → `<Redirect to="/">`.
4. Render the matched component; if none matched → `<NotFound>`.
5. If authenticated, wrap the page in `<div class="with-nav">` and also render `<Nav>` (the bottom tab bar).

14.4 The dependency graph

```
main.jsx
└─ App.jsx
  └─ styles/tokens.css, styles/global.css (imported for side effect)
  └─ router.jsx → (react)
  └─ auth.jsx → api.js, router.jsx, (react)
  └─ components/Nav.jsx → router.jsx, auth.jsx
  └─ components/Spinner.jsx
  └─ pages/*
    └─ (every page) → hooks/useApi.js → api.js
    └─ (every page) → components/* (Button, Input, Card, Spinner,
      └─ ErrorMessage, EmptyState)
    └─ auth pages → auth.jsx, format.js (describeError)
    └─ RoutineDetail, SetForm → components/ExerciseSelect → components/Select
    └─ WorkoutSession → pages/SetForm, pages/SetList
    └─ WorkoutDetail → pages/SetList
    └─ History, WorkoutDetail, WorkoutSession → format.js (formatDate)

api.js → (nothing – it only uses the global fetch)
```

`api.js` is the single leaf that touches the network. Everything else is above it.

14.5 Folder conventions

- `src/pages/` — one file per **screen**. Each imports its own co-located `.css`. `Workout.jsx` is a tiny dispatcher (`WorkoutStart` vs `WorkoutSession`); `SetForm.jsx` and `SetList.jsx` live here too because they are workout-screen building blocks, not general components.
- `src/components/` — **reusable presentational** pieces. Each has one `.css`.
- `src/hooks/` — custom hooks (`useApi`).
- `src/styles/` — `tokens.css` (design variables) + `global.css` (reset).
- `.css` files are plain global CSS imported for their side effect; class names are unique enough by convention that there is no CSS-Modules or CSS-in-JS machinery.

14.6 The design system — `styles/tokens.css`

Every colour, space, radius and font size is a **CSS custom property** on `:root`:

```
:root {
  --c-bg: #0f172a; --c-surface: #1e293b; --c-text: #f1f5f9;
  --c-primary: #22c55e; --c-danger: #f87171; --c-focus: #38bdf8;
  --s-1: 4px; --s-2: 8px; --s-3: 12px; --s-4: 16px; --s-5: 24px; --s-6: 32px;
  --r-sm: 6px; --r-md: 10px; --r-lg: 16px;
  --t-sm: .85rem; --t-base: 1rem; --t-lg: 1.25rem; --t-xl: 1.6rem;
  --content-max: 560px; /* mobile-first: cap width on desktop */
  --tap-min: 44px; /* minimum touch target (Apple/Google guidance) */
}
```

Components reference `var(--...)` and never hard-code values, so a full restyle is an edit to this one file. Two tokens encode product decisions: `--content-max: 560px` (the app is a phone-width column, centred on desktop — Ch. 22 §22.11) and `--tap-min: 44px` (every button/link is at least a comfortable thumb target).

`global.css` is a small reset plus one important rule: text-like form controls get `appearance: none` (so custom CSS actually applies on mobile — Ch. 35 §3), while radios and checkboxes keep native rendering. And `:focus-visible { outline: 2px solid var(--c-focus) }` — a visible keyboard focus ring on every interactive element.

15. React fundamentals, taught through this project

For each concept: **what it means** → **why React has it** → **where this project uses it** → **what it prevents** → **what happens internally**.

15.1 Components & JSX

- **What:** a component is a function returning JSX. `<Button pending>Log set</Button>` calls `Button({pending: true, children: 'Log set'})`.
- **Why:** composition. Build a screen from small, independently understandable pieces.
- **Here:** 10 page components + 8 reusable components + a few in-file helpers (`CreateRoutineForm`, `AddExerciseForm`, `Shell`, `NotFound`).
- **Internally:** JSX compiles to `React.createElement` calls producing a tree of plain objects (the "virtual DOM"). React diffs the new tree against the last one and applies the minimal real-DOM changes.

15.2 Props

- **What:** inputs passed from a parent, read-only inside the child.
- **Why:** data flows **down**; the child stays reusable and ignorant of context.
- **Here:** `<SetForm workoutId={id} exercises={library.data} sets={sets} exerciseId={exerciseId} onExerciseChange={setExerciseId} onLogged={workout.reload} />` — data props (`sets`, `exercises`) plus **callback props** (`onExerciseChange`, `onLogged`). Behaviour flows **up**: the child doesn't know what "logged" should do; it calls the function the parent handed it (`workout.reload`).

15.3 State — `useState`

- **What:** `const [x, setX] = useState(initial)`. `x` is this render's value; `setX(next)` schedules a re-render with the new value.
- **Why:** a component needs to *remember* things between renders and trigger updates when they change.
- **Here:** form field values (`useState('')`), `pending` / `submitting` flags, `error` objects, `status` in `AuthProvider`, `path` in `Router`, `{data,error,loading}` in `useApi`.
- **Prevents:** the "stale DOM" bug of imperative UIs — you never "find the element and update its text"; you change state and React re-derives the DOM.
- **Internally:** React stores state per component instance, in order of the `useState` calls (this is why hooks must not be called conditionally). `setX(next)` does nothing if `Object.is(x, next)`.

15.4 Effects — `useEffect`

- **What:** `useEffect(fn, deps)` runs `fn` **after** the render is painted, and again whenever a value in `deps` changed. `fn` may return a **cleanup** function, run before the next effect and on unmount.
- **Why:** to synchronise with things *outside* React — network, timers, event listeners, subscriptions.
- **Here:**
 - `AuthProvider`: one effect (`deps: []`) fires `GET /api/me` on mount; another registers the global 401 handler.
 - `Router`: an effect adds a `window` `popstate` listener; cleanup removes it.
 - `useApi`: an effect runs the fetcher whenever `[...deps, nonce]` changes; cleanup sets a `cancelled` flag so a late response from an unmounted component is ignored.
 - `Redirect`: an effect calls `navigate(to)` (navigating *during render* would be an illegal side effect).
- **Prevents:** doing side effects during render (which React may run multiple times or abandon).

15.5 Memoised derived values — `useMemo`

- **What:** `const v = useMemo(() => compute(a, b), [a, b])` — recompute `v` only when `a` or `b` changed; otherwise reuse last render's `v`.
- **Why:** skip expensive recomputation; also keep a stable reference for a child's `deps`.
- **Here:**
 - `Exercises.jsx`: `filtered = useMemo(() => exercises.filter(matches search), [exercises, search])`.
 - `SetForm.jsx`: `nextSetNumber = useMemo(() => sets.filter(s => s.exercise_id === chosen).length + 1, [sets, exerciseId])`.
 - `ExerciseSelect.jsx`: `groups = useMemo(() => group exercises by muscle, [exercises])`.
- **Key idea:** these are **derived data**, never stored in `useState`. Storing them would create a second copy that can drift from its source (Ch. 16).

15.6 Refs — `useRef`

- **What:** `const r = useRef(initial)`. `r.current` is a mutable box that **persists across renders** and — crucially — **changing it does not trigger a re-render**.
- **Why:** for values that aren't UI (a DOM node, a timer id, a "request in flight" flag) or that must change *synchronously*.
- **Here:** every mutation form has `const inFlight = useRef(false)`. See Ch. 24 for why this — and not `disabled={pending}` — is what actually stops a double-submit.

15.7 Context

- **What:** `createContext()` + `<Provider value={...}>` makes a value available to *every* descendant without passing it through props at each level ("prop drilling").
- **Why:** for truly app-wide values.
- **Here:** two — `RouterContext` ({path, navigate}) and `AuthContext` ({user, status, login, signup, logout}). Consumed via `useRouter()` / `useNavigate()` and `useAuth()`. Both hooks `throw` if used outside their provider — a clear error instead of a confusing `null`.

15.8 Controlled inputs

- **What:** the input's `value` comes from state; `onChange` updates that state. React "controls" the field — the DOM never holds a value React doesn't know.

```
<input value={username} onChange={(e) => setUsername(e.target.value)} />
```

- **Why:** one source of truth; you can validate, transform, or reset the field from code.
- **Here:** every text field, every `<select>` (`ExerciseSelect`, the routine picker), every radio (`WorkoutStart` 's mode). After a successful "add exercise", `setExerciseId('')`; `setSets('')`; `setReps('')` clears the form — possible *because* it's controlled.

15.9 Rendering, re-rendering, conditional rendering, lists & keys

- **Render** = React calls the component function to get JSX. **Re-render** = it calls it again because state/props/context changed. Re-rendering is cheap; React only touches the DOM where the output actually differs.
- **Conditional:** `{loading && <Spinner/>}`, `{error ? <A/> : <B/>}`, early `return`. `Exercises.jsx` uses **four** mutually exclusive conditions (loading / error / empty-library / no-search-match / list).
- **Lists:** `{items.map(x => <Row key={x.id} ... />)}`. The **key** must be stable and unique among siblings so React can match a re-rendered list item to its previous instance. `SetList` keys rows by `s.id` (the set's DB id). `RoutineDetail` and `WorkoutSession` 's "plan chips" key by ``${ex.id}-${i}`` because **duplicate exercises are allowed** so `ex.id` alone isn't unique; index-in-key is acceptable there because those lists only ever *append*.

15.10 Derived data — the recurring lesson

`filteredExercises` is **not** state. It is `exercises` (server state) filtered by `search` (UI state), recomputed every render. If it were its own `useState`, you would have two representations of the same fact, and the moment `exercises` refreshed and you forgot to re-filter, the screen would lie. A pure function of the current inputs *cannot* go stale. Same for `nextSetNumber` and `groupedSets`.

16. State management

No Redux, no Zustand, no React Query. The project separates state into **three kinds** and handles each with the plainest tool.

16.1 Server state

Data that lives in the database and is *fetched*. Examples: `routines`, `workouts`, `sets`, `exercises`, `user`.

- **Held via `useApi`** (`{data, error, loading, reload}`). The component keeps the last server response in state and re-fetches when needed.
- **After a mutation, re-fetch** — do not patch the local copy. `useApi` exposes `reload()`; forms call it through an `onCreated` / `onAdded` / `onLogged` callback prop. The server stays the single source of truth. A re-fetch *cannot be wrong*; a hand-maintained local copy can. At ~10 users the extra GET is free.
- **The one exception is auth state**, held in `AuthContext` rather than `useApi`, because it is app-wide and has extra transitions (login/logout).

16.2 UI state

Ephemeral, browser-only, never sent to the server. Examples: the search term (`Exercises`), the selected exercise (`WorkoutSession` lifts `exerciseId` so the chips and the dropdown share it), form field values, `pending` / `submitting` flags, `error` objects, the current `path` (`Router`).

- **Held via `useState`**, as locally as possible. `exerciseId` is lifted only one level (to `WorkoutSession`) because two children need it; `reps` / `weight` stay inside `SetForm` because nothing else cares.

16.3 Derived state

Computed from server + UI state on every render, **never stored**:

Value	Derived from	Where
<code>filtered</code> (exercises)	<code>exercises</code> + <code>search</code>	<code>Exercises.jsx</code>
<code>nextSetNumber</code>	<code>sets</code> + <code>exerciseId</code>	<code>SetForm.jsx</code>
<code>groups</code> (exercises by muscle)	<code>exercises</code>	<code>ExerciseSelect.jsx</code>
grouped sets by exercise	<code>sets</code> prop	<code>SetList.jsx</code> (<code>groupByExercise</code> , not even memoised — it's cheap)
<code>isFreestyle</code>	<code>data.routine_id == null</code>	<code>WorkoutDetail.jsx</code>
<code>effectiveMode</code>	<code>mode</code> + <code>noRoutines</code>	<code>WorkoutStart.jsx</code>
route <code>match</code> / <code>params</code>	<code>path</code> + <code>ROUTES</code>	<code>App.jsx</code> <code>Shell</code>

Storing any of these in `useState` would introduce a second source of truth that must be manually kept in sync — the exact bug class React exists to remove.

17. The custom router — `src/router.jsx`

~90 lines, zero dependencies. Understanding it teaches how *any* client-side router works.

17.1 The problem it solves

A **Single-Page Application (SPA)** loads one HTML page and then never loads another. But the URL bar should still change (so Back/Forward/bookmarks/deep links work), and different URLs should show different screens. The **History API** is the browser feature that makes this possible.

17.2 The History API primitives

- `window.location.pathname` — the current path (`/history/5`).
- `history.pushState(state, '', url)` — change the URL bar **and add a history entry**, *without a page reload*.
- `history.replaceState(state, '', url)` — same, but replace the current entry (used for redirects, so Back doesn't return to the redirecting URL).
- **popstate event** — fired by the browser when the user presses **Back or Forward**. (It is *not* fired by `pushState` / `replaceState` — your own code must update state itself after calling those.)

17.3 `<Router>`

```
export function Router({ children }) {
  const [path, setPath] = useState(() => window.location.pathname);

  useEffect(() => {
    const onPop = () => setPath(window.location.pathname); // Back/Forward
    window.addEventListener('popstate', onPop);
    return () => window.removeEventListener('popstate', onPop);
  }, []);

  const navigate = useCallback((to, { replace = false } = {}) => {
    if (to === window.location.pathname) return; // no-op if already there
    window.history[replace ? 'replaceState' : 'pushState']({}, '', to);
    setPath(to); // pushState doesn't fire popstate
    window.scrollTo(0, 0); // new "page" starts at the top
  }, []);

  return <RouterContext.Provider value={{ path, navigate }}>{children}</RouterContext.Provider>;
}
```

- Holds the current `path` in state. Anything that reads `useRouter().path` re-renders when it changes — that's what swaps the screen.
- `navigate()` is the programmatic move: update the URL bar, then update `path` state (and scroll to top). `replace: true` for redirects.
- The `popstate` listener handles Back/Forward by re-reading the URL into state.

17.4 `matchPath(pattern, path)` — route matching & params

```
export function matchPath(pattern, path) {
  const pp = pattern.split('/'); // "/routines/:id" → [ "", "routines", ":id" ]
  const ap = path.split('/');    // "/routines/42" → [ "", "routines", "42" ]
  if (pp.length !== ap.length) return null; // different segment count → no match
  const params = {};
  for (let i = 0; i < pp.length; i++) {
    if (pp[i].startsWith(':')) params[pp[i].slice(1)] = decodeURIComponent(ap[i]); // :id → params.id
    else if (pp[i] !== ap[i]) return null; // a literal segment differs → no match
  }
  return params; // {} for a match with no params
}
```

`matchPath('/routines/:id', '/routines/42') → { id: "42" }`. Note **id is a string** (it came from a URL). `matchPath('/routines/:id', '/exercises') → null` (segment counts differ).

17.5 The route table & how a param reaches a component — App.jsx

```
const ROUTES = [
  { path: '/login',      component: Login,      public: true },
  { path: '/signup',    component: Signup,      public: true },
  { path: '/',          component: Dashboard },
  { path: '/exercises', component: Exercises },
  { path: '/routines',  component: Routines },
  { path: '/routines/:id', component: RoutineDetail },
  { path: '/workout',   component: Workout },
  { path: '/workout/:id', component: Workout },
  { path: '/history',   component: History },
  { path: '/history/:id', component: WorkoutDetail },
];
// Shell: find first match, then:
const View = match ? match.route.component : NotFound;
const params = match ? match.params : {};
return <View {...params} />; // {id: "42"} spread as a prop
```

So `RoutineDetail({ id })` receives `id = "42"`. It passes that straight to `api.routine(id) → GET /api/routines/42`. Because `id` is in the URL and drives the fetch, a **hard refresh of `/routines/42` behaves exactly like navigating there** — nothing depends on prior React state (Ch. 25 `URL` carries the state).

17.6 `<Link>` and `<Redirect>`

```
export function Link({ to, children, ...rest }) {
  const navigate = useNavigate();
  return (
    <a href={to} onClick={(e) => {
      if (e.metaKey || e.ctrlKey || e.shiftKey || e.button !== 0) return; // let "open in new tab" work
      e.preventDefault(); // stop the browser's full-page navigation
      navigate(to);
    }} {...rest}>{children}</a>
  );
}
```

A real `<a href>` (so it's keyboard-focusable, right-clickable, and shows the URL on hover), but a left-click is intercepted and turned into an in-app `navigate`.

```
export function Redirect({ to }) {
  const navigate = useNavigate();
  useEffect(() => { navigate(to, { replace: true }); }, [to, navigate]);
  return null;
}
```

Navigates in an **effect** (never during render). `replace: true` so the guarded URL doesn't pollute history.

17.7 Browser Back / Forward / refresh — end to end

- **Back:** browser pops the history stack → fires `popstate` → Router's listener `setPath(location.pathname) → Shell` re-matches → previous screen renders. **VERIFIED** on-device.
- **Forward:** same mechanism in reverse.
- **Hard refresh of `/history/5`:** the browser asks the server for `/history/5`; nginx has no such file → `try_files ... /index.html` serves the SPA shell → React boots → Router initial state `path = '/history/5'` → renders `WorkoutDetail` with `id="5"`. This is why the nginx **SPA fallback** is mandatory (Ch. 29).

17.8 Why not `react-router`

`react-router` is the industry-standard library. This project hand-rolls ~90 lines instead because: (1) it's one concept to *learn* rather than an API to *use*; (2) zero dependencies and zero bundle cost; (3) 10 static routes don't need nested layouts, loaders, data APIs, or lazy boundaries. **If V2 needs those** (e.g. code-splitting per route, or route-level data loading), `react-router` would replace `router.jsx` and `App.jsx`'s `Shell` matching — the page components and `useApi` calls would be unaffected.

17.9 Documentation note

`router.jsx`'s header comment says "~~9 static routes"; the table has 10 entries (the two `/workout` variants both map to `Workout`). Cosmetic; the code is correct.

18. The API client — `src/api.js`

The **only** module that calls `fetch`. Everything else asks it.

18.1 `ApiError`

```
export class ApiError extends Error {
  constructor(status, message) {
    super(message || `Request failed (${status})`);
    this.name = 'ApiError';
    this.status = status;
  }
}
```

One error type. Callers branch on `err.status` (a stable contract), never on `err.message` (wording can change). `status === 0` is a special value meaning "the request never got a response" — offline, server down, DNS failure — as opposed to a real HTTP status like `404`.

18.2 `request(method, path, body)`

```
async function request(method, path, body) {
  let res;
  try {
    res = await fetch(path, {
      method,
      headers: body === undefined ? {} : { 'Content-Type': 'application/json' },
      body: body === undefined ? undefined : JSON.stringify(body),
      credentials: 'same-origin',
    });
  } catch {
    throw new ApiError(0, 'Network error – is the server running?');
  }

  const text = await res.text();
  const data = text ? JSON.parse(text) : null;

  if (!res.ok) {
    if (res.status === 401 && path !== '/api/me') onUnauthorized();
    throw new ApiError(res.status, data?.error);
  }
  return data;
}
```

- **fetch only rejects on a network-level failure** — never on `4xx / 5xx`. A `404` still *resolves*; you check `res.ok` (true for `200 – 299`) yourself. Hence two error paths: the `catch` ($\rightarrow$ `ApiError(0)`) and the `!res.ok` branch ($\rightarrow$ `ApiError(res.status, body.error)`).
- **credentials: 'same-origin'** — attach cookies for same-origin requests. Since dev (Vite proxy) and prod (nginx) both make `/api` same-origin, this is enough (and slightly safer than `'include'`).
- **Only sets Content-Type: application/json when there's a body** — a bodyless `POST /api/logout` sends no content-type.
- **Empty body handling:** `text ? JSON.parse(text) : null` — tolerates a `204` or empty response.

18.3 HTTP error vs network error — where each originates

	Network error (status 0)	HTTP error (status 4xx/5xx)
Meaning	the request never reached a server that answered	a server received, understood, and refused the request
Cause	offline, server process down, wrong host, DNS	validation failed (<code>400</code>), no session (<code>401</code>), not found / not yours (<code>404</code>), conflict (<code>409</code>), server bug (<code>500</code>)
In code	<code>fetch rejects</code> $\rightarrow$ <code>catch</code> $\rightarrow$ <code>ApiError(0)</code>	<code>fetch resolves</code> , <code>res.ok === false</code> $\rightarrow$ <code>ApiError(status, body.error)</code>
User message	"Could not reach the server. Check your connection..."	status-specific (Ch. 26)

18.4 The global 401 handler

```
let onUnauthorized = () => {};  
export function setUnauthorizedHandler(fn) { onUnauthorized = fn; }  
// ...in request(): if (res.status === 401 && path !== '/api/me') onUnauthorized();
```

`AuthProvider` registers a handler that clears the user and navigates to `/login`. So if the session expires while the app is open, *any* API call's `401` triggers a clean logout — no page has to handle "session expired" itself.

`GET /api/me` is exempt. That endpoint's whole job is to *report* whether you are logged in; a `401` there is an expected answer, not an expiry. Without the exemption, a logged-out visitor hitting `/signup` would be bounced to `/login` on every load (the initial `me()` `401` would fire the handler). This exemption was a bug fix during development (Ch. 35 §4).

18.5 The named helpers

One thin function per endpoint — the single readable list of everything the frontend can ask for:

```
export const api = {
  me: () => request('GET', '/api/me'),
  signup: (u,p) => request('POST', '/api/signup', { username:u, password:p }),
  login: (u,p) => request('POST', '/api/login', { username:u, password:p }),
  logout: () => request('POST', '/api/logout'),
  exercises: () => request('GET', '/api/exercises'),
  routines: () => request('GET', '/api/routines'),
  routine: (id) => request('GET', `/api/routines/${id}`),
  createRoutine: (name) => request('POST', '/api/routines', { name }),
  addRoutineExercise: (rid, payload) => request('POST', `/api/routines/${rid}/exercises`, payload),
  workouts: () => request('GET', '/api/workouts'),
  workout: (id) => request('GET', `/api/workouts/${id}`),
  startWorkout: (routineId) =>
    request('POST', '/api/workouts', routineId ? { routine_id: routineId } : {}),
  logSet: (wid, payload) => request('POST', `/api/workouts/${wid}/sets`, payload),
};
```

`startWorkout(undefined)` sends `{}` → the server creates a freestyle workout. `startWorkout(3)` sends `{ routine_id: 3 }`. Callers pass a real `Number` (`WorkoutStart` does `Number(routineId)` first), because the backend's `positiveInt` rejects the string `"3"`.

19. The `useApi` hook — `src/hooks/useApi.js`

19.1 What it is

```
export function useApi(fetcher, deps = []) {
  const [state, setState] = useState({ data: null, error: null, loading: true });
  const [nonce, setNonce] = useState(0);
  const reload = useCallback(() => setNonce((n) => n + 1), []);

  useEffect(() => {
    let cancelled = false;
    setState((s) => ({ ...s, loading: true, error: null }));
    fetcher()
      .then((data) => { if (!cancelled) setState({ data, error: null, loading: false }); })
      .catch((error) => { if (!cancelled) setState({ data: null, error, loading: false }); });
    return () => { cancelled = true; };
  }, [...deps, nonce]); // eslint-disable-line react-hooks/exhaustive-deps

  return { ...state, reload };
}
```

```
const { data, error, loading, reload } = useApi(() => api.routines(), []);
```

- `fetcher` — a function returning a promise (usually an inline arrow like `() => api.workout(id)`).
- `deps` — like `useEffect` `deps`: the fetch re-runs when they change (e.g. `[id]` on `RoutineDetail` — navigate to a different routine, re-fetch).
- `reload()` — bumps an internal `nonce`, which is in the effect's dep array, forcing a re-fetch. Called after a mutation.

19.2 The three (four) states every screen renders

```
mount      → { loading: true,  data: null,  error: null }
fetch ok   → { loading: false, data: [...],  error: null }
fetch fails → { loading: false, data: null,   error: ApiError }
reload()   → briefly { loading: true, data: <kept or null> } then ok/fail again
```

Pages render from this shape, in order:

```
{loading && <Spinner ... />}
{!loading && error && <ErrorMessage error={error} onRetry={reload} />}
{!loading && !error && data && data.length === 0 && <EmptyState ... />}
{!loading && !error && data && data.length > 0 && <ul>...</ul>}
```

Some screens keep last render's `data` visible while `loading` is `true` on a reload (`WorkoutSession`: `if (workout.loading && !workout.data)` — the big spinner shows only on the *first* load, so logging a set doesn't blank the screen). `WorkoutDetail` does the same (`if (loading && !data)`).

19.3 The `cancelled` flag

```
let cancelled = false;
fetcher().then(d => { if (!cancelled) setState(...); });
return () => { cancelled = true; };
```

If the component unmounts (or `deps` change) while a fetch is in flight, the cleanup sets `cancelled = true`, so the late `.then` / `.catch` does **not** call `setState` on a component that no longer exists. This avoids a stale response overwriting fresh state (e.g. navigate `/routines/1` → `/routines/2` quickly; request 1's response must not win).

19.4 Why `fetcher` is not in the dep array

Callers pass a **new inline arrow every render** (`() => api.routine(id)`). Putting `fetcher` in `deps` would re-run the effect on every render (infinite loop-ish). The real triggers are the caller's `deps` plus `nonce`; the `eslint-disable` line is a deliberate, documented choice.

20. The auth context — `src/auth.jsx`

20.1 The core idea: "the cookie is the source of truth"

The session cookie is `httpOnly` — **JavaScript cannot read it**. So the frontend can never know "am I logged in?" by inspecting a variable. It must **ask the server**: `GET /api/me`. It does this once, on load. That single request is why a refresh keeps you logged in — the state is *rebuilt from the cookie every time*, never persisted in JS or `localStorage`.

```
app loads → AuthProvider useEffect → api.me()
    └─ resolves {id, username, ...} → setUser(u); setStatus('authenticated')
    └─ rejects (401) → setUser(null); setStatus('anonymous')
```

20.2 The state machine

```
const [user, setUser] = useState(null);
const [status, setStatus] = useState('loading'); // 'loading' | 'authenticated' | 'anonymous'
```

- **loading** — the initial `GET /api/me` hasn't resolved. `App.jsx`'s `Shell` shows a full-screen spinner and renders **nothing else**, so a logged-in user never sees the login page flash.
- **authenticated** — `user` is set.
- **anonymous** — no valid session.

20.3 The two effects

```
// 1. establish auth state on mount
useEffect(() => {
  let cancelled = false;
  api.me()
    .then((u) => { if (!cancelled) { setUser(u); setStatus('authenticated'); } })
    .catch(() => { if (!cancelled) { setUser(null); setStatus('anonymous'); } });
  return () => { cancelled = true; };
}, []);

// 2. one global reaction to a 401 from ANY api call
useEffect(() => {
  setUnauthorizedHandler(() => {
    setUser(null); setStatus('anonymous');
    navigate('/login', { replace: true });
  });
}, [navigate]);
```

20.4 `login / signup / logout`

```
const login = useCallback(async (u, p) => {
  const user = await api.login(u, p);
  setUser(user); setStatus('authenticated');
  navigate('/', { replace: true });
}, [navigate]);

const signup = useCallback(async (u, p) => {
  const user = await api.signup(u, p); // backend auto-logs-in
  setUser(user); setStatus('authenticated');
  navigate('/', { replace: true });
}, [navigate]);

const logout = useCallback(async () => {
  try { await api.logout(); } catch { /* clear locally regardless */ }
  setUser(null); setStatus('anonymous');
  navigate('/login', { replace: true });
}, [navigate]);
```

- All `useCallback` (stable identity → safe in a child's deps).
- `navigate(..., { replace: true })` — after login you shouldn't be able to press Back into the login page.
- `logout` clears local state **even if the network call fails** — the user asked to leave; honour it.
- The `Login / Signup` page components call these, `catch` the `ApiError`, and render a message; on success the provider navigates away and the page unmounts.

20.5 useAuth

```
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
```

Returns { user, status, login, signup, logout }. Used by `Shell` (routing guard), `Nav` (logout button, username), `Dashboard` (`user.username`), and the auth pages.

20.6 Refresh, precisely

There is **no** auth state in `localStorage`, in a JS variable that survives reload, or in the URL. On every page load `AuthProvider` mounts fresh, `status` starts `'loading'`, `api.me()` fires, the browser attaches the `httpOnly` cookie automatically, and the server's answer sets the state. A refresh is just this happening again. **VERIFIED** on-device: deep-link `/exercises`, hard refresh, still authenticated.

21. Component-by-component walkthrough

The 8 reusable components in `src/components/`. Each is small and presentational.

21.1 Button.jsx

```
export default function Button({ pending=false, pendingLabel='Working...',
  variant='primary', className='', disabled=false, children, ...rest }) {
  return (
    <button className={`btn btn-${variant} ${className}`.trim()}
      disabled={pending || disabled}
      aria-busy={pending || undefined}
      {...rest}>
      {pending ? pendingLabel : children}
    </button>
  );
}
```

- **Responsibility:** a button that understands "an async action is running."
- **Props:** `pending` (disables + swaps label + `aria-busy`), `pendingLabel` ("Starting...", "Logging..."), `variant` (`primary` / `secondary` / `danger` → a CSS class), `className` (merged, not overwritten — see Ch. 35 §2), plus native `<button>` props via `...rest` (`onClick`, `type`).
- **Used:** everywhere there is an action. `pending` is the *first line* of double-submit defence (Ch. 24).

21.2 Input.jsx

```
export default function Input({ label, error, hint, ...rest }) {
  const id = useId();
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      <input id={id} aria-invalid={error ? true : undefined}
        aria-describedby={([hintId, errId].filter(Boolean).join(' ') || undefined)}
        {...rest} />
      {hint && <span id={hintId} className="field-hint">{hint}</span>}
      {error && <span id={errId} className="field-error">{error}</span>}
    </div>
  );
}
```

- **Responsibility:** a labelled text input with optional hint and error text.
- `useId()` generates a unique id so `<label htmlFor>` ↔ `<input id>` are wired — the single biggest form-accessibility win (screen readers announce the label; tapping the label focuses the field).
- `aria-invalid` and `aria-describedby` connect the error/hint to the field for assistive tech.
- **Used:** every text/number field in the app.

21.3 Select.jsx

A labelled **native** `<select>` (with a decorative ▼ and the same label/ `useId` pattern as `Input`). Native because on a phone the OS's own scrollable picker beats any custom dropdown. `children` are `<option>` / `<optgroup>`. Used by `ExerciseSelect` and `WorkoutStart`'s routine picker.

21.4 Card.jsx

```
export default function Card({ as: Tag = 'div', className='', children, ...rest }) {
  return <Tag className={`card ${className}`} {...rest}>{children}</Tag>;
}
```

A surface panel. `as` lets a card be a `<form>` or `<li>` without a new component — `<Card as="form" onSubmit={...}>` is used by every form in the app.

21.5 Spinner.jsx

`{ full=false, label='Loading...' }`. `full` centres it in the viewport (the initial auth check, route transitions); otherwise it's an inline loading chunk for one section. `role="status"` so screen readers announce it.

21.6 ErrorMessage.jsx

```
export default function ErrorMessage({ error, onRetry }) {
  if (!error) return null;
  return (
    <div className="error-box" role="alert">
      <p>{describeError(error)}</p>
      {onRetry && <button className="error-retry" onClick={onRetry}>Try again</button>}
    </div>
  );
}
```

- Renders a page-level failure banner. `role="alert"` announces it immediately.
- `describeError` (from `format.js`) turns `error.status` into a sentence (Ch. 26).
- If `onRetry` is passed (it's usually `useApi`'s `reload`), shows a "Try again" button.

21.7 EmptyState.jsx

`{ title, children, action }` → a centred "there is nothing here (and that's fine)" panel with an optional next action. Used for "No routines yet" + "Create your first routine", "No workouts yet" + "Start a workout", "No sets logged", "Routine not found", "Workout not found". Distinct from `ErrorMessage` — an empty list is *valid data*, not a failure (Ch. 26 §26.6).

21.8 ExerciseSelect.jsx

```
export default function ExerciseSelect({ exercises, value, onChange, label='Exercise' }) {
  const groups = useMemo(() => {
    const byMuscle = new Map();
    for (const e of exercises) { (byMuscle.get(e.muscle_group||'Other') ?? []).push(e); }
    return [...byMuscle.entries()].sort((a,b) => a[0].localeCompare(b[0]));
  }, [exercises]);
  return (
    <Select label={label} value={value} onChange={onChange}>
      <option value="">Select an exercise...</option>
      {groups.map(([muscle, list]) => (
        <optgroup key={muscle} label={muscle}>
          {list.map(e => <option key={e.id} value={e.id}>{e.name}</option>)}
        </optgroup>
      ))}
    </Select>
  );
}
```

- **Responsibility:** the exercise-library `<select>`, grouped into `<optgroup>`s by muscle.
- **Reused** by `RoutineDetail`'s "Add an exercise" form **and** `SetForm`'s "Log a set" form — a genuine two-call-site abstraction, extracted from the routine builder during Phase 11d (not speculative).
- `value` is the selected id as a **string** (`""` = nothing).

21.9 Nav.jsx

```
const TABS = [ {to:'/',label:'Home'}, {to:'/exercises',...}, {to:'/routines',...}, {to:'/history',...} ];
// ...
const active = t.to === '/' ? path === '/' : path === t.to || path.startsWith(`${t.to}/`);
```

- The fixed bottom tab bar, rendered only when authenticated (by `shell`).
- 4 `<Link>` tabs + a "Log out" `<button>` (calls `useAuth().logout`).
- **Active tab logic:** `/` is active only on exactly `/`; other tabs stay active on their sub-routes (`/history/5` keeps "History" lit). This `startsWith` check was a Phase 11f fix.
- CSS keeps all 5 items on one line down to 320 px (`white-space: nowrap` + a `clamp()` font size).

22. Screen-by-screen walkthrough

The 10 route components (plus the inline `NotFound`). For each: route, data, state, actions, and the loading/error/empty handling.

22.1 Login — `/login` (public)

- **Data:** none fetched. **State:** `username`, `password`, `error`, `pending`, `inFlight` (ref).
- **Action:** submit → local check (both fields non-empty) → `useAuth().login()` → on success the provider navigates to `/`; on failure `setError`.
- **Error mapping:** `error.status === 401` → "Invalid username or password."; otherwise `describeError(error)` (e.g. network → "Could not reach the server...").
- `<form noValidate>`; the button shows `pending`.

22.2 Signup — `/signup` (public)

Same shape as Login. Local checks: username non-empty, password ≥ 6. Calls `useAuth().signup()` (backend auto-logs-in). Error mapping: `409` → "That username is already taken."; else `describeError`.

22.3 Dashboard — `/`

- **Data:** `useAuth().user` only. **No fetch.**
- Greets `Hi, {user.username}`. One primary button "Start a workout" → `navigate('/workout')`. Two text links: "Your routines", "Exercise library".
- A fuller dashboard (recent workouts) was deliberately kept out of V1 scope.

22.4 Exercises — `/exercises`

- **Data:** `useApi(() => api.exercises(), [])` — the library, fetched once.
- **State:** `search` (a string). **Derived:** `filtered` (`useMemo`).
- **The four-state model** — this screen is the canonical example: 1. `loading` → `<Spinner>`. 2. `error` → `<ErrorMessage onRetry={reload}>`. 3. request OK but `exercises.length === 0` → `<EmptyState>` "No exercises available". 4. library has exercises but `filtered.length === 0` → a distinct `<p role="status">` "No exercises match "{search}"." — **not** the generic empty state, because the cause is different (the filter, not the data).
- Search matches `name` **or** `muscle_group`, case-insensitively (`.toLowerCase().includes(q)`). It runs **in memory** — no API call per keystroke.

22.5 Routines — `/routines`

- **Data:** `useApi(() => api.routines(), [])`. **Reload** passed to the create form as `onCreated`.
- **CreateRoutineForm** (in-file): `name`, `error`, `pending`, `inFlight`. Submit → trim → local blank check → `api.createRoutine` → `setName('')` + `onCreated()` (parent re-fetches). A `400` shows *inline on the field* (`error` prop to `<Input>`); any other error shows as a box.
- List: `<Link to={ /routines/${r.id} >` per routine; empty → `<EmptyState>` "Create your first routine above."

22.6 Routine Detail — `/routines/:id`

- **Data:** two `useApi` calls — `routine = useApi(() => api.routine(id), [id])` and `library = useApi(() => api.exercises(), [])`.
- **States:** `routine.loading` → spinner; `routine.error.status === 404` → `<EmptyState>` "Routine not found / This routine doesn't exist, or it isn't yours."; other error → `<ErrorMessage>`; `routine.data.exercises.length === 0` → `<EmptyState>` "No exercises in this routine".
- **AddExerciseForm** (in-file): `exerciseId`, `sets`, `reps`, `error`, `pending`, `inFlight`. `parseTarget` validates the optional targets. Submit → `api.addRoutineExercise(id, payload)` → clear the form + `onAdded()` (= `routine.reload`).
- Exercise list keys by ``${ex.id}-${i}`` (duplicates allowed).

22.7 Workout (dispatcher) — `/workout` and `/workout/:id`

```
export default function Workout({ id }) {
  return id ? <WorkoutSession id={id} /> : <WorkoutStart />;
}
```

No `id` → the start screen. `id` present → the active session. Putting the `id` in the URL is what makes a mid-workout refresh recover (Ch. 23, Ch. 25).

22.8 Workout Start — `/workout`

- **Data:** `useApi(() => api.routines(), [])`.
- **State machine:** `mode` (`'routine'` / `'freestyle'`), `routineId` (string), `startErr`, `starting`, `inFlight`. `noRoutines` (derived) → `effectiveMode` forced to `'freestyle'` and the radios hidden.
- **Action:** `start()` → if routine mode, validate `routineId` → `routineArg = Number(routineId)` (or `undefined` for freestyle) → `api.startWorkout(routineArg)` → `navigate( /workout/${workout.id} )` using the **server-created id**.

- **VERIFIED** on-device: no routines → freestyle only; double-click "Start" → exactly one workout created.

22.9 Workout Session — `/workout/:id`

The most stateful screen. Detail in Chapter 23.

- **Data:** three `useApi` calls — `workout (api.workout(id))`, `library (api.exercises())`, and `routine` (conditional: `routineId ? api.routine(routineId) : Promise.resolve(null)`, `deps: [routineId]`).
- **UI state:** `exerciseId` — **lifted here** so the "Today's plan" chips and the `SetForm` dropdown control the same value.
- **States:** `workout.loading && !workout.data` → full spinner (first load only); `workout.error.status === 404` → `<EmptyState>` "Workout not found"; `sets.length === 0` → `<EmptyState>` "No sets yet".
- Renders: header (`routine_name || 'Freestyle workout' + "Started " + formatDate(w.date)`); "Today's plan" chips (routine workouts only); `<SetForm>`; "Logged sets" `<section>` with `<SetList>`; a "Finish workout" secondary button → `navigate('/')`.

22.10 SetForm & SetList (workout building blocks)

SetForm (Ch. 23, 24): the "Log a set" card. `reps`, `weight`, `error`, `submitting`, `inFlight` locally; `exerciseId` from props. `nextSetNumber` derived (`useMemo`). On success calls `onLogged (= workout.reload)` and **keeps** exercise/reps/weight for fast repeats.

SetList: pure display. `groupByExercise(sets)` builds `[{exercise_id, name, muscle_group, rows:[...]}]` in first-appearance order. `weightLabel(w) → w === 0 ? 'bodyweight' : `${w} kg``. Each exercise name is an

(so the page heading hierarchy is `h1` → section `h2` → exercise `h3`, and a screen-reader user can jump between exercises). The same component renders both the live workout and the history detail — the API's set shape is identical.

22.11 History — `/history`

- **Data:** `useApi(() => api.workouts(), [])`.
- Renders the array **exactly as the server returned it** — newest first, **no client-side sort** (the API owns the ordering; re-sorting would be a second copy of a rule that can drift). Each row: `routine_name || 'Freestyle', set_count + (set_count === 1 ? 'set' : 'sets')`, `formatDate(w.date)`; links to `/history/:id`.
- Empty → `<EmptyState>` "No workouts yet" + "Start a workout" button.

22.12 Workout Detail — `/history/:id`

- **Data:** `useApi(() => api.workout(id), [id])` — the *same* endpoint the live session uses.
- `isFreestyle = data.routine_id === null`. Header: `routine_name || 'Freestyle workout' + {isFreestyle ? 'Freestyle' : 'Routine'}` · `{formatDate(data.date)}`.
- `<section>` with `<h2>Sets</h2>`; `data.sets.length === 0` → `<EmptyState>` "No sets logged"; else `<SetList sets={data.sets} />`.
- **Everything is rebuilt from that one GET.** Nothing depends on prior React state → a hard refresh is identical to navigating here.

22.13 NotFound (inline in `App.jsx`)

Rendered for any unmatched path. `<h1>Not found</h1>` + a `<Link to="/">Go home</Link>` (a `<Link>`, not an `<a href>` — a Phase 11f fix so it navigates in-app, not a full reload). When authenticated it renders **inside** the shell, so the bottom nav is still there.

22.14 Desktop behaviour

Mobile-first, but usable on a laptop: `#root { max-width: var(--content-max); margin: 0 auto }` centres the 560 px column; the bottom nav is capped and centred to match. Nothing stretches. **VERIFIED** at 1536 px, and at 320/375 px with zero horizontal overflow.

23. The complete workout flow

23.1 Routine workout — every layer

```
— START —
/workout : WorkoutStart
  mode='routine', pick "Push Day" (routineId="1"), tap "Start workout"
  start(): routineArg = Number("1") = 1
    api.startWorkout(1) → POST /api/workouts {routine_id:1}
Express: requireAuth (userId 7)
  optionalPositiveInt(1,'routine_id') → ok
  routine_id != null → SELECT id FROM routines WHERE id=1 AND user_id=7
    row? no → 400 "routine_id does not exist"
    row? yes → continue
  INSERT INTO workouts (user_id, routine_id) VALUES (7, 1) → id 5
  SELECT id, routine_id, date FROM workouts WHERE id=5
  201 { id:5, routine_id:1, date:"2026-09-06 09:31:02" }
WorkoutStart: navigate("/workout/5") ← the SERVER's id

— ACTIVE SESSION —
/workout/5 : Workout → WorkoutSession id="5"
useApi #1: api.workout("5") → GET /api/workouts/5
  Express: WHERE w.id=5 AND w.user_id=7 → metadata + sets:[]
  → { id:5, date, routine_id:1, routine_name:"Push Day", sets:[] }
useApi #2: api.exercises() → GET /api/exercises → the 21-row library
useApi #3: routineId=1 → api.routine(1) → GET /api/routines/1
  → { id:1, name:"Push Day", exercises:[{id:1,name:"Barbell Bench Press",
    target_sets:4, target_reps:8}, ...] }

RENDER:
<h1>Push Day</h1> <p>Started Sat, 6 Sep, 09:31</p>
"Today's plan": [ Barbell Bench Press 4x8 ] [ Overhead Press 3x10 ] ← chips
<SetForm> (exercise dropdown + reps + weight + "Log set")
"Logged sets": <EmptyState> "No sets yet"
[ Finish workout ]

— LOG A SET —
tap chip "Barbell Bench Press" → setExerciseId("1") (lifted state)
SetForm sees exerciseId="1":
  chosen = library.find(e => e.id === 1) → {name:"Barbell Bench Press"}
  nextSetNumber = useMemo: sets.filter(s=>s.exercise_id===1).length + 1 → 1
  shows: "Logging set 1 of Barbell Bench Press"
type reps 10, weight 60, tap "Log set"
onSubmit:
  inFlight.current? no → set it true
  validReps("10")→10 validWeight("60")→60 id=1→0
  setSubmitting(true) → button "Logging..." + disabled
  api.logSet("5", {exercise_id:1, set_number:1, reps:10, weight:60})
    → POST /api/workouts/5/sets
Express: requireAuth (7)
  positiveInt ×3, nonNegativeNumber(60) → ok
  SELECT id FROM workouts WHERE id=5 AND user_id=7 → row (ok)
  SELECT id FROM exercises WHERE id=1 → row (ok)
  INSERT INTO workout_sets (workout_id, exercise_id, set_number, reps, weight)
    VALUES (5,1,1,10,60) → id 88
  201 { id:88, workout_id:5, exercise_id:1, set_number:1, reps:10, weight:60 }
SetForm: onLogged() === workout.reload()
  (does NOT clear reps/weight/exercise – fast repeat)
useApi #1 re-runs: GET /api/workouts/5
  → sets: [ {id:88, exercise_id:1, exercise_name:"Barbell Bench Press",
    muscle_group:"Chest", set_number:1, reps:10, weight:60} ]
WorkoutSession re-renders → SetList.groupByExercise →
  "Barbell Bench Press Chest
  Set 1 10 reps × 60 kg"
SetForm: nextSetNumber recomputes (sets now length 1 for exercise 1) → 2
finally: inFlight.current=false; setSubmitting(false)

— REFRESH MID-WORKOUT —
hard reload of /workout/5:
  nginx: no file "/workout/5" → try_files → index.html
  React boots → Router path="/workout/5" → Workout id="5" → WorkoutSession
  useApi #1: GET /api/workouts/5 → the workout + all logged sets, rebuilt
  The unsent form input (whatever you'd typed but not submitted) is gone –
  it was only in React state, not in the URL or on the server.

— FINISH —
tap "Finish workout" → navigate("/")
There is NO "complete" API call. The workout row and its sets simply persist.
It now appears in /history and can be reopened at /workout/5 or /history/5.
```

23.2 Freestyle workout — the differences

```
/workout : WorkoutStart
mode='freestyle' (or forced, if the user has no routines)
start(): routineArg = undefined
      api.startWorkout(undefined) → POST /api/workouts {} ← empty body
Express:
  routine_id is undefined → the "must be your routine" check is skipped
  INSERT INTO workouts (user_id, routine_id) VALUES (7, NULL)
  201 { id:6, routine_id:null, date:... }
  navigate("/workout/6")

WorkoutSession id="6":
  workout.data.routine_id === null → routineId = null
  useApi #3: routineId null → Promise.resolve(null) → NO GET /api/routines call
  RENDER:
    <h1>Freestyle workout</h1> ← not a routine name
    (no "Today's plan" card – routineId is null)
    <SetForm> (pick any exercise from the full library dropdown)
  Logging a set is identical to the routine case – same POST, same reload.
```

23.3 The relationship chain

```
users(id=7) —1:N→ workouts(id=5, user_id=7, routine_id=1)
      |
      | 1:N→ workout_sets(id=88, workout_id=5, exercise_id=1,
      |                  set_number=1, reps=10, weight=60)
      |
      |
      | N:1
      |
      |← exercises(id=1, "Barbell Bench Press")
```

GET /api/workouts/5 walks this: authorize on `users`, read `workouts`, JOIN `workout_sets` → `exercises` for the names.

24. Workout state & double submission

24.1 The bug `disabled={pending}` alone does not fix

The obvious guard: disable the submit button while a request is in flight.

```
// naive
async function onSubmit(e) {
  e.preventDefault();
  setPending(true);           // ← schedules a re-render; button is NOT disabled yet
  await api.logSet(...);      // ← during this await the button is still clickable
  setPending(false);
}
```

`setPending(true)` **does not disable the button immediately**. It *schedules* a re-render. Between `setPending(true)` and React actually re-rendering with `disabled`, there is a window. Two `submit` events fired in that window (a fast double-tap, or `form.requestSubmit()` called twice) **both** get past the check and **both** call `api.logSet` → two sets.

This was hit during development (Ch. 35 §6).

24.2 The fix: a synchronous `useRef` guard

```
const inFlight = useRef(false);

async function onSubmit(e) {
  e.preventDefault();
  if (inFlight.current) return; // ← synchronous: takes effect immediately
  // ...validation...
  inFlight.current = true;      // ← set BEFORE any await
  setSubmitting(true);         // ← still do this, for the visual disabled state
  try {
    await api.logSet(...);
    onLogged();
  } catch (err) { setError(...); }
  finally {
    inFlight.current = false;
    setSubmitting(false);
  }
}
```

A **ref** is a **plain mutable object**. `inFlight.current = true` takes effect *on that line*, not after a render. The second `submit` event enters `onSubmit`, hits `if (inFlight.current) return`, and stops. `setSubmitting` still runs so the button visibly disables and relabels — but the ref is what *guarantees* one request.

State is for rendering. A ref is for "right now."

Every mutation form uses this: `Login`, `Signup`, `CreateRoutineForm`, `AddExerciseForm`, `WorkoutStart.start`, `SetForm`. **VERIFIED** on-device: two synchronous clicks on "Start workout" → **one** workout; double `requestSubmit` on "Log set" → **one** set (server-confirmed).

24.3 Mutations vs GETs — why they're handled differently

	GET (query)	POST (mutation)
Idempotent?	yes — run it 10× nothing changes	no — run it 2× create 2 rows
Safe to retry	yes (<code>reload</code> , retry button)	no — needs the <code>inFlight</code> guard
Rendered when	whenever <code>data</code> is present	only after the server confirms (<code>201</code>), then via a re-fetch
On failure	show error + offer retry; data unchanged	show error; nothing was created ; the list is exactly what the server has

24.4 Intent vs confirmed server state

Tapping "Log set" is **intent** — "I want a set to exist." The set does **not** exist until `POST /api/workouts/:id/sets` returns `201`. The UI renders from *confirmation* (the subsequent `GET`), never from the click. So a network failure mid-log shows an error and adds **nothing** to the list — **VERIFIED** on-device (forced offline → error shown, server set count unchanged, button re-enabled). This is a fundamental full-stack idea: the client proposes; the server is the record of truth.

24.5 Form reset policy

After a **successful** set log, `SetForm` **keeps** the exercise, reps, and weight (a gym user does "3×10 @ 40" — the next set is one tap). Only `nextSetNumber` advances, and it does so *on its own* because it is derived from `sets`, which grew. After a successful **add-exercise**, `RoutineDetail` *clears* the form (you're adding *different* exercises). Deliberate, per-form choices.

25. History flow

25.1 `GET /api/workouts` → the list

The `LEFT JOIN ... LEFT JOIN ... GROUP BY w.id ... ORDER BY w.date DESC, w.id DESC` query (§10.5). Returns `[{ id, date, routine_name, set_count }]`, newest first, in **one** round trip (no $N+1$).

`History.jsx` renders the array **verbatim**. There is no `.sort()` in the component — the server's ordering is authoritative. Re-implementing the sort client-side would be a second copy of the `date DESC, id DESC` rule that could silently disagree if the backend's tiebreak ever changed.

25.2 `GET /api/workouts/:id` → the detail

Two queries (§10.7): metadata (`LEFT JOIN routines`), then the sets (`JOIN exercises` for names), returning a **flat** `sets[]` ordered by set id.

25.3 Server data model vs frontend presentation model

Server returns	Frontend shows
<code>sets: [{exercise_name:"Bench", set_number:1, ...}, {exercise_name:"Bench", set_number:2, ...}, {exercise_name:"OHP", set_number:1, ...}]</code> (flat, log order)	Bench Press — Chest Set 1 · 10 reps × 60 kg Set 2 · 9 reps × 60 kg Overhead Press — Shoulders Set 1 · 8 reps × 40 kg

`SetList.groupByExercise` does the regrouping — **on every render, never stored**. The backend stays a clean relational list; "group by exercise" is *one* of several possible views (a table, a timeline, per-muscle) and lives in the component that wants it. If a future screen wants a different arrangement, it transforms the same flat array its own way.

25.4 Dates — stored UTC, shown local

`workouts.date` is `CURRENT_TIMESTAMP` — a UTC string `"2026-09-06 09:31:02"` with **no timezone marker**. `format.js`:

```
export function formatDate(raw) {
  if (!raw) return '';
  const d = new Date(String(raw).replace(' ', 'T') + 'Z'); // "...T...Z" → parsed as UTC
  if (Number.isNaN(d.getTime())) return String(raw);
  return d.toLocaleString(undefined, { weekday:'short', day:'numeric',
                                         month:'short', hour:'2-digit', minute:'2-digit' });
}
```

Append `Z` so the browser parses it as UTC, then `toLocaleString` renders it in **the viewer's device timezone**. There is no per-user timezone setting; the stored value is never modified. Limitation: a workout logged at 11 pm IST shows "11 pm" only when viewed from an IST device (Ch. 40).

25.5 Empty workout ≠ error

V1 has no "finish" state, so a `workouts` row with zero `workout_sets` is **valid data**. `WorkoutDetail` shows `<EmptyState>` "No sets logged / This workout was started but no sets were recorded." — not an `<ErrorMessage>`. The history list shows it with `set_count: 0` and "0 sets".

26. Error handling — browser to database

26.1 Where each status originates

Status	Originates in	This project's cases
400 Bad Request	a route handler's validation	missing/blank field, wrong type, non-positive int, unknown <code>exercise_id</code> , not-your <code>routine_id</code> on <code>POST /workouts</code>
401 Unauthorized	<code>requireAuth</code> middleware	no session cookie, or an expired/destroyed session
404 Not Found	a route handler's ownership/existence check, or <code>parseId</code> returning <code>null</code>	routine/workout that doesn't exist or isn't yours ; non-numeric <code>:id</code>
409 Conflict	the signup handler catching a <code>UNIQUE</code> violation	username already taken
500 Internal Server Error	the central error handler	an unexpected exception (a bug); the client sees only <code>"internal server error"</code>
0 (client-side only)	<code>api.js</code> 's <code>catch</code> when <code>fetch</code> rejects	server process down, offline, DNS/host wrong

26.2 Backend: the central handler

```
app.use((err, req, res, next) => {
  console.error(err); // full detail → server log
  res.status(500).json({ error: 'internal server error' }); // fixed text → client
});
```

The client never sees a stack trace, SQL text, or a file path. Every *expected* failure is an explicit `return res.status(4xx).json({ error: '...' })` in the handler — the central handler is only for the *unexpected*.

26.3 Frontend: `describeError` — status → sentence

`src/format.js`:

```
export function describeError(error) {
  switch (error?.status) {
    case 0: return 'Could not reach the server. Check your connection and try again.';
    case 400: return error.message || 'That request was not valid.';
    case 401: return 'Your session has ended. Please sign in again.';
    case 404: return 'This item no longer exists, or you do not have access to it.';
    case 409: return error.message || 'That already exists.';
    case 500: return 'Something went wrong on the server. Please try again.';
    default: return error?.message || 'An unexpected error occurred.';
  }
}
```

- **Branches on the status code**, the stable contract — never on the server's message text.
- For `400` / `409` it *does* surface `error.message` (the backend's specific string, e.g. "password must be at least 6 characters") because those are meant for the user; for `401` / `404` / `500` it uses a fixed friendly sentence.
- Shared by `ErrorMessage` **and** the auth pages. The auth pages override first: `Login` maps `401` → "Invalid username or password." (a `401` on the login page means *wrong password*, not *session expired*); `Signup` maps `409` → "That username is already taken."

26.4 The full chain for a `400`

```
SetForm: user submits weight = -5
validWeight("-5") → n = -5, n >= 0 false → null
setError(new ApiError(400, 'Weight must be 0 or more (decimals like 42.5 are fine).'))
(no network call – the frontend caught it)
<ErrorMessage error={error}/> → describeError: case 400 → error.message
renders: "Weight must be 0 or more (decimals like 42.5 are fine)."
```

If the frontend check were removed, the backend would still catch it: `nonNegativeNumber(-5, 'weight')` → `"weight must be a number >= 0"` → `400` → `ApiError(400, "weight must be a number >= 0")` → `describeError` shows that.

26.5 The full chain for a 404 (cross-user access)

```
Bob opens /history/<Alice's workout id>
WorkoutDetail: useApi(() => api.workout("41"), ["41"])
GET /api/workouts/41 (Bob's cookie → userId 9)
Express: SELECT ... FROM workouts w WHERE w.id=41 AND w.user_id=9 → no row
404 { error: "workout not found" }
api.js: !res.ok, status 404, path !== '/api/me' → onUnauthorized NOT called (401 only)
throw new ApiError(404, "workout not found")
useApi: setState({ error: ApiError(404), loading:false })
WorkoutDetail: error.status === 404 → <EmptyState title="Workout not found">
  "This workout doesn't exist, or it isn't yours."
```

26.6 Empty vs error — always distinguished

GET /api/workouts returning [] means "success; you have no workouts." That is a fact about the account, not a failure. Every list screen branches: error → <ErrorMessage>; data && data.length === 0 → <EmptyState>. Conflating them would tell a new user their app is broken.

27. Security model

For each measure: **threat** → **mitigation** → **implementation** → **limitation**. The project is *secure for its scope* (a ~10-user LAN app), not "fully secure" — the limitations are stated honestly.

27.1 Password storage

- **Threat:** database leak exposes every account; users reuse passwords.
- **Mitigation:** bcrypt hashing with a per-password salt and cost 12.
- **Implementation:** `bcryptjs.hash(password, 12)` on signup; `bcryptjs.compare` on login; only `password_hash` stored.
- **Limitation:** no password-strength meter beyond "≥ 6 chars"; no breach-list check; no rate limiting on login attempts *in the app* (Ch. 27.11).

27.2 Sessions

- **Threat:** an attacker who obtains a valid token impersonates the user; a stolen token can't be revoked.
- **Mitigation:** server-side sessions — the client holds only an opaque signed id; the server can destroy any session instantly.
- **Implementation:** `express-session`, signed with `SESSION_SECRET`, `userId` kept server-side.
- **Limitation: MemoryStore** — sessions are lost on restart (a nuisance, not a vulnerability) and don't scale past one process. `V2-BACKLOG.md` lists a SQLite-backed store.

27.3 `httpOnly` cookie

- **Threat:** XSS — a malicious script reading `document.cookie` to steal the session.
- **Mitigation:** `httpOnly: true` — the cookie is invisible to JavaScript.
- **Limitation:** does not *prevent* XSS, only limits its payoff. XSS prevention is §27.7.

27.4 `SameSite=Lax`

- **Threat:** CSRF — another website auto-submitting a request to `/api` that the browser attaches the victim's cookie to.
- **Mitigation:** `sameSite: 'lax'` — the browser won't send the cookie on **cross-site** `POST`s (it is sent on top-level GET navigations, which are harmless).
- **Implementation:** the cookie option + the fact that the frontend is **same-origin** with the API in both dev (Vite proxy) and prod (nginx), so there is no legitimate cross-origin call to accommodate.
- **Limitation:** if V2 makes the frontend cross-origin (e.g. a separate mobile app host), this protection weakens and a CSRF token (or `SameSite=Strict` + careful design) becomes necessary. `client/ARCHITECTURE.md` flags this.

27.5 No HTTPS (a documented concession)

- **Threat:** on an untrusted network, plain HTTP lets anyone on the path read the session cookie and all traffic.
- **Current state:** the V1 deployment is **plain HTTP on a home LAN**. `secure: false` on the cookie (so it sends over HTTP at all).
- **Why acceptable now:** a trusted home Wi-Fi, ~10 known users, no sensitive data beyond workout logs.
- **Fix for V2:** a domain + TLS certificate (Let's Encrypt / Cloudflare); flip the cookie to `secure: true` and consider `sameSite: 'strict'`. **DEFERRED**.

27.6 Parameterized SQL

- **Threat:** SQL injection.
- **Mitigation:** every user value is a bound `?` parameter; **no string interpolation of user input anywhere**.
- **Implementation:** `db.prepare('... WHERE x = ?').get(value)` throughout.
- **Limitation:** none for this attack. (Ch. 8.5.)

27.7 XSS protection

- **Threat:** a user storing `<script>` in a routine name (say) that runs when another view renders it.
- **Mitigation: React escapes all interpolated text by default.** `{routine.name}` renders as text, never as HTML. `dangerouslySetInnerHTML` is not used anywhere in the codebase (verified by search).
- **Limitation:** if V2 ever renders user-supplied HTML/markdown, it must sanitise it.

27.8 Error disclosure

- **Threat:** leaking internals (SQL, stack traces, paths) helps an attacker.
- **Mitigation:** the central handler returns a fixed `"internal server error"`; auth errors are generic ("invalid username or password"); cross-user access is `404` not `403` (no existence disclosure).
- **Limitation:** the `409 / 400` messages *do* echo backend text — but those strings are deliberately user-facing and contain nothing sensitive.

27.9 Network exposure

- **Threat:** an unnecessary listening port is attack surface.
- **Mitigation:** Express binds `127.0.0.1` (loopback); only nginx's `:8080` is on the network.
- **Implementation:** `app.listen(PORT, HOST)` with `HOST` defaulting to `127.0.0.1`; verified on-device (`:3000` on the LAN IP refuses connections).
- **Limitation:** nginx `:8080` is reachable by anything on the LAN. That is intended (other household devices), but there is no per-device access control — the security boundary on the LAN is "you're on my Wi-Fi." No internet exposure, no port forwarding.

27.10 Secret management

- `SESSION_SECRET` lives in `server/.env`, which is **gitignored**. Unset, the code falls back to an insecure literal (fine for local dev, wrong for prod).
- On the phone it was generated with `node -e "...randomBytes(32).toString('hex')"` and written to `.env` via `sed` **without ever being printed** to the terminal or committed.
- `server/.env.example` is the committed template with a `change-me` placeholder.

27.11 Login rate limiting

- **Threat:** online brute-force of a password.
- **Current state: not implemented in the app.** `bcrypt` cost 12 makes each attempt ~250 ms, which is a soft limiter.
- **Design:** `deploy/nginx-gym-tracker.conf` contains a **commented-out** `limit_req` block (10 attempts/min/IP) — enable it with a matching `limit_req_zone` line in `nginx.conf`. It is documented as a config toggle, not an app change, precisely so it never blocks a release. **DEFERRED**.

27.12 `.gitignore` — what never reaches the repo

```
node_modules/      *.db *.db-journal *.db-wal *.db-shm
.env               *.log
client/dist/
```

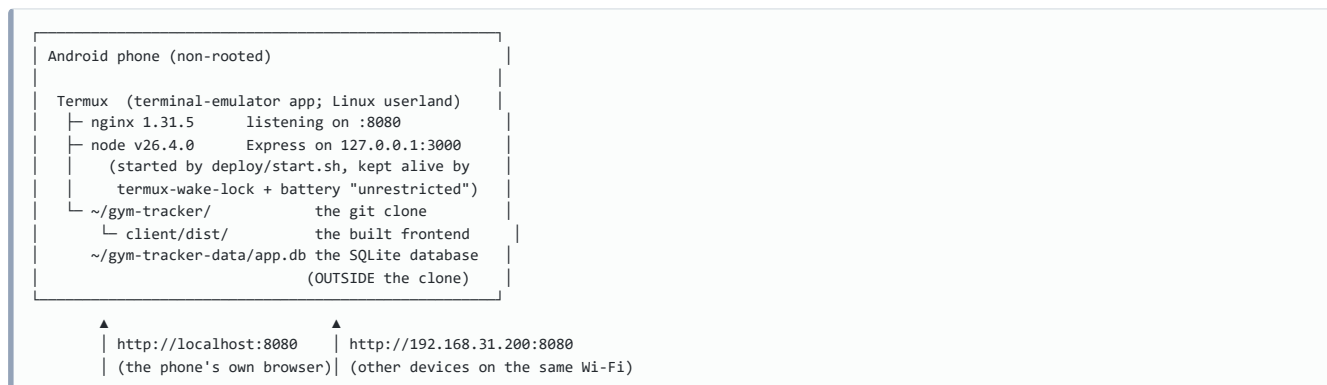
So the repository is **source only** — no runtime state, no secrets, no generated artifacts, no database. Verified: `git ls-files` contains none of these.

27.13 Verified security tests

`smoke.sh` (65/65) + the on-device re-run assert: password stored as `$2a$... (60 chars)`; ownership `404 s` hold for both users; `POST` into another user's workout writes nothing; logout invalidates the session (`/api/me` → `401` after); malformed requests are rejected with `4xx`.

28. Deployment architecture

28.1 The target



28.2 Termux and why `node:sqlite` matters here

Termux provides a Debian-like environment on Android without root. It has its own package manager (`pkg`), its own filesystem prefix (`/data/data/com.termux/files/usr`), and can run Node, nginx, git, curl.

Native Node add-ons (packages with compiled C++) are fragile on Termux — Android's Bionic C library differs from glibc, so many prebuilt binaries don't work and building from source often fails. This project sidesteps that entirely:

- `node:sqlite` is compiled *into the Node binary*. No add-on.
- `bcryptjs` is pure JavaScript. No add-on.
- `express`, `express-session` — pure JavaScript.

So `npm ci --omit=dev` on the phone installs 73 packages, **none of which compile anything**. This is a direct consequence of the technology choices in Chapter 1 and 6.

28.3 Why Claude Code / any dev tooling is not on the phone

The frontend is **built on the developer's machine** (or once on the phone) into plain static files. nginx serves them. The phone needs Node only to run `server/src/index.js`. There is no bundler, no transpiler, no dev server in production. The repo transfers via `git clone` / `git pull`.

28.4 On-device verification VERIFIED 2026-09-06

	observed
Node / npm / nginx / git	v26.4.0 / 12.0.2 / 1.31.5 / 2.55.0
<code>node:sqlite</code>	loads (DatabaseSync, StatementSync, Session, constants, backup)
Express bind	127.0.0.1:3000 ; not reachable at http://192.168.31.200:3000
DB	~/gym-tracker-data/app.db , 0600 , 36 KB
<code>health-check.sh</code>	HEALTHY (all layers)
Full journey (Chrome)	pass — signup → ... → logout → login
Deep links / Back-Forward	pass
Second-user isolation (via <code>curl</code> on device)	pass — 404 / 400 , no writes
<code>pkill node</code> → restart	data persists; sessions reset
<code>backup.sh</code>	valid .db copy (3 users, 2 workouts)
LAN access from another device	works

Two deploy-only fixes were made during this session (Ch. 35 §9–§10): commits `e540ba1` (nginx pid path) and `9a23929` (health-check nginx detection).

28.5 Known operational limitation

During a ~40-minute screen-off pause, Android **killed Termux entirely** (it relaunched fresh) despite `termux-wake-lock`. Enabling Termux → *Allow background activity* and notifications reduces this. It is reliable while the app is in active use (screen on). For unattended 24/7 availability, the backend belongs on an always-on host — which is the V2 direction.

29. nginx

29.1 What a reverse proxy is

A **reverse proxy** sits in front of one or more backend servers. Clients talk to it; it forwards to the backend and relays the response. Here it does two jobs at once: **serve static files** (the built React app) and **proxy** `/api/*` to Express. To the browser it is one origin.

29.2 The server block — `deploy/nginx-gym-tracker.conf`

```
server {
    listen 8080;
    server_name _;

    root /data/data/com.termux/files/home/gym-tracker/client/dist;
    index index.html;
    server_tokens off;

    # — API → Express —————
    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection "";
        # (optional) limit_req zone=gymauth burst=5 nodelay;
    }

    # — hashed build assets: cache hard —————
    location /assets/ {
        try_files $uri =404;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # — SPA fallback —————
    location / {
        try_files $uri $uri/ /index.html;
    }

    location = /index.html { add_header Cache-Control "no-store"; }
}
```

29.3 Location matching — why `/api/me` must not return `index.html`

nginx picks a `location` for each request. Prefix locations are ranked by length; `/api/` (longer, more specific) is chosen over `/` for any `/api/...` path.

- `GET /api/me` → `location /api/` → `proxy_pass` → Express → `401 {"error": "Not authenticated"} (JSON)`. It never reaches the `try_files` fallback. If it did (misordered or missing `location /api/`), the frontend would receive the HTML of `index.html` where it expected JSON → `JSON.parse` throws → everything breaks. **VERIFIED** on-device: `curl :8080/api/me` → `401 JSON`, `Server: nginx`, `X-Powered-By: Express`.
- `GET /assets/index-abc.js` → `location /assets/` → the real file, cached one year (the filename contains a content hash, so a new build = a new filename; the old one can be cached forever).

29.4 SPA fallback — why `/history/5` must return `index.html`

```
location / { try_files $uri $uri/ /index.html; }
```

`try_files` tries each argument in order:

1. `$uri` — is there a file at `client/dist/history/5`? No.
2. `$uri/` — a directory? No.
3. `/index.html` — serve this.

So a hard refresh of `/history/5` (or a bookmark, or a shared link) gets the SPA shell; React boots and its router reads `/history/5` and renders `WorkoutDetail`. Without this line, nginx would return its own `404` for every client-side route. **VERIFIED**: `curl :8080/history/5` → `200`, body is `index.html`.

`index.html` itself is sent `Cache-Control: no-store` so that after an update (`npm run build` produces a new `index.html` pointing at new hashed assets) users pick it up on the next load rather than seeing a stale app.

29.5 Proxy headers

- `Host $host` — pass the original Host through.

- `X-Real-IP` / `X-Forwarded-For` / `X-Forwarded-Proto` — standard "the real client was ..." headers. Express does **not** use them here (the app doesn't need the client IP, and `trust proxy` is left at its default), but they are set so a future need (logging, rate limiting) has them.
- `proxy_http_version 1.1` + `Connection ""` — enable keep-alive to the backend.

Cookies pass through untouched. Same host, plain HTTP, `SameSite=Lax` — there is no cookie-domain rewriting to do. **VERIFIED**: signup through nginx returns `Set-Cookie: connect.sid=...; HttpOnly; SameSite=Lax` and a subsequent `/api/me` with that jar → `200`.

29.6 The whole-file config — `deploy/nginx.conf.example`

A minimal complete `nginx.conf` that `include`s the server block. It deliberately sets **no** `error_log` / `pid` / `access_log` paths — nginx uses its compiled-in defaults, which the Termux package already creates. Hard-coding them caused a real failure on-device: this nginx build compiles `--pid-path=$PREFIX/tmp/nginx_pid`, not `.../var/run/nginx.pid` (Ch. 35 §9).

29.7 Port 8080, no root

A non-root process may only bind ports ≥ 1024 . `8080` needs no privileges. nginx runs entirely as the Termux user.

30. Startup & shutdown

30.1 `deploy/start.sh`

```
#!/data/data/com.termux/files/usr/bin/bash
set -eu
REPO_DIR="$(cd "$(dirname "$0")/.." && pwd)"
cd "$REPO_DIR/server"

if [ -f .env ]; then
  set -a; . ./env; set +a      # export every var defined in .env
  echo "loaded server/.env"
fi

node -e "require('node:sqlite') 2>/dev/null || {
  echo 'ERROR: this Node build has no node:sqlite (need Node >= 22.5).' >&2
  echo '      node -v -> $(node -v)' >&2
  exit 1
}"

echo "starting gym-tracker API"
echo "  repo:      $REPO_DIR"
echo "  node:      $(node -v)"
echo "  DB_PATH:    ${DB_PATH:-$REPO_DIR/server/data/app.db}"
echo "  HOST:       ${HOST:-127.0.0.1}  PORT: ${PORT:-3000}"
exec npm start
```

- `set -a; . ./env; set +a` — source `server/.env` and mark every assigned variable for export, so `PORT`, `HOST`, `DB_PATH`, `SESSION_SECRET` reach the Node process's environment. (The app has **no** dotenv library — it reads `process.env` directly.)
- **Asserts `node:sqlite`** before starting, with a clear message and `exit 1` if the Node build is too old. Fail fast, fail loud.
- `exec npm start` — replace the shell process with Node, so signals go straight to Node and there's no extra shell in the tree.
- `npm start` runs `node --disable-warning=ExperimentalWarning src/index.js` (the flag silences the "SQLite is experimental" notice; on Node 24 it is still emitted, on newer Node it may not be).

30.2 Keeping it running

- `termux-wake-lock` — a Termux command that acquires an Android wake lock so the CPU isn't suspended when the screen turns off.
- **Android battery setting** → Termux → *Unrestricted / Allow background activity / allow notifications*.
- `nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &` — run detached, output to a gitignored log, survive closing the Termux session.

30.3 Shutdown / restart

```
pkill -f 'node .*src/index.js'      # stop
cd ~/gym-tracker && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 & # start
```

Restart consequence: `MemoryStore` is a fresh empty object, so **every user is logged out** and must sign in again. **The database is a file on disk — untouched.** All routines, workouts, and sets persist. This is the single most important operational fact and was **VERIFIED** on-device (workout count identical before/after; old cookie → 401).

30.4 The `HOST` / `PORT` / `DB_PATH` env contract

var	default	set where	effect
<code>PORT</code>	<code>3000</code>	<code>server/.env</code>	Express listen port; nginx proxies here
<code>HOST</code>	<code>127.0.0.1</code>	<code>server/.env</code>	bind address (loopback)
<code>DB_PATH</code>	<code><repo>/server/data/app.db</code>	<code>server/.env</code> (phone: <code>~/gym-tracker-data/app.db</code>)	SQLite file location; <code>db.js</code> <code>mkdir -p</code> s its directory
<code>SESSION_SECRET</code>	insecure literal	<code>server/.env</code>	cookie signing key

31. Backups

31.1 `deploy/backup.sh`

```
#!/data/data/com.termux/files/usr/bin/bash
set -eu
REPO_DIR="$(cd "$(dirname "$0")/.." && pwd)"
[ -f "$REPO_DIR/server/.env" ] && . "$REPO_DIR/server/.env"
DB_PATH="${DB_PATH:-$REPO_DIR/server/data/app.db}"
DEST_DIR="${1:-$HOME/gym-tracker-backups}"
mkdir -p "$DEST_DIR"
[ -f "$DB_PATH" ] || { echo "no database at $DB_PATH" >&2; exit 1; }

STAMP="$(date +%Y%m%d-%H%M%S)"
OUT="$DEST_DIR/app-$STAMP.db"

if command -v sqlite3 >/dev/null 2>&1; then
  sqlite3 "$DB_PATH" ".backup '$OUT'"          # online backup – safe while running
else
  cp "$DB_PATH" "$OUT"                         # fallback: plain copy (stop the server first)
fi
echo "backed up -> $OUT  $(du -h "$OUT" | cut -f1)"

# keep the 14 most recent
ls -lt "$DEST_DIR"/app-*.db 2>/dev/null | tail -n +15 | while read -r old; do rm -f "$old"; done
```

31.2 Why SQLite can be backed up while the server runs

`sqlite3 <db> ".backup <out>"` uses SQLite's **online backup API**. It copies the database page by page, coordinating with any concurrent writers so the copy is a **consistent snapshot** — it never captures a half-finished transaction. A plain `cp` while the server is writing could copy a torn file, hence it is only the fallback (and the comment says to stop the server first). On the phone `sqlite3` is available, so `.backup` is used. **VERIFIED** on-device: the backup opened cleanly and contained the live data (3 users, 2 workouts).

31.3 Retention

`ls -lt` lists newest first; `tail -n +15` skips the first 14 and pipes the rest to `rm`. So the directory keeps the **14 most recent** backups.

31.4 What to back up, and what not to

Back up	Don't bother
<code>app.db</code> — the only irreplaceable artifact	the source code (it's on GitHub)
<code>server/.env</code> — your <code>SESSION_SECRET</code> and paths (keep private, never commit)	<code>client/dist/</code> (rebuilt from source)
a customised <code>nginx.conf</code> , if any	<code>node_modules/</code> (reinstalled with <code>npm ci</code>)

31.5 Recovery

Stop the server, replace `$DB_PATH` with a backup file, restart:

```
pkill -f 'node .src/index.js'
cp ~/gym-tracker-backups/app-20260906-155032.db ~/gym-tracker-data/app.db
cd ~/gym-tracker && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &
```

Recommended: a `Termux:Tasks` / cron entry running `backup.sh` daily. No cloud backup in V1 (over-engineering at this scale).

32. Health check

32.1 `deploy/health-check.sh` — what each probe proves

Run `./deploy/health-check.sh` any time. It reads `server/.env` for `PORT`, `HOST`, `DB_PATH` and assumes `NGINX_PORT=8080`.

Check	Command (essence)	Proves
node running	<code>pgrep -f 'node .*src/index.js'</code>	the Express process exists
nginx running	<code>pgrep -f 'nginx: master'</code> (with <code>pgrep nginx</code> fallback)	the reverse proxy exists
db file exists	<code>[-f "\$DB_PATH"]</code>	the database is where the app expects
db writable	<code>[-w "\$DB_PATH"]</code>	the app can persist writes
backend /	<code>curl :\$PORT/ → 200</code>	Express is up and answering (liveness)
backend /api/me	<code>curl :\$PORT/api/me → 401</code>	<code>requireAuth</code> works; the API is wired
nginx /	<code>curl :8080/ → 200</code>	the built frontend is being served
nginx /history/1	<code>curl :8080/history/1 → 200</code>	the SPA fallback works
nginx /api/me	<code>curl :8080/api/me → 401</code>	<code>/api/*</code> is proxied , not caught by the fallback
session round-trip	login as a fixed <code>__healthcheck__</code> user via nginx, then <code>/api/me</code> with the cookie → both 200	cookies survive the proxy end-to-end; sessions actually work in production

Prints `HEALTHY` (exit 0) only if **every** line is `ok`, else `UNHEALTHY` and the failing lines.

32.2 What it does not prove

- It does not exercise the full workout flow, ownership checks, or the frontend UI — that's `E2E-CHECKLIST.md` and the manual acceptance test.
- The `__healthcheck__` account is created once (login first; sign up only if that fails) so repeated runs don't litter the `users` table — a Phase 11f refinement to the script.

32.3 A "healthy" health check that was actually a bug

On the first on-device run, the script reported `FAIL nginx not running` while **every functional nginx check passed**. `pgrep -x nginx` requires an *exact* process name, but nginx renames its processes to `nginx: master process ...`. The fix (`pgrep -f 'nginx: master'`) is Ch. 35 §10. Lesson: a health check's own probes can be wrong — cross-check "HEALTHY" against the individual lines.

33. Testing architecture

33.1 `server/test/smoke.sh` — 65 end-to-end API checks VERIFIED

A bash script. Prerequisites: `bash`, `curl`, `node` (all present in Termux). Usage: `cd server && npm start` in one terminal, `./test/smoke.sh` in another. Exit 0 = all pass.

How it works:

- `BASE` defaults to `http://localhost:3000`.
- A `check "label" expected actual` helper counts pass/fail.
- A `code METHOD PATH JAR [JSON]` helper runs `curl`, writes the body to a temp file, and echoes the HTTP status. A `body` helper cats that file. A `jget "EXPR"` helper parses the body as JSON into `d` and prints `eval(EXPR)` via Node — so assertions can be `d.length`, `d.every(e => ...)`, etc.
- **Two cookie jars**, `A` (Alice) and `B` (Bob), plus per-check anonymous jars. Usernames are `user_$$_a` / `user_$$_b` (`$$` = the shell PID), so re-runs against an existing database don't collide.

What it covers (the section headers):

Section	Sample assertions
auth	signup → 201; <code>/api/me</code> authenticated → 200, unauthenticated → 401; duplicate signup → 409; missing password → 400; wrong password → 401
exercises	unauthenticated → 401; <code>count</code> === 21; every row has exactly <code>{id,name,muscle_group}</code> ; deterministically ordered
routines	create → 201; blank/absent name → 400; client <code>user_id</code> ignored; list is per-user; Bob can't see or open Alice's routine (404); non-numeric id → 404; add exercise: unknown id → 400, missing id → 400, bad target → 400, Bob → 404; duplicates allowed (2 exercises); joined <code>name</code> present
workout logging	start from routine → 201 with a date; freestyle → <code>routine_id</code> : null; bad <code>routine_id</code> → 400; start from Bob's routine → 400; log set validation (missing reps, reps 0, negative weight, string weight, unknown exercise → all 400); Bob can't log to Alice's workout → 404; non-numeric workout id → 404
history	list length 2; newest-first (<code>d[0].id >= d[1].id</code>); routine name present; freestyle <code>routine_name</code> === null; <code>set_count</code> correct; empty workout <code>set_count</code> === 0; Bob's history empty ; detail <code>sets</code> length; <code>exercise_name</code> joined; <code>reps / weight</code> are numbers; Bob can't open Alice's workout detail → 404
regression	after all of the above, Alice's <code>/api/me</code> still 200, <code>/api/exercises</code> still 200

Result: passed: 65 failed: 0. Run before and after every change during development; re-run on the phone (65/65) during deployment.

33.2 `E2E-CHECKLIST.md` — the manual browser pass

A checked-in, framework-free checklist a human runs in a real browser. Sections: Authentication (signup / login / wrong password / refresh / logout / network / double-submit) · Exercises (load / search / no-match / error+retry) · Routines (empty / create / validation / detail / add / duplicates / 404 / cross-user) · Workout — routine + freestyle (start / chips / log / double-submit / set-number / switch / decimals / bodyweight / validation / network / 404) · Active workout refresh · History (empty / order / routine / freestyle / counts / detail / zero-sets / 404 / error / refresh / back) · Security (second-user) · Responsive (320/375, overflow) · Accessibility (labels / headings / focus / semantics).

This is the "automated browser E2E framework" the project consciously does **not** have (Ch. 40).

33.3 There are no unit tests

There is **no** Jest/Vitest/Mocha suite, no test files under `src/`. The validation helpers, `matchPath`, `groupByExercise`, `describeError`, and `formatDate` are all pure functions that *would* be trivial to unit-test; V1 doesn't. The API contract is instead locked by `smoke.sh` (which exercises those functions transitively), and the UI by the manual checklist. Adding a small `node:test` suite for the pure functions is a reasonable early V2 step.

34. Testing strategy

The layers, and why each exists:

pure-function logic (validation.js, matchPath, groupByExercise, describeError, formatDate) → NOT unit-tested in V1. Small, pure, exercised transitively.
API contract (smoke.sh) → does every endpoint return the right status, shape, ordering? → run on every backend change; the "did I break the contract?" net
authorization (smoke.sh, two-user section) → can user B touch user A's data? (must be: no, everywhere) → verified at BOTH the HTTP layer AND the DB layer (write count)
browser behaviour (E2E-CHECKLIST.md, run manually per milestone) → loading/error/empty states, forms, navigation, refresh recovery
deployment (health-check.sh) → is every layer up and correctly wired, right now?
real-device acceptance (DEPLOYMENT.md §13, run once on the phone) → the whole thing, in Chrome, over nginx, on the actual hardware → VERIFIED 2026-09-06

The idea: catch a regression at the **cheapest** layer that can catch it. `smoke.sh` (seconds) catches a broken query. The manual checklist (minutes) catches a broken loading state. The phone run (once) catches "cookies don't survive nginx" — which *only* the real stack can show.

35. Real debugging examples

Every one of these actually happened during development or deployment. For each: **symptom** → **investigation** → **root cause** → **fix** → **lesson**.

35.1 Corrupt npm inside the nvm-installed Node

- **Symptom.** After `nvm-windows` installed Node 24, `npm -v` crashed with `Cannot find module '@npmcli/config'`.
- **Investigation.** Inspected `.../nvm/v24.0.0/node_modules/npm/node_modules/@npmcli/` — `config` and `arborist` were missing. `nvm-windows`'s bundled npm was incomplete.
- **Root cause.** A known `nvm-windows` issue: its npm extraction can drop files.
- **Fix.** Downloaded the official `npm-11.3.0.tgz` from the registry and replaced the broken directory with the clean one.
- **Lesson.** An "environment" failure is not an "application" failure — diagnose which. No project code was touched.

35.2 `<Button>` silently dropped its CSS classes

- **Symptom.** The primary button rendered as a plain grey browser button, not green.
- **Investigation.** `getComputedStyle` showed `className` was just `btn-block` — the `btn btn-primary` was gone.
- **Root cause.** `<Button className="btn-block">` and `{...rest}` — the spread included `className` and, being last, **overwrote** the component's own `className={ btn btn-${variant} }`.
- **Fix.** Destructure `className` explicitly and **merge**: ``btn btn-${variant} ${className}`.trim()`; don't let it fall into `...rest`.
- **Lesson.** `{...rest}` spread order matters; pull out any prop you also set yourself.

35.3 Mobile inputs ignored `background-color`

- **Symptom.** After fixing §35.2 on desktop, the button was still grey **in a mobile viewport**.
- **Investigation.** Setting `background` even inline had no effect under mobile emulation.
- **Root cause.** Mobile browsers render a *native* control chrome for `<button>` / `<input>` and ignore custom `background` unless `appearance: none`.
- **Fix.** `global.css`: `appearance: none; -webkit-appearance: none` on `button, select, textarea, input:not([type=radio]):not([type=checkbox])` — excluding radios/checkboxes so their native dot/tick survives.
- **Lesson.** Test in the *actual target viewport*; desktop-only testing hid this completely.

35.4 `GET /api/me`'s `401` bounced logged-out users off `/signup`

- **Symptom.** A logged-out visitor navigating to `/signup` was immediately redirected to `/login`.
- **Investigation.** The initial `AuthProvider` `api.me()` returned `401` on load; `api.js`'s global `401` handler fired → `navigate('/login')`.
- **Root cause.** The global handler treated every `401` as "session expired," but a `401` from `/api/me` is the *expected answer* for "you're not logged in."
- **Fix.** `if (res.status === 401 && path !== '/api/me') onUnauthorized();` — exempt that one endpoint.
- **Lesson.** A status code's *meaning* is endpoint-dependent; a blanket handler needs an exception list.

35.5 Native number-input validation blocked form submission

- **Symptom.** Submitting the "add exercise" form with `target_sets = 0` did nothing — no error, no request.
- **Investigation.** `form.requestSubmit()` never called `onSubmit`.
- **Root cause.** `<input type="number" min="1">` with value `0` fails the browser's *native constraint validation*, which silently blocks the submit (and would show a native bubble on a real click).
- **Fix.** Add `noValidate` to the `<form>` so native validation is off and the component's own `parseTarget` check runs and shows the app's own message. `min / type` stay as hints/keyboard.
- **Lesson.** If you render your own validation UI, turn off the browser's with `noValidate`.

35.6 Synchronous double-submit slipped past `disabled={pending}`

Covered in full in Chapter 24. Symptom: two rapid submits created two rows. Root cause: `setPending(true)` schedules a re-render; the button isn't disabled yet. Fix: a synchronous `useRef(false)` guard checked at the top of `onSubmit`.

35.7 Nav tab not highlighted on detail pages

- **Symptom.** On `/history/5` the "History" bottom-nav tab wasn't lit.
- **Root cause.** `const active = path === t.to` — exact match only, so `'/history/5' !== '/history'`.
- **Fix.** `t.to === '/' ? path === '/' : path === t.to || path.startsWith(t.to + '/')`.
- **Lesson.** "Active tab" usually means "on this section," not "on this exact path."

35.8 `NotFound` lost the nav and did a full reload

- **Symptom.** Hitting a bad URL while logged in showed a bare page with no bottom nav, and "Go home" fully reloaded.
- **Root cause.** `Shell` returned `<NotFound>` before the authenticated branch that adds `<Nav>`; and `NotFound` used `<a href="/">`.
- **Fix.** Treat `NotFound` as just another `View` that flows through the normal shell (so it gets `<Nav>` when authenticated); use `<Link to="/">`.
- **Lesson.** Error/empty screens deserve the same layout as real screens.

35.9 nginx config hard-coded a pid path this build doesn't use VERIFIED (on phone)

- **Symptom.** On the phone, `nginx -t` context: `nginx.conf.example` had `pid /data/.../var/run/nginx.pid;`.
- **Investigation.** `nginx -V 2>&1 | tr ' ' '\n' | grep pid-path` → `--pid-path=/data/data/com.termux/files/usr/tmp/nginx_pid`.
- **Root cause.** Different nginx builds compile different default paths; hard-coding one is not portable.
- **Fix (commit `e540ba1`).** Remove **all** explicit `error_log` / `pid` / `access_log` directives — let nginx use its compiled defaults (which the package creates). Deploy-config only; no app code.
- **Lesson.** For nginx on an unfamiliar build, don't specify paths you can let it default.

35.10 `health-check.sh` couldn't see a running nginx VERIFIED (on phone)

- **Symptom.** `health-check.sh` printed `FAIL nginx not running` while every functional nginx probe (`/` → 200, SPA fallback, `/api` proxy, session round-trip) **passed**.
- **Root cause.** `pgrep -x nginx` — `-x` needs an *exact* process name, but nginx renames its processes to `nginx: master process ...`.
- **Fix (commit `9a23929`).** `pgrep -f 'nginx: master' || pgrep nginx`.
- **Lesson.** A monitoring probe can be wrong; verify "HEALTHY" against the detail lines, not the summary.

35.11 Node/`node:sqlite` compatibility (a concern that was checked, not a bug)

- **Concern.** `node:sqlite` needs Node ≥ 22.5 and was marked experimental.
- **Check.** On the dev machine, Node had to be upgraded (nvm) from v20 to a ≥ 22.5 build. On the phone, `node -e "require('node:sqlite')"` was run **before** any other step and printed the module's keys — VERIFIED on Node v26.4.0.
- **Guard.** `deploy/start.sh` re-checks this on every start and refuses to run (with a clear message) if it fails.
- **Non-fix.** The rule for this case is explicit: **do not** swap in `better-sqlite3` / `sqlite3`; get a newer Node.

36. Failure modes

Diagnosis for each thing that can break. (`gym-tracker.log` = the `nohup` output from `start.sh`)

36.1 Node stopped

- **Symptom:** the frontend still loads (nginx serves static files); every `/api/*` call fails; the UI shows "Could not reach the server." **VERIFIED** : `curl :8080/api/me` → `502` (nginx: backend unreachable); `curl :3000/` → connection refused.
- **Diagnose:** `pgrep -af 'src/index.js'` (nothing); read `~/gym-tracker.log`.
- **Fix:** `cd ~/gym-tracker && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &`. Recovers immediately.

36.2 nginx stopped

- **Symptom:** the whole app is unreachable at `:8080` (connection refused).
- **Diagnose:** `pgrep -af 'nginx: master'` (nothing).
- **Fix:** `nginx`. Recovers immediately.

36.3 Database unavailable / path unwritable at startup

- **Symptom:** Node **exits at boot** with an error, before listening. **VERIFIED** locally: an unwritable `DB_PATH` → `Error: EPERM ... mkdir` (on Termux it would be `SQLITE_CANTOPEN / EACCES`), process exits non-zero. It does **not** start in a broken state.
- **Diagnose:** the error names the path; check the directory exists and is writable (`ls -ld`), and that `DB_PATH` in `.env` is absolute.
- **Fix:** `mkdir -p` the directory; correct the path; restart.

36.4 Port 3000 occupied

- **Symptom:** `start.sh` fails with `Error: listen EADDRINUSE ... 127.0.0.1:3000`.
- **Cause:** a previous Node is still running. **VERIFIED** on-device (a stray `nohup` + a new one).
- **Fix:** `kill -f 'node .src/index.js'`, wait a second, start again.

36.5 nginx 502 Bad Gateway on /api/...

- **Cause:** nginx is up but the backend (`127.0.0.1:3000`) is down or crashed.
- **Diagnose:** §36.1.

36.6 SPA deep link returns nginx 404

- **Cause:** the SPA fallback isn't active — a config error, a wrong `root`, or a missing `location / { try_files ... /index.html; }`.
- **Diagnose:** `nginx -t`; confirm `root` points at a directory containing `index.html`.
- **Fix:** correct the config; `nginx -s reload`.

36.7 /api/... returns HTML instead of JSON

- **Cause:** `location /api/` is missing, so `/api/*` falls through to the SPA fallback and gets `index.html`.
- **Fix:** ensure the `location /api/ { proxy_pass ... }` block is present; `nginx -t && nginx -s reload`.

36.8 Cookie doesn't stick (logged in, then instantly logged out)

- **Diagnose:** are you browsing nginx (`:8080`) or Express (`:3000`) directly? Only nginx (or the Vite proxy in dev) makes it same-origin. Check the login response has `Set-Cookie: connect.sid=...; HttpOnly; SameSite=Lax` (`curl -i`).
- **Do not** set the cookie `secure` flag on plain HTTP — the browser would then never send it.

36.9 "I was logged out" after a restart

- **This is expected** — `MemoryStore`. Log in again; the data is intact. Not a bug (Ch. 30.3, Ch. 40).

36.10 Frontend build missing / client/dist stale

- **Missing:** nginx `root` points at `client/dist`; if it doesn't exist, nginx serves its own error. Fix: `cd client && npm ci && npm run build`.
- **Stale:** you `git pull` ed new frontend code but didn't rebuild. Symptom: old UI. Fix: rebuild; hard-refresh (the config sends `index.html` as `no-store`, so this should be rare).

36.11 Failure-mode summary table

Situation	Frontend	/api	Data	Recovery
Node stopped	works (static)	502	safe	restart Node
nginx stopped	unreachable	unreachable	safe	nginx
DB path unwritable at boot	—	—	safe	Node exits loudly; fix path, restart
Node restart	works	brief 502	safe	automatic; users re-login
Termux killed by Android	unreachable	unreachable	safe	re-open Termux, <code>start.sh</code> + <code>nginx</code>

37. Development workflow

37.1 The safe loop

```
git status                # clean start? on main?

# — edit —
#  backend change → also update/extend server/test/smoke.sh if the contract changed
#  frontend change → verify in the browser

# — verify —
cd server && npm start      # terminal 1
bash server/test/smoke.sh  # terminal 2 – MUST stay 65/65
cd client && npm run dev     # terminal 3 – click through the affected screens
# for a prod-like check:
cd client && npm run build && npm run preview  # :4173, same /api proxy

git diff                  # review every hunk – no debug logs, no stray edits
git add -A && git commit -m "..." # coherent message
git push
git status                # "main...origin/main" – synchronized
```

37.2 Never commit

`server/.env` · any `*.db` · `node_modules/` · `client/dist/` · `*.log` · editor scratch files. All are in `.gitignore`; `git status --porcelain` should be empty of them.

37.3 Commit-message convention

`feat(client): ...` for features, `fix(deploy): ...` for deploy-script fixes, `docs: ...` for documentation. The 16-commit history reads as one line per milestone (Appendix / Ch. 39). Commits end with a `Co-Authored-By:` trailer.

38. Production update workflow

DEPLOYMENT.md §8, in order, and why it is this order:

```
cd ~/gym-tracker
./deploy/backup.sh           # 1. ALWAYS back up the DB first
git pull                     # 2. new code

cd server && npm ci --omit=dev # 3. ONLY if server/package.json changed
cd ../client && npm ci && npm run build # 4. rebuild the frontend (always)

pkill -f 'node *.src/index.js' # 5. restart the backend
cd .. && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &

nginx -s reload              # 6. ONLY if you changed nginx config
./deploy/health-check.sh     # 7. confirm HEALTHY
```

- **Backup before pull** — if the new code has a migration bug, you have a known-good file.
- **git pull is safe for the database** because `DB_PATH` points *outside* the clone (`~/gym-tracker-data/`), so `git` never touches it — even a full re-clone leaves the data alone. This is the whole reason `DB_PATH` exists (Ch. 8, Ch. 30).
- **Rebuild the frontend every time** — `dist/` is gitignored, so a `git pull` never updates it; you must regenerate it.
- **Deps only if package.json changed** — `npm ci` is otherwise wasted time.
- A few seconds of `502` during the Node restart is acceptable at ~10 users; V1 does not attempt zero-downtime deploys.

39. V1 design decisions

Decision	Reason	Alternative	Why not the alternative	Reconsider in V2?
Express 4 (not 5)	largest body of tutorials; a learning project	Express 5, Fastify, Hono	routing differences are minor; the point is to learn the common patterns	only if a concrete 5 feature is needed
CommonJS (backend)	matches the majority of Express tutorials	ESM	works either way; <code>require</code> reduces friction for a learner following along	no
SQLite	zero-ops, one file, runs on a phone, forces raw SQL	PostgreSQL, MySQL	needs a server process + credentials + monitoring; can't run un-rooted on Android	only at a scale far beyond 10 users
<code>node:sqlite</code>	compiled into Node — no native add-on to build on Termux	<code>better-sqlite3</code> , <code>sqlite3</code>	native add-ons are fragile on Android's Bionic libc — the exact problem to avoid	no (it's the reason the deploy is simple)
<code>bcryptjs</code> (not <code>bcrypt</code>)	pure JS — no compilation	native <code>bcrypt</code> (faster)	same Termux add-on risk; the speed difference is irrelevant for ~10 users	no
<code>express-session</code> + MemoryStore	simplest thing that is secure at this scale	JWT; a Redis/SQLite session store	JWT can't be revoked and its scaling advantage doesn't apply; a real store is more infra than 10 users need	yes — SQLite-backed store (restart = logout is annoying)
React 19	"component architecture" is an explicit learning goal; the workout screen is genuinely stateful	vanilla JS + templates	you learn components' <i>absence</i> , not components; DOM-sync bugs dominate	no
Vite	instant dev loop, one-command build, minimal config	webpack, Parcel, esbuild-by-hand	more configuration for no benefit here	no
Custom ~90-line router	one concept to learn; zero deps; enough for 10 static routes	react-router	a library to <i>use</i> vs a mechanism to <i>understand</i> ; no nested layouts / loaders needed	maybe — if V2 wants per-route code-splitting or data loading
No state library	React Context (auth) + <code>useState</code> (everything else) is sufficient	Redux, Zustand, Jotai	nothing here is complex enough to need a store; it would be ceremony	no
Raw SQL (no ORM)	every query is visible and teachable	Prisma, Drizzle, Sequelize	an ORM hides the exact thing the project exists to teach	no
Same-origin deployment (nginx / Vite proxy)	one origin → simple cookies, real <code>SameSite</code> protection, no CORS	separate API subdomain	cross-origin cookies need <code>SameSite=None; Secure</code> + CORS + a CSRF strategy	only if V2 splits the hosts
No CSRF library	<code>SameSite=Lax</code> + same-origin is sufficient at this scope	<code>csrf</code> / double-submit token	unnecessary machinery given the deployment shape	revisit if the frontend becomes cross-origin
Mobile-first CSS , hand-written, token-based	the target is a phone; consistency > component-count	Tailwind, a UI kit	a dependency + bytes; the design is small enough to own	no
No TypeScript	the backend is JS; keep one language	TS everywhere	added toolchain for a small codebase; JSDoc could add types later if wanted	optional

40. Tradeoffs

Where V1 sacrifices something for simplicity — and why that is acceptable *at V1 scale* (~10 users, one phone, trusted LAN).

Sacrifice	Consequence	Why acceptable now	V2 fix
MemoryStore sessions	server restart logs everyone out	restarts are rare (a manual deploy); no data loss; users just sign in again	SQLite-backed session store
No history pagination	<code>GET /api/workouts</code> returns <i>all</i> your workouts	dozens of rows; the query is a single fast aggregate	<code>LIMIT / OFFSET</code> when a user has hundreds
No automated browser E2E	UI regressions rely on a human running <code>E2E-CHECKLIST.md</code>	milestone-by-milestone manual passes were disciplined; <code>smoke.sh</code> locks the API	Playwright on the dev machine
No HTTPS on the LAN	traffic and the cookie are readable on the network	trusted home Wi-Fi, known users, non-sensitive data	domain + TLS; <code>secure: true</code> cookie
No persistent "active workout" beyond the URL	a hard refresh of bare <code>/workout</code> (no id) shows the start screen; unsent form input is lost	the <i>confirmed</i> workout is recovered via <code>/workout/:id</code> ; logged sets are always in the DB	a "current unfinished workout" endpoint + a resume prompt
Weight assumed kilograms	<code>1b</code> users must convert	one setting saved; no user asked for it	a per-user unit preference
No workout "completed" state	a workout is just a row; "Finish" only navigates away; any workout reopens at <code>/workout/:id</code>	at this scale nobody needs the distinction	a <code>completed_at</code> column + a Finish action
Dates shown in device timezone	a workout shows the <i>viewer's</i> local time, not where it was logged	single-user-ish; the device is usually in one place	store/display an explicit timezone
Single shared db connection, no pool, no transactions	one writer at a time (implicit)	<code>node:sqlite</code> is synchronous and single-process; every write is one statement	matters only with concurrent multi-write endpoints

Every one of these is *documented* (in `V1-STATUS.md` and `V2-BACKLOG.md`), not hidden.

41. What happens for a real user

The whole story, one action at a time. Assume production: the phone's Chrome, pointed at <http://localhost:8080>.

41.1 "I open the app."

```
1 Chrome requests GET http://localhost:8080/
2 nginx: location / → try_files "/" "/" /index.html → serves client/dist/index.html
  (an 0.8 KB shell: <div id="root"></div> + <script src="/assets/index-<hash>.js">)
3 Chrome parses it, requests GET /assets/index-<hash>.js and /assets/index-<hash>.css
4 nginx: location /assets/ → the files, with Cache-Control: immutable
5 The JS runs: main.jsx → createRoot(#root).render(<StrictMode><App/></StrictMode>)
6 <App> renders <Router><AuthProvider><Shell/>
  Router: path = window.location.pathname = "/"
  AuthProvider: status = "loading"; useEffect fires api.me() → GET /api/me
    (Chrome attaches Cookie: connect.sid=... IF one exists)
7 Shell: status === "loading" → renders <Spinner full/> and NOTHING else
8 GET /api/me reaches Express:
  express-session reads the cookie:
    - valid cookie, session in store → req.session = {userId: 7}
      requireAuth: userId set → req.userId = 7
      handler: SELECT id,username,created_at FROM users WHERE id=7
        200 {id:7, username:"sam", created_at:"..."}
    - no cookie / expired → requireAuth: 401 {error:"Not authenticated"}
9 api.js:
  200 → returns the user object → AuthProvider: setUser(u); setStatus("authenticated")
  401 → (path === "/api/me" → global handler NOT fired) throw ApiError(401)
    → AuthProvider .catch: setUser(null); setStatus("anonymous")
10 Shell re-runs:
  status "authenticated" → matchPath("/", "/") → {} → renders <Dashboard/> inside
    <div class="with-nav"> plus <Nav/>
  status "anonymous" → "/" is a protected route, not authenticated → <Redirect to="/login"/>
    → navigate("/login", replace) → renders <Login/>
```

First-time visitor: step 8 → 401 → sees the **login screen**. **Returning visitor (valid cookie):** step 8 → 200 → sees the **dashboard**, "Hi, sam".

41.2 "I log in."

```
1 Types username "sam", password "correcthorse", taps "Log in"
2 <form onSubmit={onSubmit}>:
  e.preventDefault() (no browser POST + reload)
  inFlight.current ? no → set it true
  !username.trim() || !password ? no
  setPending(true) (button → "Working...", disabled)
  useAuth().login("sam", "correcthorse")
3 auth.jsx login(): api.login("sam", "correcthorse")
  → request("POST", "/api/login", {username:"sam", password:"correcthorse"})
  → fetch("/api/login", {method:"POST",
    headers:{'Content-Type':'application/json'},
    body: '{"username":"sam","password":"correcthorse"}',
    credentials:"same-origin"})
4 nginx: location /api/ → proxy_pass 127.0.0.1:3000
5 Express:
  express.json() → req.body = {username:"sam", password:"correcthorse"}
  express-session → no session yet; req.session is a blank object
  routes/auth POST /login:
    username && password ? yes
    db.prepare('SELECT * FROM users WHERE username = ?').get("sam") → {id:7, password_hash:"$2a$12$..."}
    await bcrypt.compare("correcthorse", "$2a$12$...") → true
    req.session.userId = 7
    → express-session: generate a session id, store {userId:7},
      queue Set-Cookie: connect.sid=s%3A<id>.<sig>; Path=/; HttpOnly; SameSite=Lax; Max-Age=604800
    res.json({id:7, username:"sam"}) → 200
6 nginx relays the 200 + Set-Cookie header unchanged
7 Chrome stores the cookie (invisible to JS – httpOnly)
8 api.js request(): res.ok → returns {id:7, username:"sam"}
9 auth.jsx: setUser({id:7,username:"sam"}); setStatus("authenticated");
  navigate("/", {replace:true})
10 Router: pushState-replace → setPath("/") → Shell re-matches → <Dashboard/> + <Nav/>
  (Login unmounts; its setPending(false) in the catch never runs – success path)
```

Wrong password: step 5 → `bcrypt.compare` → `false` → `401 {error:"invalid username or password"}` → step 8 `ApiError(401)` → `login()` rejects → `Login.onSubmit` catch: `setPending(false)`, `setError(err)` → `error.status === 401` → renders "Invalid username or password." The form stays, fields kept.

41.3 "I create a routine."

```
1 Nav "Routines" → <Link> → navigate("/routines") → Shell → <Routines/>
2 Routines: useApi(() => api.routines(), [])
  → GET /api/routines (cookie → user_id 7)
  → SELECT id,name FROM routines WHERE user_id=7 ORDER BY name
  → 200 [] (new user: no routines)
  → useApi: {data:[], loading:false}
  → renders <CreateRoutineForm/> + <EmptyState "No routines yet"/>
3 Types "Push Day" in the "New routine" field → onChange → setName("Push Day")
4 Taps "Create routine":
  onSubmit: preventDefault; inFlight guard; name.trim()="Push Day" (non-blank)
  inFlight.current=true; setPending(true)
  api.createRoutine("Push Day") → POST /api/routines {name:"Push Day"}
5 Express: requireAuth(7); nonEmptyString("Push Day","name") → null (ok)
  db.prepare('INSERT INTO routines (user_id,name) VALUES (?,?)').run(7,"Push Day")
  → {lastInsertRowid: 3n}
  res.status(201).json({id:3, name:"Push Day"})
6 api.js: returns {id:3, name:"Push Day"}
7 CreateRoutineForm: setName(""); onCreate() ← onCreate === useApi's reload
8 useApi: setNonce(1) → effect re-runs → GET /api/routines
  → SELECT ... WHERE user_id=7 ORDER BY name → 200 [{id:3, name:"Push Day"}]
  → setState({data:[...], loading:false})
9 Routines re-renders: data.length > 0 → the <ul> with one <Link to="/routines/3">
10 finally: inFlight.current=false; setPending(false)
```

The new routine appears because of **step 8's re-fetch**, not step 7's local knowledge.

41.4 "I start a workout." / "I log a set." / "I view history."

These are traced in full in Chapter 23 (§23.1 routine workout end-to-end, §23.2 freestyle) and Chapter 12 (§12.2 log set, §12.3 history). The shape is always the same:

```
UI event → e.preventDefault → (validate) → inFlight guard → api.X() → fetch
  → cookie attached automatically → nginx /api/ → Express
  → express.json → express-session → requireAuth (req.userId)
  → validation helpers → ownership SELECT → INSERT/SELECT
  → res.json → nginx relays → api.js returns / throws ApiError
  → setState (or useApi.reload → GET) → React re-renders → DOM updates
```

41.5 "I log out."

```
1 Nav "Log out" <button onClick={logout}>
2 auth.jsx logout():
  try { await api.logout() } → POST /api/logout
    Express: req.session.destroy() → removed from the store
    res.clearCookie('connect.sid')
    res.json({ok:true})
  catch {} (clear locally even if that failed)
  setUser(null); setStatus("anonymous"); navigate("/login", {replace:true})
3 Shell: anonymous + "/login" is public → renders <Login/>
4 Any later /api call with the (now dead) cookie → requireAuth → 401
```

VERIFIED on-device: after logout, GET /api/me → 401, protected pages redirect to /login.

42. Request/response traces

Compact sequence diagrams. **B** = browser, **N** = nginx (or Vite proxy in dev), **E** = Express, **S** = SQLite, **R** = React.

42.1 Signup

```
B ->>POST /api/signup {u,p}>> N ->> E
    express.json -> req.body
    validate (present? len>=6?)
    bcrypt.hash(p,12) -> "$2a$12$..."
    INSERT users ->> S ->> rowid 7
    req.session.userId = 7
B <<-201 {id:7,u} + Set-Cookie- N <<- E
R AuthProvider.signup: setUser; setStatus('authenticated'); navigate('/')
```

42.2 Login

```
B ->>POST /api/login {u,p}>> N ->> E
    SELECT * FROM users WHERE username=? ->> S ->> row|Ø
    bcrypt.compare(p, row.hash) ->> true|false
    true: req.session.userId = row.id
    (false ->> 401 {error:"invalid username or password"})
B <<-200 {id,u} + Set-Cookie- N <<- E
```

42.3 Protected GET (/api/routines)

```
B ->>GET /api/routines Cookie:connect.sid->> N ->> E
    session: verify sig, store.get(sid) -> {userId:7}
    requireAuth: userId? yes -> req.userId=7
    SELECT id,name FROM routines WHERE user_id=7 ORDER BY name ->> S
B <<-200 [{id,name},...] - N <<- E
    (no/blocked cookie -> requireAuth -> 401)
```

42.4 Create routine

```
B ->>POST /api/routines {name}>> N ->> E
    requireAuth(7); nonEmptyString(name) -> ok
    INSERT routines (7, name) ->> S ->> rowid 3
B <<-201 {id:3,name} - N <<- E
R form: setName(''); onCreated() -> useApi.reload -> (42.3 again)
```

42.5 Add exercise to routine

```
B ->>POST /api/routines/3/exercises {exercise_id, target_sets?, target_reps?}>> N ->> E
    requireAuth(7)
    positiveInt(exercise_id) || optionalPositiveInt(target_sets) || optionalPositiveInt(target_reps) -> ok
    SELECT id FROM routines WHERE id=3 AND user_id=7 ->> S ->> row? Ø -> 404
    SELECT id FROM exercises WHERE id=exercise_id ->> S ->> row? Ø -> 400
    INSERT routine_exercises (3, exercise_id, sets??null, reps??null) ->> S ->> rowid
B <<-201 {id, routine_id:3, exercise_id, target_sets, target_reps} - N <<- E
R form clears; onAdded() -> routine.reload -> GET /api/routines/3 (2 queries)
```

42.6 Start workout (routine)

```
B ->>POST /api/workouts {routine_id:1}>> N ->> E
    requireAuth(7); optionalPositiveInt(1) -> ok
    SELECT id FROM routines WHERE id=1 AND user_id=7 ->> S ->> row? Ø -> 400
    INSERT workouts (7, 1) ->> S ->> rowid 5
    SELECT id,routine_id,date FROM workouts WHERE id=5 ->> S
B <<-201 {id:5, routine_id:1, date} - N <<- E
R WorkoutStart: navigate('/workout/5')
```

42.7 Log set

```
B -POST /api/workouts/5/sets {exercise_id,set_number, reps, weight} → N → E
  requireAuth(7)
  positiveIntx3 (exercise_id, set_number, reps) || nonNegativeNumber(weight) → ok
  SELECT id FROM workouts WHERE id=5 AND user_id=7 → S → row? ∅ → 404 (nothing written)
  SELECT id FROM exercises WHERE id=exercise_id → S → row? ∅ → 400
  INSERT workout_sets (5, exercise_id, set_number, reps, weight) → S → rowid 88
B ← 201 {id:88, workout_id:5, ...} ← N ← E
R SetForm: onLogged() → workout.reload → GET /api/workouts/5 → sets:[...] → SetList re-renders
```

42.8 History list

```
B -GET /api/workouts → N → E
  requireAuth(7)
  SELECT w.id, w.date, r.name AS routine_name, COUNT(ws.id) AS set_count
    FROM workouts w
   LEFT JOIN routines r      ON r.id = w.routine_id
   LEFT JOIN workout_sets ws ON ws.workout_id = w.id
 WHERE w.user_id = 7
 GROUP BY w.id
 ORDER BY w.date DESC, w.id DESC → S
B ← 200 [{id,date,routine_name,set_count}, ...] (newest first) ← N ← E
R History: render the array AS-IS (no sort)
```

42.9 Logout

```
B -POST /api/logout Cookie:connect.sid → N → E
  req.session.destroy() → store: delete sid
  res.clearCookie('connect.sid')
B ← 200 {ok:true} + Set-Cookie: connect.sid=; Expires=<past> ← N ← E
R AuthProvider: setUser(null); setStatus('anonymous'); navigate('/login')
```

42.10 Unauthorized request

```
B -GET /api/workouts (no cookie, or a destroyed session) → N → E
  express-session: no valid session → req.session.userId undefined
  requireAuth: → res.status(401).json({error:"Not authenticated"}) (handler never runs)
B ← 401 {error:"Not authenticated"} ← N ← E
api.js: !res.ok, status 401, path !== '/api/me' → onUnauthorized()
  → AuthProvider handler: setUser(null); setStatus('anonymous'); navigate('/login')
  → also throw ApiError(401) (the caller's useApi records it, but we've navigated away)
```

43. Data lifecycle

One value — a set's `weight` — from keystroke to pixel and back.

```
① USER INPUT
<input type="number" ...> user types "62.5"
onChange(e) → setWeight("62.5") ← a STRING in React state

② CLIENT VALIDATION (UX)
SetForm.onSubmit → validWeight("62.5")
  n = Number("62.5") = 62.5
  Number.isFinite(62.5) && 62.5 >= 0 → returns 62.5 (a NUMBER)

③ HTTP BODY
api.logSet("5", {exercise_id:1, set_number:1, reps:10, weight:62.5})
JSON.stringify → '{"exercise_id":1,"set_number":1,"reps":10,"weight":62.5}'
fetch POST /api/workouts/5/sets Content-Type: application/json

④ PARSING (server)
express.json() → req.body.weight === 62.5 (a JS number again)

⑤ SERVER VALIDATION (integrity)
nonNegativeNumber(62.5, 'weight')
  typeof 62.5 === 'number' && Number.isFinite && >= 0 → null (accepted)
(a string "62.5" here would FAIL typeof → 400 – the backend does not coerce)

⑥ SQL PARAMETER
db.prepare('INSERT INTO workout_sets (... , weight) VALUES (... , ?)').run(... , 62.5)
62.5 is a BOUND parameter – never interpolated into the SQL text

⑦ DATABASE ROW
workout_sets.weight column is REAL → stored as the float 62.5

⑧ READ BACK
GET /api/workouts/5 →
SELECT ..., ws.weight FROM workout_sets ws JOIN exercises e ... WHERE ws.workout_id=5
→ row.weight === 62.5 (number)

⑨ JSON RESPONSE
res.json({ ..., sets:[{ ..., weight: 62.5 }] }) → '"weight":62.5'

⑩ REACT STATE
useApi: setState({ data: { ..., sets:[{ ..., weight:62.5 }] })

⑪ DERIVED DISPLAY
SetList: weightLabel(62.5) → "62.5 kg" (weightLabel(0) → "bodyweight")

⑫ DOM
<span class="set-row-detail">10 reps × 62.5 kg</span>
```

Every arrow is a boundary where the value's *representation* changes (string → number → JSON text → SQL param → DB float → JSON → JS → display string) but its *meaning* is preserved. Validation guards the boundaries into the system (②, ⑤); the parameterized `?` guards the boundary into SQL (⑥); React's escaping guards the boundary into the DOM (⑫ — `{...}` renders as text).

44. Glossary

API — Application Programming Interface. Here, the set of HTTP endpoints the frontend calls (`/api/*`).

Authentication — establishing *who* a request is from. (This app: a session cookie → `req.session.userId`.)

Authorization — deciding whether that identity may access a *specific* resource. (This app: `WHERE ... AND user_id = ?` in every user-owned query.)

bcrypt — a deliberately slow password-hashing algorithm with a tunable cost and a built-in per-password salt.

BigInt — a JavaScript number type for integers beyond 2^{53} . `node:sqlite`'s `lastInsertRowid` is a BigInt; the code does `Number(...)` on it.

Bound parameter — a `?` placeholder in SQL whose value is passed separately; the database treats it as pure data, defeating SQL injection.

CommonJS — Node's older module system: `require()` / `module.exports`. (This project's backend.)

Component — a function that returns a description of UI (JSX). React's unit of composition.

Context (React) — a way to pass a value to every descendant component without prop-drilling. (This app: `RouterContext`, `AuthContext`.)

Controlled input — a form field whose value comes from React state and whose `onChange` updates that state.

Cookie — a small `name=value` string the server sets and the browser auto-attaches to every subsequent request to that origin.

CORS — Cross-Origin Resource Sharing. Browser rules that block a page on origin A from reading responses from origin B unless B opts in. Avoided here by being **same-origin**.

CSRF — Cross-Site Request Forgery. Another site tricking the browser into sending an authenticated request to your API. Mitigated by `SameSite=Lax` + `same-origin`.

Derived data — a value computed from other state on every render, never stored separately. (`filteredExercises`, `nextSetNumber`, grouped sets.)

Effect (`useEffect`) — code that runs *after* render to synchronise with the outside world (network, listeners, timers).

ESM — ECMAScript Modules: `import` / `export`. (This project's frontend.)

Express — a minimal Node web framework providing routing and middleware.

`fetch` — the browser API for HTTP requests. Rejects only on network failure; a `4xx` / `5xx` still resolves (you check `res.ok`).

Foreign key — a column whose value must reference an `id` in another table. Enforced in SQLite only with `PRAGMA foreign_keys = ON`.

Hook (React) — a function starting with `use` that lets a component use state or other React features (`useState`, `useEffect`, `useMemo`, `useRef`, `useContext`, `useCallback`, `useId`).

HMR — Hot Module Replacement. Vite swapping a changed module into the running page without a full reload.

HTTP — the request/response protocol of the web. A request: method + path + headers + optional body. A response: status + headers + body.

HTTPS — HTTP over TLS (encrypted). **Not** used in the V1 LAN deployment.

`httpOnly` — a cookie attribute making it unreadable by JavaScript.

Idempotent — an operation that has the same effect run once or many times. `GET` is idempotent; `POST` (create) is not.

IDOR — Insecure Direct Object Reference. Accessing a resource by id without an ownership check. Prevented here by filtering on `user_id` in SQL.

JOIN — an SQL operation combining rows from two tables on a matching condition. `INNER JOIN` keeps only matches; `LEFT JOIN` keeps every left row.

JSX — the XML-like syntax in `.jsx` files; compiles to `React.createElement(...)`.

JWT — JSON Web Token. A signed, self-contained token the client holds. **Not** used here (sessions instead).

Middleware — a function (`req`, `res`, `next`) every request passes through, in registration order.

Mutation — a request that changes server state (`POST` here). Needs exactly-once handling.

N+1 query — fetching a list then firing one extra query per item (1 + N). Avoided in history with a single `LEFT JOIN ... GROUP BY`.

`next` — the middleware callback: `next()` → continue; `next(err)` → jump to the error handler.

`node:sqlite` — SQLite built into the Node binary (v22.5+). No native add-on.

ORM — Object-Relational Mapper. A library that hides SQL. **Not** used.

`PRAGMA` — an SQLite-specific configuration/inspection command.

Prop — an input passed from a parent React component to a child; read-only in the child.

Query — a request that only reads (`GET` here). Idempotent, safe to retry.

React — the UI library. You describe UI as a function of state; React updates the DOM.

Ref (`useRef`) — a mutable box that persists across renders and does **not** trigger a re-render when changed.

Reverse proxy — a server in front of your backend that clients talk to instead. (Here: nginx — serves static + proxies `/api`.)

`SameSite` — a cookie attribute controlling whether the browser sends it on cross-site requests. `Lax` here.

Salt — random data mixed into a password before hashing so identical passwords hash differently. Stored inside the bcrypt string.

`secure` (cookie) — attribute meaning "only send over HTTPS." `false` in V1 (plain HTTP LAN).

Session — server-side state for a logged-in user, keyed by an id the client holds in a cookie.

SPA — Single-Page Application. Loads one HTML page; JavaScript then swaps "pages" client-side.

SPA fallback — the nginx rule (`try_files ... /index.html`) that serves the app shell for any client-side route.

SQL injection — an attack that rewrites a query's logic via crafted input. Prevented by bound parameters.

SQLite — a serverless, single-file relational database.

State (`useState`) — data a component remembers between renders; changing it triggers a re-render.

Transaction — a group of database writes that all succeed or all fail. **Not used** (every write is a single statement).

Vite — the frontend build tool / dev server.

XSS — Cross-Site Scripting. Injecting a script into a page. Mitigated by React's automatic text escaping and by never using `dangerouslySetInnerHTML` .

45. Interview knowledge

Questions you could be asked about *this* project, answered from *this* code. (No invented accomplishments — everything below is in the repo.)

"Why React and not vanilla JS?"

The workout-logging screen is genuinely stateful — a set list that grows, a form that resets, a set number that must recompute, buttons that disable mid-request. In vanilla JS the hard part is keeping the DOM in sync with data after every change; that's where the bugs live. React's model — describe the UI as a function of state, let it diff and patch — removes that entire class of bug. And "component architecture" was an explicit learning goal; you don't learn it by faking it with template strings.

"Why SQLite and not Postgres?"

~10 users, one process, a phone as the host. SQLite is a file — zero operational surface, nothing to secure or monitor, and it runs un-rooted on Android where a Postgres *server* does not. It also forces raw SQL, which is a learning goal. `node:sqlite` (compiled into Node) means no native add-on to build on Termux, which is exactly why the deployment is simple.

"Why sessions and not JWT?"

One server, ~10 users — JWT's stateless-scaling advantage doesn't apply. Sessions can be **revoked instantly** (delete from the store); a JWT can't be, without a blacklist that defeats the point. And a session id in an `httpOnly` cookie is unreadable by a XSS script, whereas a JWT usually sits in JS-readable storage.

"How does authentication work here, end to end?"

Login → `bcrypt.compare` the password against the stored hash → set `req.session.userId` → `express-session` stores `{userId}` server-side and sends a signed `httpOnly` session-id cookie. Every later request: the browser auto-attaches the cookie → `express-session` looks it up → `req.session.userId` → `requireAuth` middleware either 401s or attaches `req.userId` and continues. The frontend never knows the auth state directly — it calls `GET /api/me` on load and rebuilds its state from the answer, which is why a refresh keeps you logged in.

"How does authorization work? How do you stop IDOR?"

Every user-owned query filters by the session's user id **in the SQL**: `WHERE id = ? AND user_id = ?`. `req.userId` comes from the session, never from the request. If the row isn't yours, the query returns nothing and the handler returns `404` (not `403` — so an attacker can't even confirm the id exists). This is safer than "fetch by id, then check ownership in JS," where the row is already in memory and one refactor from leaking. Verified with a two-user test suite at both the HTTP and the database layer.

"Explain a JOIN from this project."

`GET /api/workouts` (history): `workouts LEFT JOIN routines LEFT JOIN workout_sets, GROUP BY w.id. LEFT on routines` because a freestyle workout has `routine_id = NULL` and a plain JOIN would drop it. `LEFT on workout_sets` because a just-started workout has no sets and must still appear; `COUNT(ws.id)` then gives `0` (COUNT ignores NULLs). `GROUP BY w.id` collapses the row-multiplication the sets-join caused. It's one query instead of the N+1 "fetch workouts then loop fetching sets."

"What's your security posture — and what are its limits?"

Passwords bcrypt-hashed (cost 12, salted). Sessions server-side, `httpOnly` + `SameSite=Lax` cookie. All SQL parameterized. Ownership enforced in SQL. React escapes output; no `dangerouslySetInnerHTML`. Express bound to loopback; nginx is the only network entry. Secrets in a gitignored `.env`. **Limits, stated honestly**: no HTTPS on the LAN (documented concession — `secure: false`); no app-level login rate limiting (nginx `limit_req` is a documented toggle); MemoryStore sessions reset on restart; the LAN boundary is "you're on my Wi-Fi." It is secure *for a ~10-user home deployment*, not "fully secure."

"Why nginx? What does it do?"

It's a reverse proxy doing two jobs: serve the built React files as static content, and proxy `/api/*` to Express on `127.0.0.1:3000`. Key config: a `location /api/` block matched before the SPA fallback (so `/api/me` returns JSON, not `index.html`), and `location / { try_files $uri $uri/ /index.html; }` so a hard refresh of `/history/5` serves the app shell and lets the client router take over.

"How is state managed on the frontend?"

Three kinds. **Server state** (routines, workouts, sets) via a `useApi` hook; after a mutation, re-fetch rather than patch the local copy. **UI state** (search term, selected exercise, form values) via `useState`, as local as possible. **Derived state** (filtered list, next set number, grouped sets) computed each render with `useMemo`, never stored — two copies of one fact drift apart. Auth is the one app-wide value, in a Context.

"You wrote your own router. Why, and how does it work?"

~90 lines on the History API. `history.pushState` changes the URL without a reload; a `popstate` listener handles Back/Forward; a `matchPath` function turns `/routines/:id + /routines/42` into `{id:"42"}`. I hand-rolled it because it's one concept to *learn* and 10 static routes don't need react-router's nested layouts or data loaders. If V2 wants per-route code-splitting, that's when react-router earns its place.

"Tell me about a bug you fixed."

The synchronous double-submit. `disabled={pending}` doesn't disable the button *immediately* — `setPending(true)` schedules a re-render, and two submit events fired in that gap both create a row. Fix: a `useRef(false)` "in flight" flag checked synchronously at the top of the handler — a ref mutates on the line, not after a render. Every mutation form uses it now; verified that a double-click "Start workout" creates exactly one workout.

"How did you test it? How did you deploy it?"

Testing: a 65-check bash `smoke.sh` that hits every endpoint with two users and asserts status/shape/ordering and cross-user isolation (at the HTTP *and* DB-write level); a checked-in manual `E2E-CHECKLIST.md` for the browser; a `health-check.sh` that probes every deployment layer. Deployment: Termux on an Android phone — `git clone`, `npm ci --omit=dev`, `npm run build`, an nginx config from `deploy/`, `start.sh` under `nohup`. Verified the full journey in Chrome on the device, plus restart-persistence and second-user isolation.

46. How to extend the project (V2)

From `V2-BACKLOG.md`. **None of this is implemented.** For each: what it touches.

46.1 Persistent SQLite-backed session store DEFERRED

- **DB:** a new `sessions` table (id, data JSON, expires).
- **API:** `session({ store: new SqliteStore(db) })` in `index.js` — a ~30-line custom store or a small library. **Zero route changes.**
- **Frontend:** none.
- **Deployment:** none. **Effect:** a restart no longer logs everyone out.
- **Testing:** add a `smoke.sh` check: log in, restart the server, `/api/me` still `200`.

46.2 HTTPS DEFERRED

- **DB / API code:** cookie `secure: true`, consider `sameSite: 'strict'`, and `app.set('trust proxy', 1)` if TLS terminates at nginx.
- **Deployment:** a domain, a certificate (Let's Encrypt via a tunnel, or Cloudflare), an nginx `listen 443 ssl` block + a redirect.
- **Frontend:** none. **Testing:** re-verify the cookie flows over HTTPS.

46.3 Login rate limiting DEFERRED

- **Deployment only:** uncomment `limit_req_zone` in `nginx.conf` and `limit_req` in the `/api/` block (10/min/IP). **No app change.**

46.4 Workout completion state DEFERRED

- **DB:** `ALTER TABLE workouts ADD COLUMN completed_at TEXT`.
- **API:** a `POST /api/workouts/:id/finish` (sets `completed_at`); `GET /api/workouts` could expose it.
- **Frontend:** "Finish workout" calls the new endpoint; history distinguishes finished/in-progress.

46.5 Resume current workout DEFERRED

- **API:** `GET /api/workouts/current` → the latest workout without `completed_at` (needs §46.4).
- **Frontend:** bare `/workout` checks it and offers "Resume".

46.6 Previous performance DEFERRED

- **API:** on the workout-session data, include "last time you did this exercise" — a query: `SELECT ... FROM workout_sets JOIN workouts ... WHERE user_id = ? AND exercise_id = ? ORDER BY date DESC LIMIT 1`.
- **Frontend:** show it next to the set form.

46.7 Set types DEFERRED

- **DB:** `ALTER TABLE workout_sets ADD COLUMN set_type TEXT` (the schema comment anticipates this).
- **API:** accept/return `set_type` in the sets endpoints.
- **Frontend:** a set-type picker in `SetForm`; `SetList` shows it.

46.8 Rest timer DEFERRED

- **Frontend only:** a countdown in `WorkoutSession`. **No backend change.**

46.9 1RM (one-rep max) DEFERRED

- **Frontend only** (or a computed API field): the Epley formula `weight × (1 + reps/30)` on the workout detail / a per-exercise view.

46.10 Progress charts DEFERRED

- **API:** an aggregate endpoint — `GROUP BY date` volume/weight per exercise over time.
- **Frontend:** a charting library (the first real UI dependency) + a new screen.

46.11 Unit preference (kg/lb) DEFERRED

- **DB:** `ALTER TABLE users ADD COLUMN weight_unit TEXT DEFAULT 'kg'`.
- **API:** include it in `/api/me`; accept it on a settings endpoint.
- **Frontend:** convert for display and input; store canonically.

46.12 Editing / deletion DEFERRED

- **API:** `PATCH / DELETE` routes for routines, workouts, sets — each needs the same ownership clause. `DELETE` on a parent needs a cascade decision (add `ON DELETE CASCADE` to the FKs, or delete children in a **transaction**).
- **Frontend:** edit/delete affordances + confirmation.
- **Testing:** new `smoke.sh` sections, including "B can't delete A's row."

46.13 History pagination DEFERRED

- **API:** `GET /api/workouts?limit=&offset=` (or cursor-based).
- **Frontend:** "load more" or infinite scroll on `History`.
- **DB:** at that point, `CREATE INDEX ON workouts(user_id, date)`.

46.14 The bigger goal: "reachable from anywhere"

`V1-STATUS.md`'s "Next milestone" section: this is **infrastructure, not a rewrite** — move the backend to an always-on host (a cheap VPS / free-tier PaaS; SQLite comes along fine at this scale), add a domain + HTTPS (§46.2), swap in a persistent session store (§46.1), and make the frontend an installable **PWA** (a web app manifest + a service worker — additive; the React code is unchanged). Optionally wrap the same build with **Capacitor** for app-store presence.

47. Learning roadmap

What to learn next, in order, each tied to a part of this project you can now point at.

Beginner — solidify the foundations

Topic	Why, via this project
JavaScript fundamentals — <code>async</code> / <code>await</code> , promises, array methods (<code>map</code> / <code>filter</code> / <code>reduce</code>), destructuring, spread, <code>??</code> / <code>?.</code>	<code>api.js</code> , <code>SetList.groupByExercise</code> , every route handler use these constantly
HTTP basics — methods, status codes, headers, request/response, <code>curl</code>	run <code>smoke.sh</code> line by line; read Chapter 42
JSON — shape, <code>JSON.stringify</code> / <code>parse</code> , why the API is JSON	trace <code>weight</code> through Chapter 43
SQL basics — <code>SELECT</code> / <code>INSERT</code> , <code>WHERE</code> , <code>ORDER BY</code> , primary keys	open <code>app.db</code> with the <code>sqlite3</code> CLI and re-run the queries from Chapter 10 by hand
Git — branch, commit, diff, push; <code>.gitignore</code>	the 16-commit history is a worked example of milestone commits

Intermediate — the core stack

Topic	Why, via this project
Express deeply — middleware, routers, <code>req</code> / <code>res</code> , error handling	Chapters 4–5; then add a new endpoint and a <code>smoke.sh</code> check for it
SQL joins & aggregates — <code>INNER</code> vs <code>LEFT JOIN</code> , <code>GROUP BY</code> , <code>COUNT</code> , the N+1 problem	Chapter 10; then write the "previous performance" query (§46.6) yourself
React deeply — the render model, <code>useState</code> / <code>useEffect</code> / <code>useMemo</code> / <code>useRef</code> , dependency arrays, Context, controlled inputs	Chapter 15; then implement the rest timer (§46.8 — frontend-only)
Authentication & sessions — cookies, <code>httpOnly</code> / <code>SameSite</code> , hashing, salts, work factors	Chapter 6; then add a SQLite session store (§46.1)
Authorization patterns — ownership checks, IDOR, <code>404</code> vs <code>403</code>	Chapter 7; extend it to <code>DELETE</code> routes (§46.12) and test cross-user
Client-side routing — History API, <code>pushState</code> , <code>popstate</code> , route matching	Chapter 17; then try swapping in <code>react-router</code> and see what changes
Frontend build tooling — what Vite does, dev vs prod, hashed assets, code-splitting	Chapter 2 §2.2; inspect <code>client/dist/</code> after a build

Advanced — operations & scale

Topic	Why, via this project
Linux / shell — processes, <code>pgrep</code> / <code>pkill</code> , <code>nohup</code> , env vars, <code>set -a</code>	read every script in <code>deploy/</code> line by line
nginx — locations, <code>proxy_pass</code> , <code>try_files</code> , headers, caching, <code>limit_req</code>	Chapter 29; then enable rate limiting (§46.3) and observe it
Networking — loopback vs LAN, ports, DNS, TLS/HTTPS, reverse proxies	Chapter 28; then do the HTTPS milestone (§46.2)
Database design — normalization, indexes, migrations, transactions, cascades	Chapter 9; add the <code>completed_at</code> migration (§46.4) and an index (§46.13)
Security — the full OWASP top 10 mapped onto this app; threat modelling	Chapter 27 is a starting threat model — extend it
Testing — unit (pure functions), integration (the API), E2E (Playwright)	Chapter 33–34; add a <code>node:test</code> suite for <code>validation.js</code> + <code>matchPath</code>
Performance — profiling, N+1, caching, bundle size, when to optimise	Chapter 10.6, Chapter 9.9 — measure before changing anything
PWA / mobile — manifests, service workers, offline, Capacitor	§46.14 — the concrete next step for "an app on any phone"

The thread through all of it: this project is small enough to hold in your head and real enough that every concept above has a line of code you can put your finger on.

Appendix A — Complete API table

Base: `http://<host>:8080/api` (prod) or `http://localhost:3000/api` (dev direct). All responses JSON. All errors `{ "error": "<message>" }`. **Auth** = **session cookie** unless "public".

#	Method	Path	Auth	Body	Success	Errors
1	POST	<code>/signup</code>	public	<code>{username, password}</code>	<code>201 {id, username} + Set-Cookie</code>	<code>400</code> , <code>409</code>
2	POST	<code>/login</code>	public	<code>{username, password}</code>	<code>200 {id, username} + Set-Cookie</code>	<code>400</code> , <code>401</code>
3	POST	<code>/logout</code>	any	—	<code>200 {ok:true}</code>	—
4	GET	<code>/me</code>	✓	—	<code>200 {id, username, created_at}</code>	<code>401</code>
5	GET	<code>/exercises</code>	✓	—	<code>200 [{id, name, muscle_group}]</code> (21)	<code>401</code>
6	POST	<code>/routines</code>	✓	<code>{name}</code>	<code>201 {id, name}</code>	<code>400</code>
7	GET	<code>/routines</code>	✓	—	<code>200 [{id, name}]</code>	<code>401</code>
8	GET	<code>/routines/:id</code>	✓	—	<code>200 {id, name, exercises: [{id,name,muscle_group,target_sets,target_reps}]}</code>	<code>401</code> , <code>404</code>
9	POST	<code>/routines/:id/exercises</code>	✓	<code>{exercise_id, target_sets?, target_reps?}</code>	<code>201 {id, routine_id, exercise_id, target_sets, target_reps}</code>	<code>400</code> , <code>404</code>
10	POST	<code>/workouts</code>	✓	<code>{routine_id?}</code>	<code>201 {id, routine_id, date}</code>	<code>400</code>
11	POST	<code>/workouts/:id/sets</code>	✓	<code>{exercise_id, set_number, reps, weight}</code>	<code>201 {id, workout_id, exercise_id, set_number, reps, weight}</code>	<code>400</code> , <code>404</code>
12	GET	<code>/workouts</code>	✓	—	<code>200 [{id, date, routine_name, set_count}]</code> newest first	<code>401</code>
13	GET	<code>/workouts/:id</code>	✓	—	<code>200 {id, date, routine_id, routine_name, sets: [{id,exercise_id,exercise_name,muscle_group,set_number,reps,weight}]}</code>	<code>401</code> , <code>404</code>

(Non-API: `GET /` → `200 "hello"` — a liveness probe on port 3000.)

Appendix B — Database reference

File: `$DB_PATH` (default `server/data/app.db` ; phone `~/gym-tracker-data/app.db`). **Engine:** `node:sqlite` `DatabaseSync` . `PRAGMA foreign_keys = ON` .

Table	Column	Type	Notes
users	id	INTEGER	PK, AUTOINCREMENT
	username	TEXT	UNIQUE , NOT NULL
	password_hash	TEXT	NOT NULL — bcrypt <code>\$2a\$12\$...</code> , 60 chars
	created_at	TEXT	DEFAULT CURRENT_TIMESTAMP (UTC)
exercises	id	INTEGER	PK
	name	TEXT	NOT NULL
	muscle_group	TEXT	nullable (all 21 seeded rows have one)
routines	id	INTEGER	PK
	user_id	INTEGER	NOT NULL, FK → users(id)
	name	TEXT	NOT NULL
routine_exercises	id	INTEGER	PK
	routine_id	INTEGER	NOT NULL, FK → routines(id)
	exercise_id	INTEGER	NOT NULL, FK → exercises(id)
	target_sets	INTEGER	nullable
	target_reps	INTEGER	nullable
workouts	id	INTEGER	PK
	user_id	INTEGER	NOT NULL, FK → users(id)
	routine_id	INTEGER	nullable , FK → routines(id) — NULL = freestyle
	date	TEXT	DEFAULT CURRENT_TIMESTAMP (UTC)
workout_sets	id	INTEGER	PK
	workout_id	INTEGER	NOT NULL, FK → workouts(id)
	exercise_id	INTEGER	NOT NULL, FK → exercises(id)
	set_number	INTEGER	client-supplied
	reps	INTEGER	
	weight	REAL	<code>0</code> = bodyweight; decimals allowed

Relationships: users 1:N routines · users 1:N workouts · routines 1:N routine_exercises · workouts 1:N workout_sets · exercises 1:N routine_exercises · exercises 1:N workout_sets.

Indexes: implicit on every PK and on `users.username` (UNIQUE). No secondary indexes.

No ON DELETE clauses — there are no DELETE endpoints in V1, so cascade behaviour is unspecified/moot.

Seed: `seed.js` inserts 21 exercises **only if** `SELECT COUNT(*) FROM exercises` **is 0** — safe on every boot.

Appendix C — Important commands

Development

```
cd server && npm install && npm start          # API on :3000
cd client && npm install && npm run dev          # UI on :5173 (proxies /api → :3000)
# open http://localhost:5173
rm -f server/data/app.db                      # wipe the DB (re-seeds on next start)
```

Testing

```
cd server && npm start                          # terminal 1
bash server/test/smoke.sh                      # terminal 2 – expect: passed: 65   failed: 0
# BASE=http://localhost:8080 bash server/test/smoke.sh # (through nginx)
# manual browser pass: E2E-CHECKLIST.md
```

Build (frontend)

```
cd client && npm run build                      # → client/dist/ (~67 KB gzip JS)
cd client && npm run preview                   # :4173 – the built app + /api proxy
```

Deployment (on the phone, Termux)

```
# first time
pkg install nodejs git nginx
git clone https://github.com/Sumit-0610/gym-tracker.git && cd gym-tracker
cd server && npm ci --omit=dev && cp .env.example .env && nano .env
mkdir -p ~/gym-tracker-data
cd ../client && npm ci && npm run build
cp $PREFIX/etc/nginx/nginx.conf $PREFIX/etc/nginx/nginx.conf.orig
cp ~/gym-tracker/deploy/nginx.conf.example $PREFIX/etc/nginx/nginx.conf
nginx -t && nginx

# every session
termux-wake-lock
cd ~/gym-tracker && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &
pgrep nginx >/dev/null || nginx
./deploy/health-check.sh                # expect: HEALTHY
```

Backup / restart

```
./deploy/backup.sh                      # → ~/gym-tracker-backups/app-<ts>.db
./deploy/backup.sh /sdcard/Download      # to somewhere you can copy off-device
killall -f 'node .*src/index.js'        # stop the API
cd ~/gym-tracker && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 & # start it
nginx -s reload                         # after an nginx config change
nginx -s stop ; nginx                   # full nginx restart
```

Update to a new version

```
cd ~/gym-tracker
./deploy/backup.sh
git pull
cd server && npm ci --omit=dev            # only if server/package.json changed
cd ../client && npm ci && npm run build
killall -f 'node .*src/index.js' ; cd .. && nohup ./deploy/start.sh > ~/gym-tracker.log 2>&1 &
nginx -s reload                         # only if nginx config changed
./deploy/health-check.sh
```

Git

```
git status                             # clean? on main?
git diff                               # review before committing
git add -A && git commit -m "feat(...): ..."
git push
git status                             # "main...origin/main" = synchronized
```

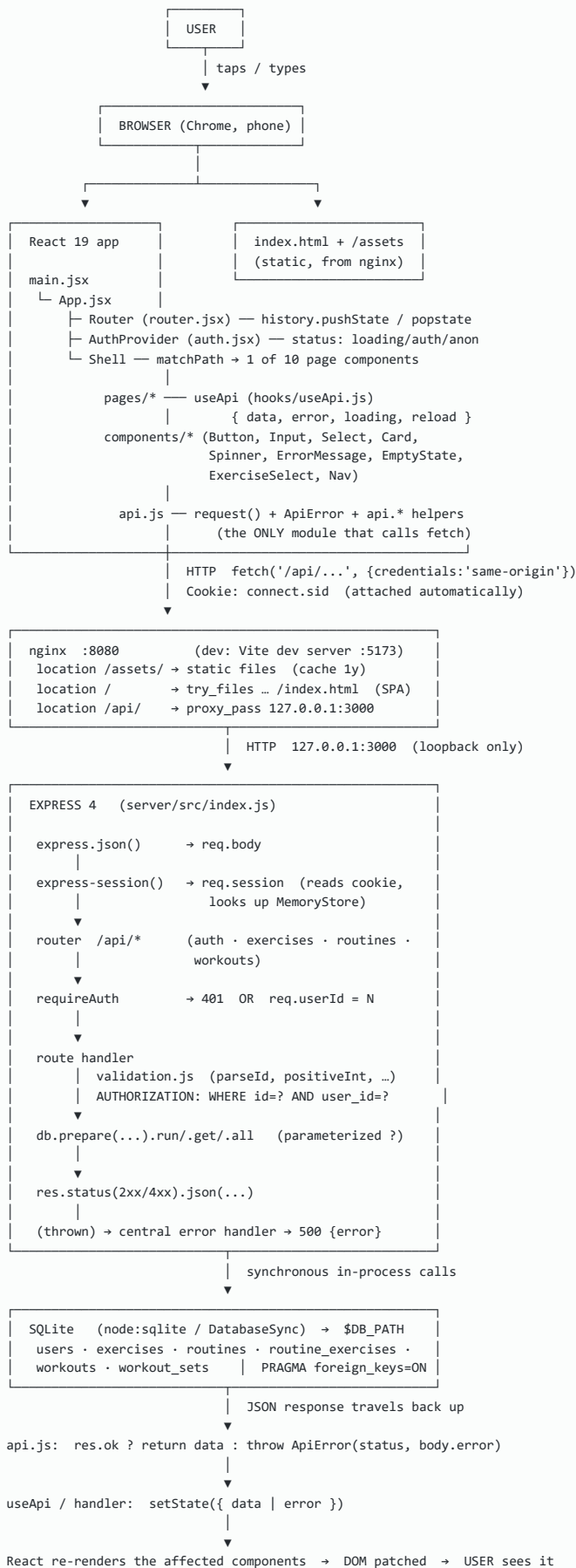
Appendix D — Environment variables

Read directly from `process.env` (no `dotenv` library). `deploy/start.sh` sources `server/.env` and exports them. `server/.env` is **gitignored**; `server/.env.example` is the committed template.

Variable	Default (if unset)	Production requirement	Effect
<code>PORT</code>	<code>3000</code>	any free port ≥ 1024	the port Express <code>listen</code> s on; nginx <code>proxy_pass</code> must match
<code>HOST</code>	<code>127.0.0.1</code>	leave as <code>127.0.0.1</code>	bind address. Loopback = unreachable from the network (nginx is the entry point). <code>0.0.0.0</code> exposes it on the LAN — only for debugging
<code>DB_PATH</code>	<code><repo>/server/db... server/data/app.db</code>	an absolute path outside the git clone (phone: <code>/data/data/com.termux/files/home/gym-tracker-data/app.db</code>)	the SQLite file. <code>db.js</code> creates the parent directory. Keeping it outside the clone means <code>git pull</code> / <code>re-clone</code> never touch the data
<code>SESSION_SECRET</code>	<code>'dev-only-secret-change-me'</code> (insecure)	a long random string — e.g. <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>	signs the session cookie so the session id can't be forged. Never commit the real value.

A real secret value never appears in this document, the repository, or any command output.

Appendix E — Complete project map



End of guide. Generated from github.com/Sumit-0610/gym-tracker at commit [0d2eb73](#). The source tree is authoritative; where a repo Markdown file disagreed with the code, the discrepancy is noted in the relevant chapter (see §17.9, §25.4 note, §33.3).